Technical Reference

OpenMX 4.0 GPU Version Implementation Guide

CUDA implementation details and AMD HIP integration
with a shared CPU/GPU ELPA library
— GPU libraries, CMake builds, runtime controls, and validation —

Integration revision: October 17, 2026, commit `c0689442`
Original CUDA survey: July 23, 2026, commit `9fe8ce13`
Backends: NVIDIA CUDA / AMD HIP; ELPA 2026.02.002
Revised: October 17, 2026 (4th edition)

> **Note:** This document describes an **unofficial** port of the first-principles code OpenMX (GPL v3) to NVIDIA and AMD GPUs; it is not an official document of the OpenMX development team. Detailed CUDA listings and historical change counts retain the original survey baseline. Current integration, build, and validation information is in § § 9.4, 9.7–9.8 and 11.5.

Table of Contents

1. Introduction

2. Overall Architecture

Design policy / Layer structure / Enabling and dispatch / Map of GPU-offloaded targets / Handling a GPU shared by multiple ranks

3. How cuBLAS Is Used

Routines used / Handles and streams / Memory management / The actual GEMM engine per solver / Error handling / Caveats

4. How GEMMul8 Is Used

Principle and origin / Integration architecture / Precision parameters / Workspace management / Complex support / Build / Caveats

5. How cuSOLVER (the gpusolver Layer) Is Used

Layer structure and naming intent / APIs used / Generalized eigenvalue problem / devInfo handling / Adaptive concurrency control / ELPA fallback / Device assignment / Caveats

6. OpenACC versus Raw CUDA

Four patterns of data residency / host_data interoperation / set_hamiltonian_gpu.cu / NVHPC-specific caveats

7. GPU Implementation Details by Computation Path

Band / Cluster / DC / DC-LNO / Krylov / Matrix elements / VNA projectors / Density grid / Total energy / Forces / Mixing schemes / Derived solvers

8. Changes Unrelated to GPU Offloading

OpenMP bug fixes / More robust memory management / Other fixes

9. Build System

CUDA toolchain / Link configuration / Shared ELPA / Native Makefiles / CUDA and HIP with CMake / Regression checks

10. Runtime Control Reference

Input keywords / Environment variable list / GPU diagnostics on stdout / Tuning guidelines

11. Summary of Constraints and Caveats

Appendix A. Changed Files (July 2026; 47 files)

Appendix B. Newly Added Files (July 2026)

Appendix C. Development History (July 2026; 91 commits)

References

1. Introduction

1.1 Purpose and Audience

This document describes the "OpenMX 4.0 GPU version", a port of the density-functional-theory (DFT) first-principles code **OpenMX 4.0** to NVIDIA CUDA and AMD HIP. Based on the repository sources, it describes the CUDA implementation in detail, the integrated HIP backend, the shared CPU/GPU ELPA library, and the build and runtime interfaces. Chapters 3–7 retain the CUDA-specific API discussion; § 11.5 explains the HIP implementation and its differences. Intended readers:

- Users who want to run OpenMX fast on GPUs (mainly Chapters 2, 10, and 11)
- Developers who want to understand or modify this implementation (Chapters 3–9)
- Developers maintaining the CUDA and HIP backends (§ § 9.7 and 11.5)

Basic knowledge of DFT and the LCAO method, C, MPI, and CUDA programming is assumed.

1.2 OpenMX and the Position of This Port

OpenMX (Open source package for Material eXplorer) is a DFT code based on localized pseudo-atomic orbitals (LCAO) and norm-conserving pseudopotentials. Besides cluster and band calculations it offers O(N) methods such as the divide-and-conquer (DC) method, DC-LNO, and the Krylov subspace method, and non-equilibrium Green's function (NEGF) transport. It is licensed under GPL v3.

This GPU version starts from the official OpenMX 4.0 sources. The original July 2026 survey covered **91 commits**. The early development (up to the 51 commits covered by the 2nd edition) offloaded the dense-matrix eigenvalue problems and matrix products (GEMM) that dominate SCF; the following 40 commits broadened the scope substantially. Today, matrix-element construction (the Set_Hamiltonian grid integrals, Set_Nonlocal, and Set_ProExpn_VNA), density-grid construction (Set_Density_Grid), the major force traces (#2/#3/#4/#4B/#5/HNL), the total-energy grid reductions, and charge/DM mixing (a GPU path for RMM-DIISH plus fast paths for every other scheme) all run on the GPU or on equivalent fast paths. Sparse-matrix handling, communication, and I/O keep the original structure.

1.3 Scale of the Changes

The following counts and Appendices A–C describe the July 23 CUDA survey, not the current tree. The subsequent integration adds the HIP backend, isolated CMake builds, and shared modern ELPA.

Item	Value	Notes
Modified source files	47 files (plus the `makefile` → `Makefile` rename and full rewrite)	Two sources were deleted: `my_cublasDgemm.c` / `my_cublasZgemm.c` (dormant wrappers)
Changed lines (raw diff)	about +47,000 / −21,200	Includes whole-file re-indentation (`Force.c` etc.); `Force.c` alone is +15,123/−11,241
New source files	14 files, about 4,000 lines	gpusolver wrappers, the GEMMul8 bridge, GPU kernels for the total energy, the density grid, and Set_Hamiltonian, etc.
New build files	3 files	`Makefile` / `Makefile.bunshiken` / `makefile.cpu_intel.bak` (byte-identical backup of the original)
third_party (git submodules)	2	GEMMul8 (pinned commit) and FFTW 3.3.11 (built in-tree); both are fetched and built automatically by `make`

1.4 Target Environment

- **CUDA:** NVIDIA HPC SDK (nvc/OpenACC, nvfortran, nvcc). The current main Makefile defaults to NVHPC 26.9 and CUDA 13.4; paths can be overridden. The original survey used NVHPC 26.3 / CUDA 13.1.
- **HIP:** ROCm clang/clang++, hipcc, flang, OpenMP target offload, compatible MPI, and AOCL. MI300A and Radeon have device-specific policies (§ 11.5).
- **Parallel model:** MPI process parallelism. GPUs are assigned round-robin by intra-node local rank, and the design explicitly assumes **several ranks sharing one GPU**. Sharing is detected per physical GPU via its UUID, and concurrency is tuned automatically from VRAM estimates and real reservations (§ 2.5, § 5.5)
- **OpenMP:** MPI/OpenMP hybrid runs are supported. Bind each rank to the cores needed by `-nt`. Some CUDA paths, including optional radial-integral offload in `Set_OLP_Kin`, require one thread (§ 11.1).

1.5 Notation

- Source locations are written as `file:line` (relative to `source/`, as of 2026-07-23).
- `GPU` runs on the GPU by default; `conditional` opt-in via environment variables or dependent on memory conditions; `CPU` intentionally kept on the CPU; `dormant` implemented but not called on any current path.
- "Original" refers to the official OpenMX 4.0 source distribution.

> **How to read this document:** if you only need to know how the libraries are used, read Chapters 3–5; runtime parameters are in Chapter 10 and known pitfalls in Chapter 11. Chapter 7 is a per-computation-path (per-solver) walkthrough, cross-referenced with the per-library Chapters 3–6. The main differences from the 2nd edition (2026-07-05) are the GPU offloading of matrix-element construction, the density grid, forces, and mixing (§ 7.7–7.12), the adaptive concurrency control and ELPA fallback on all solver paths (§ 5.5–5.6), and the inter-phase GPU memory coordination (§ 2.5).

2. Overall Architecture

2.1 Design Policy

1. **Runtime switching with the CPU paths fully preserved** — GPU offloading is implemented as **added branches** in the existing code, not as a wholesale `#ifdef` replacement. It is enabled by default (`scf.eigen.lib` defaults to `gpusolver`); a single input line, `scf.eigen.lib elpa2` (or `elpa1` / `lapack`), runs the original ELPA/ScaLAPACK/LAPACK paths unchanged. The gate is concentrated in one runtime flag, `scf_eigen_lib_flag == GPUSOLVER` (enum value 3).
2. **Only large dense problems go to the GPU** — below a threshold dimension the transfer cost dominates, so the code falls back to the CPU. Each dense path carries its own measured default: band Col/NonCol 800 (vs n / 2n — the band CPU fallback solves every k point twice with a serial per-world ELPA, so the GPU pays off early), cluster Col/NonCol 400 (the one-rank root-dense solve beat the multi-rank ELPA at every measured size), DC Col 400 / NonCol 300 (vs NUM / 2·NUM of the truncated cluster), DC-LNO Col 800 / NonCol 400 (LNO-contracted dimensions; the collinear proxy overhead demands a higher bar), Krylov 300 (vs the projected dimension Msize3); the remaining paths keep the global `GPU_CPU_SWITCH_NUM = 1000` (`openmx_common.h:4135`). Each threshold is tunable with its own knob (§ 10).
3. **Per-MPI-rank GPU use, designed for sharing** — round-robin assignment by intra-node local rank. The situation where several ranks share one GPU is addressed head-on: **device-sharing rank groups are detected via the physical GPU UUID**, and the code layers adaptive concurrency control based on VRAM estimates/real reservations, runtime ELPA fallback, allocation retries, and a registry through which GPU phases publish their memory needs (§ 2.5).
4. **Device-aware GEMM selection** — GeForce defaults to GEMMul8 FP64 emulation on INT8 Tensor Cores (Ozaki Scheme II); A100/H100 generally select native cuBLAS FP64, with a size-dependent A100 cluster forward-transform exception. Explicit settings override the device policy, and unavailable workspace can trigger native FP64 fallback. HIP has its own MI300A/Radeon policy (§ 11.5).
5. **Diagonalization via cuSOLVER's 64-bit generic API** — mainly `cusolverDnXsyevdx` (partial eigenvalues). The generalized problem $Hc = \varepsilon Sc$ is handled by Löwdin (symmetric) orthogonalization via the eigendecomposition of S, implemented on the GPU (Chapter 5). Where only eigenvalues are needed, the solver runs with `jobz=NOVECTOR`.
6. **Vendor-neutral naming (gpusolver)** — the wrapper-layer identifiers and input keyword use `gpusolver` in both builds. CMake selects CUDA or HIP sources at build time (§ 9.7). cuSOLVER remains a CUDA implementation detail; HIP uses hipSOLVER and the compatibility helpers (§ 11.5). MAGMA is now used for multi-GPU capacity recovery and forced `gpusolver3` execution; it is optional in the CUDA build and built as a dependency of the HIP backend.
7. **Care for numerical reproducibility** — the GPU paths are written to preserve the CPU loop's summation order wherever it matters: density-matrix and grid-trace reductions use `#pragma acc loop seq` or element-wise sequential accumulation; the force batches reproduce the CPU code down to the position of the float-multiply→double-add cast; the mixing fast paths guarantee bitwise equivalence via pointer rotation. GEMMul8 itself is bitwise reproducible. (Some scatter-adds use atomics and are order-nondeterministic; see Chapter 11.)
8. **Fallbacks are quiet and correct** — when VRAM is insufficient, each phase decides via an estimate or a real reservation and falls back to the conventional CPU/ELPA path. Notices are limited to once per node (or only

when a value changes), and correctness is unaffected. Aborts are reserved for unrecoverable errors and for contradictions with an explicit force flag.

2.2 Layer Structure

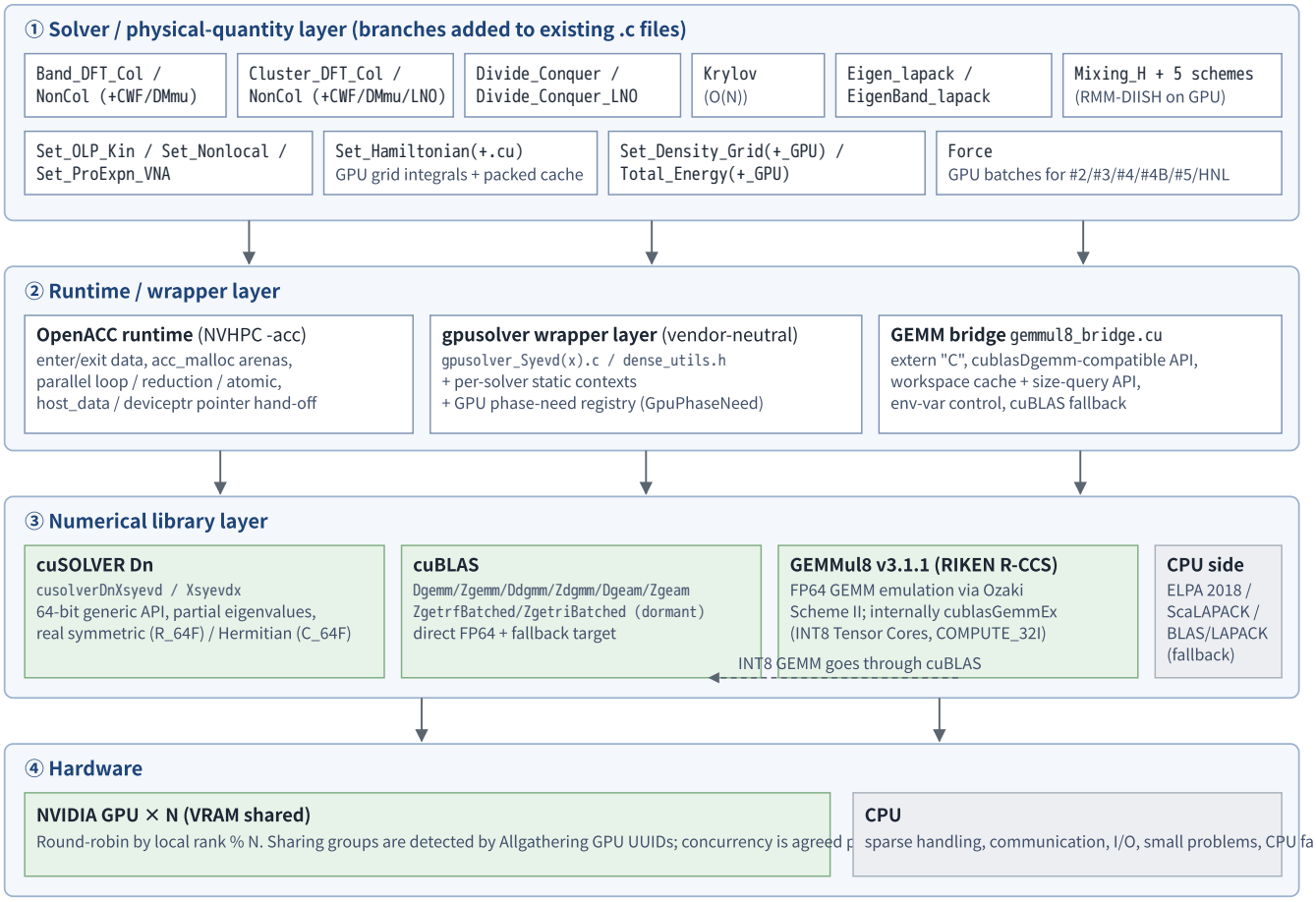


Figure 1: Layer structure of the OpenMX 4.0 GPU version. Green = GPU execution, gray = CPU.

Three forms of GPU execution coexist:

1. **OpenACC directives** — loop parallelism for grid integrals, reductions, and matrix assembly. Besides `enter data` / `exit data`, the forces, mixing, and cluster paths now favor **single-allocation arenas** (`acc_malloc` / `cudaMalloc`) exposed through `deviceptr` / `acc_map_data` (Chapter 6).
2. **Per-solver static contexts + raw CUDA API** — persistent device buffers from `cudaMalloc` and cuBLAS/cuSOLVER handles held in file-scope structs, calling the libraries directly (`BandColGpuSolverCtx` etc.; Chapters 3 and 5).
3. **CUDA C++ behind extern "C" bridges** — GEMMul8 (Chapter 4) and the shared-memory-tiled matrix-element kernel `set_hamiltonian_gpu.cu` for Set_Hamiltonian (§ 7.7); a single-translation-unit scheme for calling C++/CUDA from C.

2.3 Enabling Flow and GPU Points in the SCF Loop

The GPU paths become active as follows.

1. **Input parsing** (`Input_std.c`): `scf.eigen.lib gpusolver` — the default when the keyword is absent — sets `scf_eigen_lib_flag = GPUSOLVER`. The old name `cusolver` is accepted as a deprecated alias with a warning; the

user's choice is saved in `scf_eigen_lib_flag_input`.

2. **Device initialization** (`DFT_GPU_DeviceInit()` in `DFT.c`): the intra-node local rank is obtained with `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` and `cudaSetDevice` and `acc_set_device_num` are set **as a pair**. If no CUDA/OpenACC device is found, a warning is printed and the run demotes itself to `scf_eigen_lib_flag = ELPA2`.
3. **Per-phase entry checks**: a phase enters its GPU path only when `GPUSOLVER flag && size condition && memory condition (estimate or real reservation)` holds; otherwise the conventional path (ELPA2/CPU) runs. Verdicts are made collective (`Allreduce(MIN)`) over the device-sharing group or all of `mpi_comm_level1` so every rank takes the same side.

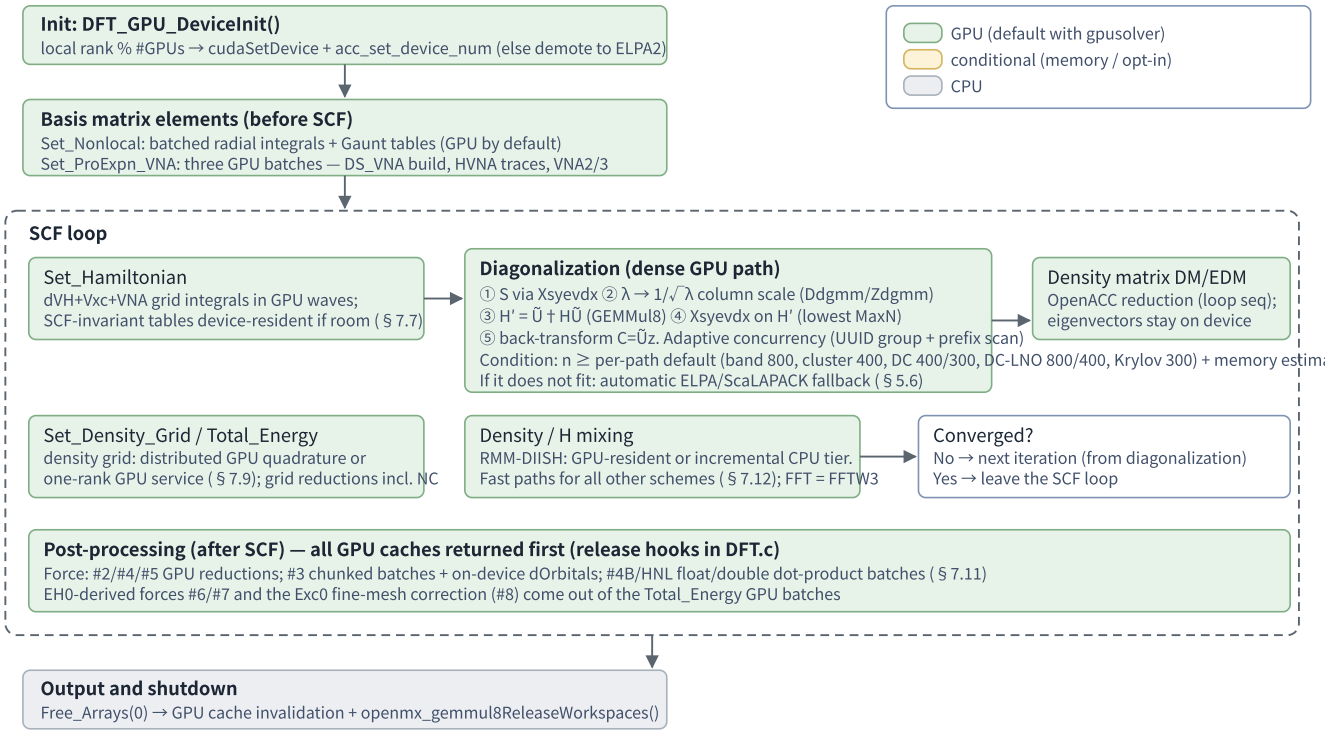


Figure 2: One SCF cycle and its GPU points (collinear band/cluster case).

2.4 Map of GPU-Offloaded Targets

Table 2-1 shows the GPU status per computation path (`scf.EigenvalueSolver`) and per auxiliary computation. Details are in Chapter 7.

Path / computation	Main file	What runs on the GPU	Trigger conditions
Cluster (Col)	`Cluster_DFT_Col.c`	Full root-dense diagonalization (S eigendecomposition → Löwdin → H′ → back-transform) plus GPU accumulation of DM/EDM; eigenvectors stay device-resident into the DM step. When both spin owners cannot fit, spin world 0's owner solves both spins serially	GPUSOLVER and n ≥ 400 (`OPENMX_CLUSTER_GPU_SWITCH_NUM`) and a **successful real reservation** (else ELPA)
Cluster (NonCol)	`Cluster_DFT_NonCol.c`	Root-dense diagonalization (Dsyevx/Zheevx) on a single cudaMalloc arena + U † HU (GEMMul8 × 12) + DM/EDM reductions. S/Ss2, sparse→dense index, cublas handle, and zheevdx workspace cached across iterations	GPUSOLVER and 2n ≥ 400 (`OPENMX_CLUSTER_NONCOL_GPU_SWITCH_NUM`) and the arena reservation succeeds (else ELPA)

Path / computation	Main file	What runs on the GPU	Trigger conditions
Band (Col)	`Band_DFT_Col.c`	Whole per-k-point dense pipeline device-resident. GEMMs via GEMMul8 Zgemm, S-transform cached, NOVECTOR diagonalization on the first pass. Adaptive concurrency (UUID group + descending prefix scan)	GPUSOLVER and n ≥ 800 (`OPENMX_BAND_GPU_SWITCH_NUM`) and one turn's VRAM estimate fits (else ELPA)
Band (NonCol)	`Band_DFT_NonCol.c`	OpenACC-resident root-dense path; concurrency decided separately for the eigenvalues-only / eigenvectors / DM stages	GPUSOLVER and 2n ≥ 800 (`OPENMX_BAND_NONCOL_GPU_SWITCH_NUM`) and the estimate fits (else ELPA)
DC (O(N))	`Divide_Conquer.c`	Per-atom cluster matrices via Xsyevdx; GEMMul8→cuBLAS→CPU multi-stage fallback; device-resident S12 cache (S-cache) + cublasDgeam device transposes	GPUSOLVER; collinear NUM ≥ 400 (`OPENMX_DC_GPU_THRESHOLD`), noncollinear 2·NUM ≥ 300 (`OPENMX_DC_NONCOL_GPU_THRESHOLD`)
DC-LNO	`Divide_Conquer_LNO.c`	Direct per-rank dispatch; **noncollinear solves now on GPU too** (spin-doubling transform + GEMMul8 Zgemm × 3). Group-wide memory-fit test with CPU fallback	GPUSOLVER; collinear ≥ 800 (`OPENMX_DCLNO_GPU_THRESHOLD`), noncollinear spinor dimension ≥ 400 (`OPENMX_DCLNO_NONCOL_GPU_THRESHOLD`)
Krylov (O(N))	`Krylov.c`	Per-atom projected-solve pipeline fused into one device visit (3 GEMMs + Xsyevd); device-resident KU basis cache	GPUSOLVER and all three GEMMs with $m \cdot n \cdot k \geq 5 \times 10^7$ (device eigensolve at Msize3 ≥ 300)
Generic eigensolvers	`Eigen_lapack.c` `EigenBand_lapack.c`	Branch to gpusolver_Syevdx(_Complex) at n ≥ 1000; every one of their 20+ callers benefits automatically	GPUSOLVER and n ≥ 1000
Polarization / DMmu / CWF / LNO	11 files	A stereotyped GPUSOLVER branch (Dsyevx/Zheevx via dense_utils) inserted just before the ELPA2 branch	GPUSOLVER and n ≥ 1000 and na_rows==n etc.
Matrix-element construction	`Set_Hamiltonian.c` `set_hamiltonian_gpu.cu` `Set_Nonlocal.c` `Set_OLP_Kin.c`	The dVH+Vxc+VNA grid integrals of Set_Hamiltonian run **on the GPU by default** (shared-memory-tiled CUDA kernel + adaptive wave scheduling + device-resident SCF-invariant tables). Set_Nonlocal: device-generated spherical Bessel tables + batched reductions. Set_OLP_Kin remains opt-in	GPUSOLVER (+ memory tests, § 7.7)
VNA projectors	`Set_ProExpn_VNA.c`	Three GPU batches — DS_VNA construction, HVNA traces, VNA2/VNA3 (device Bessel recurrences; Gaunt/transform tables precomputed on the host)	GPUSOLVER (+ per-batch memory tests)
Density grid	`Set_Density_Grid.c` `Set_Density_Grid_GPU.c`	Three-stage fallback chain: distributed GPU quadrature (each rank integrates its own atoms) → one-rank GPU service (owner builds the whole grid and scatters) → CPU	GPUSOLVER and Cluster/Band and Cnt_switch==0
Total energy	`Total_Energy.c` `Total_Energy_GPU.c`	Grid reductions for EXC/EH1, dipole, CWF-Dc, the EH0 two-center batch, and the Exc0 fine-mesh correction — **now including the noncollinear case**	GPUSOLVER (`OPENMX_TOTALENERGY_GPU=0` disables)
Forces	`Force.c`	GPU reductions for #2/#4/#5; chunked batches for #3 (with on-device dOrbitals); **GPU dot-product batches for #4B and HNL** (#4B was CPU-only in the 2nd edition). Arena + peer-wait machinery makes the batches immune to shared-device memory races	GPUSOLVER (+ per-stage memory tests, § 7.11)

Path / computation	Main file	What runs on the GPU	Trigger conditions
Charge/DM mixing	`Mixing_H.c` + 5 files	RMM-DIISH: two tiers (GPU-resident Gram / incremental CPU). RMM-DIIS/DIISK/DIISV/GR-Pulay/Kerker: CPU fast paths (batched Gram, incremental Gram, pointer-rotated history shifts)	GPUSOLVER (`OPENMX_MIX_FAST=0` disables)
NEGF complex inverse	`Lapack_LU_inverse.c`	dormant GPU LU inverse via cublasZgetrf/ZgetriBatched implemented but its call is commented out	— (groundwork)

Table 2-1: Map of GPU-offloaded targets

2.5 Handling a GPU Shared by Multiple Ranks (Overview)

A defining feature of this implementation is its layered defense for configurations like "one 16 GB GPU shared by 8–36 MPI ranks". Since the 2nd edition this defense has evolved from fixed serialization knobs to **measurement-driven adaptive control plus runtime fallbacks**.

Mechanism	Description	Where
Detecting device-sharing groups	`MPI_Comm_split_type(SHARED)` plus an Allgather of the **GPU UUID** from `cudaGetDeviceProperties` builds a communicator of ranks sharing the same physical GPU. Free VRAM is measured by everyone after returning the OpenACC pools (`acc_clear_freelists()`) and shared via MIN	`Band_DFT_Col.c` , `Band_DFT_NonCol.c` , `Set_Hamiltonian.c` , `Mixing_H.c`
Adaptive concurrency (prefix scan)	Each rank's VRAM requirement is Allgathered and sorted in descending order; a single scan picks the largest c such that the top-c sum plus the reserve fits in the free VRAM. If everyone fits, barriers are folded away; if not, execution splits into waves (turns). The initial wave equals the device-sharing rank count (the old fixed ladder 32→…→1 is gone)	`BandCol_AutoGpuTurnLimit` , `BandNonCol_AutoGpuTurnLimit` , `Set_Hamiltonian_CreateGpuTurnPlan`
Real-reservation probes + ELPA fallback	The cluster paths decide by **actually cudaMalloc-ing** the buffers (bounded 8-try retries, full release on failure); the band paths decide from a one-turn estimate. Either way, if it does not fit, the run falls back to ELPA/ScaLAPACK, with the verdict made collective via MPI_Allreduce(MIN)	`ClusterCol_GpuDiagFits` , `ClusterNonCol_GpuDiagFits` , `BandCol_GpuDenseFits` , `BandNonCol_GpuDiagFits` (§ 5.6)
GPU phase-need registry	The diagonalizers and the density grid publish "total bytes needed on this device at full concurrency" via `OpenMX_GpuPhaseNeed_Register()` ; Set_Hamiltonian's resident cache admits itself only after subtracting those needs, and evicts itself when they grow (inter-phase memory coordination)	registry in `openmx_common.c` ; consumer in `Set_Hamiltonian.c` (§ 7.7)
Force-batch arenas + peer waiting	All force-stage allocations are packed into single 512-B-aligned arenas; mandatory allocations wait up to 180 s in 50 ms steps for peers to release memory, optional ones (reduction buffers) fall back to host computation on the spot. **No force-stage device allocation can abort the run**	`Force_gpu_arena_wait/try` (§ 7.11)
Bulk release between SCF phases	Just before each force phase, the Set_Hamiltonian resident tables, DC S-cache, Krylov KU cache, cluster solver contexts, and mixing history arenas are all released so the force batches measure the true free VRAM	release hooks at `DFT.c:2312-2319`
Allocation retry	The `wait_cudafunc` macro retries CUDA/cuBLAS/cuSOLVER calls until success with a random ≤10 µs backoff, popping the CUDA error latch after each failure. Allocations that can fail permanently use the bounded `TryDeviceMalloc` helpers instead	`openmx_common.h:4148-4162`

Mechanism	Description	Where
Per-turn workspace release	The ~1 GiB GEMMul8 workspaces are returned at the end of each turn/solve (Band, Cluster, DC-LNO; on by default)	`OPENMX_*_GEMMUL8_TURN_RELEASE`

> **Dormant code:** throughout this document, code that exists but is never called on any current path (`gpusolver_Syevd`, `LU_Zinverse_GPU`, some GEMMul8 helpers) is marked `dormant`; see §11.7 for the list. The `openmx_cuda_kernels.cu` collection described as dormant in the 2nd edition has been **removed from the tree**; the wired-in `set_hamiltonian_gpu.cu` took its place.

3. How cuBLAS Is Used

3.1 Routines and Their Roles

cuBLAS plays three roles here: (1) direct execution of non-GEMM dense operations such as diagonal scaling (`Ddgmm` / `Zdgmm`) and on-device transposes (`Dgeam` / `Zgeam`); (2) the FP64 fallback target when GEMMul8 cannot be used; (3) the engine underneath GEMMul8's INT8 Tensor Core GEMMs (`cublasGemmEx`). The direct `cublasDgemm/Zgemm/Dsymm` calls that remained in some solvers were **all unified onto the GEMMul8 bridge** by commit `66ce1e62` (Use GEMMul8 for remaining GPU GEMM paths). Table 3-1 lists the routines in use.

cuBLAS routine	Purpose	Main call sites
`cublasDgemm` / `cublasZgemm`	FP64 fallback inside the GEMMul8 bridge, and stage 2 of the DC multi-stage fallback (the only two direct call sites)	`gemmul8_bridge.cu`, `Divide_Conquer.c`
`cublasDdgmm` / `cublasZdgmm`	$1/\sqrt{\lambda}$ column scaling of the Löwdin orthogonalization. In noncollinear DC-LNO also builds the upper-left block of the spin-doubling transform $T = \mathrm{diag}(U \cdot s^{-1/2}, U \cdot s^{-1/2})$	`Cluster_DFT_Col.c`, `Band_DFT_Col.c`, `Divide_Conquer_LNO.c`
`cublasDgeam` / `cublasZgeam`	On-device transposes (pure data movement, so downstream inputs are bit-identical). DC replaced its $O(n^2)$ strided host transposes with `DC_GpuSolver_DeviceTranspose` (`Divide_Conquer.c:1175`); noncollinear DC-LNO uses Zgeam to form the conjugate of H′	`Divide_Conquer.c`, `Divide_Conquer_LNO.c`
`cublasZgetrfBatched` / `cublasZgetriBatched`	**dormant** complex LU factorization/inverse (batch=1); groundwork for NEGF surface Green's functions (§ 5.8)	`Lapack_LU_inverse.c:384,400`
`cublasGemmEx` (INT8, COMPUTE_32I)	The per-modulus INT8 GEMM inside GEMMul8 (never called directly by OpenMX)	`third_party/GEMMul8/src/oz2/common/matmult.hpp`

Table 3-1: cuBLAS routines in use

3.2 Handle and CUDA Stream Lifecycles

On the main paths, a **file-scope static context per solver** lazily creates and holds one cuBLAS/cuSOLVER handle each (one per solver type per MPI process), destroyed and recreated only when the device changes. Places that used to create/destroy per iteration are now cached too (e.g. the noncollinear cluster path replaced its 13 handle creations per iteration with the per-device cache `ClusterNonCol_CublasHandle()`, `Cluster_DFT_NonCol.c:762`; the per-atom diagonalizations of noncollinear DC and DC-LNO go through `gpusolver_Syevdx_Complex_openacc_cached`, which keeps handle, stream, and workspace; § 5.2).

Cluster_DFT_Col.c — lazy context initialization (typical form)

```
wait_cudafunc(cudaGetDevice(&current_device));

if (ctx->initialized && ctx->device_id == current_device) return;
if (ctx->initialized) ClusterCol_GpuSolver_Destroy();

wait_cudafunc(cudaStreamCreateWithFlags(&ctx->stream, cudaStreamNonBlocking));
wait_cudafunc(cublasCreate(&ctx->cublas));
wait_cudafunc(cusolverDnCreate(&ctx->gpusolver));
wait_cudafunc(cublasSetStream(ctx->cublas, ctx->stream));
wait_cudafunc(cusolverDnSetStream(ctx->gpusolver, ctx->stream));

ctx->initialized = 1;
ctx->device_id   = current_device;
```

Stream usage is **asymmetric** across solvers:

- **Cluster_DFT_Col / Cluster_DFT_NonCol / Band_DFT_NonCol (for DM) / Krylov / DC / DC-LNO:** create a dedicated non-blocking stream, bind it with `cublasSetStream` / `cusolverDnSetStream`, and order transfers with `cudaMemcpyAsync` plus `cudaStreamSynchronize` at key points.
- **Band_DFT_Col:** its solver context has no stream member and never calls `cublasSetStream`; cuBLAS runs on the default stream and transfers are synchronous `cudaMemcpy` (only the syevdx workspace-size probe uses a temporary stream).
- **Ordering against OpenACC:** the OpenACC default queue and the library streams are not connected (`acc_get_cuda_stream` / `async` clauses are unused everywhere). The code places `#pragma acc wait` before handing OpenACC-managed data to a library and `cudaStreamSynchronize` after.

> **Caveat:** because stream assumptions differ between solvers, mixing synchronization conventions when porting a helper from one solver to another risks races. Handles are **never explicitly destroyed at normal program exit**, but since commit `ae3dd189` the device-resident matrix buffers, workspaces, and caches are **returned in bulk right before each SCF cycle's force phase** (release hooks at `DFT.c:2312-2319`: `Set_Hamiltonian_Release_OpenACC_DeviceCache` / `Divide_Conquer_Release_GPU_SCache` / `Krylov_Release_GPU_KUCache` / `Cluster_DFT_Col_Release_GPU_Solver` / `Cluster_DFT_NonCol_Release_GPU_Solver` / `Mixing_H_Release_GPU`), so the force batches can measure the true free VRAM.

3.3 Device Memory Management

- **Grow-only persistent buffers:** the pattern `if (n <= ctx->matrix_dim) return;` — reuse when the requirement fits, free-then-malloc only when it grows. Same for cuSOLVER workspaces.
- **Bounded retries:** allocations that are optional, or that have a CPU/ELPA fallback ready, use the **8-attempt `TryDeviceMalloc` helpers** (`ClusterCol_TryDeviceMalloc`, `ClusterNonCol_TryDeviceMalloc`, `MixH_TryDeviceMalloc`, …) instead of the infinite `wait_cudafunc` retry, so a permanently full device produces a fallback rather than a silent freeze (design introduced with commit `ae3dd189`; § 5.6).
- **Single-cudaMalloc arenas:** the noncollinear cluster path reserves all buffers of a diagonalization cycle in one `cudaMalloc` (512-byte-aligned partitions) and exposes them to OpenACC via `acc_map_data` (`ClusterNonCol_DenseArena_TryEnsure`, `Cluster_DFT_NonCol.c:1613`). Once reserved, the GO verdict cannot be invalidated by other phases filling the device mid-cycle. The DC S-cache, the Krylov KU cache, and the Mixing_H history ring use the same arena pattern.
- **Pinned memory:** almost unused; only DC uses `cudaMallocHost` on the host side.

- **Shared-GPU considerations:** band-path buffer persistence across SCF iterations stays off by default (`OPENMX_BAND_GPU_PERSISTENT`); a source comment notes that with 16 GB shared by 8 ranks it pushes GEMMul8 into FP64 fallback and costs more than the 0.2 s/iteration it saves. The Set_Hamiltonian SCF-invariant tables, by contrast, become resident **only when they fit** after accounting for published phase needs (§ 7.7).

3.4 The Actual GEMM Engine per Solver

Since commit `66ce1e62`, **every GEMM executed on the GPU goes through the GEMMul8 bridge** (cuBLAS FP64 appears only as the bridge-internal fallback and as stage 2 of the DC fallback). Because **several helpers exist that are defined but never called**, Table 3-2 records the live paths.

Solver	GEMM engine on the live path	Notes
Band_DFT_Col	**GEMMul8** (`openmx_gemmul8Zgemm` × 3: the congruence transform and the back-transform)	The first pass (all_knum≠1) needs eigenvalues only, so it diagonalizes with `jobz=NOVECTOR` and skips the back-transform GEMM (§ 7.1)
Band_DFT_NonCol	**GEMMul8** (via `BandNonCol_GEMMul8Zgemm_OpenACC` × 6)	Uses the persistent workspace's cublas handle + stream
Cluster_DFT_Col	**GEMMul8** (`ClusterCol_GEMMul8Dgemm_Device`; H′ build × 2 + back-transform × 1)	DM/EDM accumulation is not a GEMM but an OpenACC `deviceptr` kernel (§ 7.3)
Cluster_DFT_NonCol	**GEMMul8** (12 D/Z GEMMs of the U † HU transform + back-transform)	The back-transform uses the explicit-leading-dimension helper `ClusterNonCol_GEMMul8ZgemmLd_OpenACC` with m=MaxN (`f5de630e`)
Divide_Conquer (DC)	**GEMMul8 → cuBLAS → CPU** multi-stage (`DC_GpuSolver_TryGpuDgemm`)	A GEMMul8 failure permanently disables GEMMul8 on that rank; preflight skipped for shapes ≤ an approved shape (`6f7f31b2`)
Divide_Conquer_LNO (Col)	**GEMMul8** (`openmx_gemmul8Dgemm` × 3)	—
Divide_Conquer_LNO (NonCol)	**GEMMul8** (`openmx_gemmul8Zgemm` × 3: H·T, T † (HT), back-transform)	New in `d62cac60`; the spin-doubling transform T is built with Zdgmm + a `cudaMemcpy2DAsync` block copy (§ 7.5)
Krylov	**GEMMul8** (`openmx_gemmul8Dgemm`; in the fused path three GEMMs per atom: H·u1, u1 † Hu1, back-transform)	The fused path engages only when all three GEMMs have $m \cdot n \cdot k \geq 5 \times 10^7$ (`3ee45228`, § 7.6)
Mixing_H (RMM-DIISH)	No GEMM (OpenACC Gram / optimal-residual kernels)	The only GPU-using mixing is RMM-DIISH Tier A (§ 7.12)
DMmu/CWF/LNO (11 files)	GPU for diagonalization only (GEMMs stay on the CPU)	Their GEMMul8 helpers are defined but unused (groundwork)

Table 3-2: GEMM engine per solver (live paths vs. dormant code)

3.5 Removed Generic Wrappers and Simplified-Helper Caveats

The self-contained GEMM wrappers of early development, `my_cublasDgemm.c` / `my_cublasZgemm.c` (per-call `cublasCreate` → `cudaMalloc` / `cudaMemcpy` → GEMM → destroy), were dormant for a long time and were then **completely removed** — sources, declarations, and link targets — by commit `a7f8cba6`. The names appear only in old revisions.

A related simplification persists in the current helpers: most in-solver GEMM helpers hard-code **alpha=1.0 / beta=0.0**, and there are two lineages of lda/ldb handling. Most live helpers compute lda/ldb according to the transpose flags, and the noncollinear cluster gained an explicit-leading-dimension variant

(`ClusterNonCol_GEMMul8ZgemmLd_OpenACC`, `Cluster_DFT_NonCol.c:993`), but older square-only helpers and the dormant DMmu/CWF helpers still hard-code **lda=m, ldb=k**, which is correct under transposition only for square matrices (m=n=k). Check the ld handling before reusing them with non-square or transposed inputs.

3.6 Error Handling — the wait_cudafunc Macro

Most CUDA/cuBLAS/cuSOLVER calls are wrapped in this macro:

openmx_common.h:4135-4162 (excerpt)

```
#define GPU_CPU_SWITCH_NUM 1000
#define WAITTIME           (10.0 * 1.0E-6)

#define wait_cudafunc(call)                                                 \
    while (true) {                                                          \
        if (cudaSuccess == call) {                                          \
            break;                                                          \
        }                                                                   \
        /* pop the CUDA error latched by the failed attempt so it cannot be    \
           misattributed by the next cudaGetLastError() caller once a retry    \
           succeeds */                                                      \
        (void)cudaGetLastError();                                           \
        double wait_time    = drand48() * WAITTIME;                         \
        double start_time   = MPI_Wtime();                                  \
        double current_time = start_time;                                   \
        while ((current_time - start_time) < wait_time) {                   \
            current_time = MPI_Wtime();                                     \
        }                                                                   \
    }
```

On failure it busy-waits a random 0–10 µs and **retries the same call indefinitely until it succeeds**. Success code 0 is common to `cudaError_t` / `cublasStatus_t` / `cusolverStatus_t`, so one macro covers all three APIs. The design absorbs transient `cudaMalloc` failures when several ranks contend for one GPU.

Commit `c5eb6845` added the line that pops `cudaGetLastError()` after every failed attempt, **cleaning the CUDA runtime's per-thread error latch**. Without it, the error code of an allocation failure that a retry later recovered from stays latched, and an unrelated later consumer of `cudaGetLastError()` (such as the Set_Hamiltonian CUDA kernel check) mistakes it for its own failure and aborts — a "phantom failure" that actually occurred at SCF iteration 316 (§ 7.7). For the same reason, the DC CPU-fallback helpers (`DC_GpuSolver_DisableGemmPath(Cuda)`) pop the latch before falling back.

Computation failures (eigensolver non-convergence etc.) are handled at the solver layer; whether that means `MPI_Abort` / `exit` or a CPU fallback differs per path (see Table 5-2).

3.7 cuBLAS Usage Caveats

1. **`wait_cudafunc` retries forever** — the call expression is re-evaluated every loop. A permanent error (bad leading dimension, lost device) means a hang with repeated side effects. When GPU execution "looks stuck", suspect this first. That said, the allocations that can legitimately run out of memory have moved to the bounded `TryDeviceMalloc` helpers plus fallbacks (§ 5.6), so far fewer spots can hang on a permanently full device.
2. **Hard-coded ld/alpha/beta in simplified helpers** — see § 3.5; wrong silently for transposed non-square inputs.
3. **Asymmetric stream assumptions** — Band_DFT_Col uses the default stream + synchronous memcpy; the others use dedicated streams + explicit sync. Check the convention when moving code around.

4. **Error-latch etiquette** — if you add code that uses `cudaGetLastError()` to check its own kernel, pop stale errors before launching (follow `set_hamiltonian_gpu.cu`). This tree assumes coexistence with components that recover from failures internally.
5. **Zero-size guard** — the GEMMul8 bridge returns success immediately for m,n,k ≤ 0 (all GEMMs pass through the bridge, so this one guard covers everything).
6. **No cuBLAS workspace configured** — the 32 MiB `cublasSetWorkspace` recommended by GEMMul8 is not set on any handle (a candidate performance improvement; see the notes in `gemmul8.hpp`).
7. **Column-major convention** — cuBLAS is column-major like Fortran BLAS. OpenMX's real symmetric matrices are sometimes passed as row-major flat arrays relying on symmetry (equivalent to a transpose); Hermitian matrices are repacked explicitly or transposed on device with `cublasDgeam/Zgeam` (DC, noncollinear DC-LNO).

4. How GEMMul8 Is Used

4.1 What GEMMul8 Is — Principle and Origin

GEMMul8 (GEMMulate) is a "GEMM emulation on INT8/FP8 matrix engines" library developed at RIKEN Center for Computational Science (R-CCS; lead developer Yuki Uchino; MIT license). This port pins **v3.1.1** (upstream commit `6bf409bd⋯`). Upstream: `https://github.com/RIKEN-RCCS/GEMMul8`.

The principle is **Ozaki Scheme II** — floating-point matrix-multiplication emulation based on an integer modular technique and the Chinese Remainder Theorem (CRT):

1. **Scaling + quantization:** the FP64 matrices A and B are exponent-adjusted and reduced modulo pairwise-coprime moduli p = 256, 255, 253, 251, 247, 241, … (8-bit integers, up to 20), producing `num_moduli` INT8 matrices each.
2. **Error-free integer GEMMs:** one INT8 GEMM per modulus, executed as cuBLAS `cublasGemmEx(⋯, CUDA_R_8I, ⋯, CUBLAS_COMPUTE_32I, ⋯)` — i.e. **INT8 Tensor Cores (IMMA)**. Integer arithmetic introduces no rounding error.
3. **CRT reconstruction:** the per-modulus results are combined via the CRT and the scaling is undone, yielding $\alpha AB + \beta C$.

More moduli (`num_moduli`) means higher precision and lower speed. The procedure is deterministic, so **results are bitwise reproducible for a fixed configuration** (stated in the upstream README). GEMMul8 implements Scheme II, the successor of the ozIMMU line (Ootomo, Ozaki, Yokota 2024, Ozaki Scheme I). See the references.

4.2 Integration Architecture — the Two-File Scheme

OpenMX itself is C code built with NVHPC `mpicc -std=c11` and cannot call a C++ template API directly, and the upstream build system generates many objects per routine × type × backend. The port integrates GEMMul8 with two files:

(a) gemmul8_openmx.cu — explicit template instantiation in a single translation unit

The upstream template-definition headers (`gemm_impl.hpp`, `worksize_impl.hpp`, and seven kernel-definition headers that upstream compiles into separate objects) are included into one translation unit, and only the **four symbols OpenMX needs** are explicitly instantiated (a 67-line file).

gemmul8_openmx.cu:31-44 (double version; the cuDoubleComplex and two workSize instantiations are analogous)

```
template std::vector<double> gemm<double, Backend::INT8, double, double>(
    cublasHandle_t,
    cublasOperation_t, cublasOperation_t,
    size_t, size_t, size_t,
    const double *,
    const double *const, size_t,
    const double *const, size_t,
    const double *,
    double *const, size_t,
    int, bool,
    void *const,
    void *const,
    void *const,
    bool, bool, bool, bool);
```

The instantiated symbols are `gemm<double,INT8>`, `gemm<cuDoubleComplex,INT8>`, `workSize<false,INT8,Func::gemm>`, and `workSize<true,INT8,Func::gemm>`. That single object is archived into `third_party/GEMMul8/lib/libgemmul8.a` (different contents from the upstream library of the same name). Because the scheme depends on upstream's internal header layout, **pinning the upstream commit is mandatory** (stated in a Makefile comment; § 4.7).

(b) gemmul8_bridge.cu — the extern "C" bridge

Five functions are exported to C (prototypes at `openmx_common.h:4164-4173`):

- `cublasStatus_t openmx_gemmul8Dgemm(handle, transa, transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)` — a signature **identical to cublasDgemm** (arbitrary alpha/beta and ld)
- `openmx_gemmul8Zgemm(⋯)` — the complex twin
- `size_t openmx_gemmul8DWorkspaceSize(m, n, k)` / `openmx_gemmul8ZWorkspaceSize(m, n, k)` — **workspace size queries** evaluating `gemmul8::workSize` under the currently effective num_moduli (0 if that precision is disabled by environment variables). The solver VRAM planners (band k-turn planning, the DC-LNO group-fit test) use these to account for GEMM workspaces exactly (added by commits `58ea6471`, `d62cac60`)
- `openmx_gemmul8ReleaseWorkspaces()` — release all workspaces

The bridge handles (i) environment variables, (ii) workspace query/allocation/caching, (iii) fallback to native cuBLAS when allocation is refused, and (iv) the `gemmul8::gemm` call.

gemmul8_bridge.cu (skeleton of the Dgemm entry; Zgemm is identical in structure)

```
if (gemmul8_disabled("OPENMX_GEMMUL8_DISABLE_D", "GEMMUL8_DISABLE_D")) {
    report.reason = "environment disable";
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}

cublasStatus_t status =
    ensure_workspace<false>(handle, static_cast<size_t>(m), static_cast<size_t>(n), static_cast<size_t>(k),
                            num_moduli, &work, &report);
if (status == CUBLAS_STATUS_ALLOC_FAILED) {
    log_workspace_fallback_once<false>(report);
    return cublasDgemm(handle, gemmul8_transa, gemmul8_transb, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc);
}
if (status != CUBLAS_STATUS_SUCCESS) {
    return status;
}

(void)gemmul8::gemm<double, gemmul8::Backend::INT8>(handle, gemmul8_transa, gemmul8_transb, ..., num_moduli, fastmode, work);
```

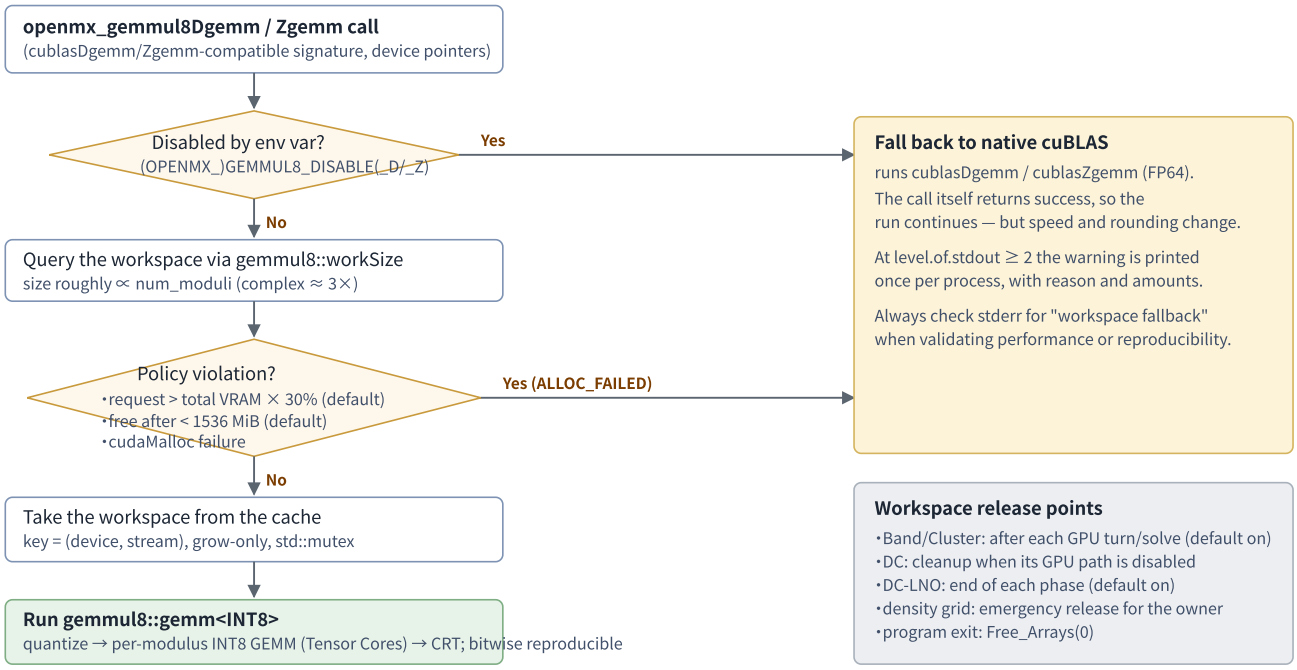


Figure 3: Dispatch flow of the GEMMul8 bridge (gemmul8_bridge.cu).

4.3 Call Parameters and Precision Settings

Parameter	Default	Environment variable (OPENMX_ name wins)	Meaning / behavior
`num_moduli`	**15**	`(OPENMX_)GEMMUL8_NUM_MOD_D` / `_Z`	Number of moduli (precision). Valid 2–20; out-of-range silently resets to 15. D and Z independent
`fastmode`	**false** (accurate)	`(OPENMX_)GEMMUL8_FASTMODE_D` / `_Z` ("1" enables)	Fast scaling path; lower accuracy
Disable	off	`(OPENMX_)GEMMUL8_DISABLE`, `_D` / `_Z`	Pin to native cuBLAS FP64
Workspace cap	**30%**	`(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT`	Refuse if the request exceeds this fraction of total VRAM (Band_DFT_NonCol setenvs 50 when unset)
Min free after	**1536 MiB**	`(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB`	Refuse if free VRAM after allocation would fall below this

- The real (D) entry silently normalizes `CUBLAS_OP_C` to `CUBLAS_OP_T`.
- `m,n,k ≤ 0` is a no-op returning success.
- OpenMX contains no code comparing GEMMul8 results against direct FP64; every fallback is triggered by memory or environment conditions, never by an accuracy test.
- Upstream extras such as skip-scaling are unused; the per-phase timing return value is discarded.

4.4 Workspace Management

- **Query:** each call computes the need via `gemmul8::workSize<is_complex,INT8>(m,n,k,num_moduli)`; solvers can pre-query via `openmx_gemmul8D/ZWorkspaceSize` and fold it into their VRAM estimates.
- **Cache:** a process-wide `std::unordered_map` keyed by `(device, cudaStream_t)`, mutex-protected, grow-only. The cuBLAS handle's stream is part of the key, so **each handle (stream) gets its own region**.

- **Size:** A side = num_moduli INT8 matrices + shift vectors; B likewise; C side = (num_moduli−1) intermediate matrices + a 32 MiB internal BLAS region + INT32 accumulators. Complex is about 3×. Around n ≈ 10,000 this is **about 1 GiB per rank**.
- **Release:** only via `openmx_gemmul8ReleaseWorkspaces()`, called from (1) `Free_Arrays(0)` at program end, (2) Band_DFT_Col at the end of each GPU turn (`OPENMX_BAND_GEMMUL8_TURN_RELEASE=0` opts out), (3) Cluster_DFT_Col/NonCol after each solve (`OPENMX_CLUSTER_GEMMUL8_TURN_RELEASE`, inheriting the BAND setting when unset), (4) DC when its GPU path is disabled, (5) DC-LNO at the end of each phase (`OPENMX_DCLNO_GPU_TURN_RELEASE`), and (6) the density-grid GPU service as an emergency release when the owner's allocation is about to fail.

4.5 Complex (Z) Support

OpenMX does not split complex matrices into real pairs; it passes `cuDoubleComplex` straight to `gemm<cuDoubleComplex, INT8>`. GEMMul8's native complex path keeps three low-precision representations per input and forms the complex product with **three real INT8 GEMMs per modulus** (a 3M/Karatsuba-style scheme, not 4M; see the CRT complex-multiplication paper in the references). Hence complex workspace and arithmetic cost are about 3× the real case.

4.6 Which GEMMs Run on GEMMul8

The bridge has **no size-based dispatch** (GEMMul8 is the default; fallbacks are memory/environment conditions only). Since commit `66ce1e62`, **every GPU GEMM passes through the bridge** (Table 3-2). The flagship example is the generalized-eigenproblem transform H′ = Ũ † HŨ in Band_DFT_Col (the largest GEMM load):

Band_DFT_Col.c — the congruence transform (first two GEMMs; the back-transform follows)

```
wait_cudafunc(openmx_gemmul8Zgemm(ctx->cublas, CUBLAS_OP_N, CUBLAS_OP_N, n, n, n, &alpha,
                                  (cuDoubleComplex *)ctx->d_H, n, (cuDoubleComplex *)ctx->d_S, n, &beta,
                                  (cuDoubleComplex *)ctx->d_tmp, n));

wait_cudafunc(openmx_gemmul8Zgemm(ctx->cublas, CUBLAS_OP_C, CUBLAS_OP_N, n, n, n, &alpha,
                                  (cuDoubleComplex *)ctx->d_S, n, (cuDoubleComplex *)ctx->d_tmp, n, &beta,
                                  (cuDoubleComplex *)ctx->d_H, n));
```

4.7 Build

- **Fetch:** `make` runs `git fetch --depth 1 origin $(GEMMUL8_COMMIT)` → `checkout` in the submodule directory, pinning commit `6bf409bd⋯` (v3.1.1). A fresh clone without the checkout also works, via the order-only rule for `gemmul8.hpp` (commit `9fe8ce13`). The Makefile comment explains why the pin is mandatory: gemmul8_openmx.cu instantiates GEMMul8 internals directly, so the checkout must stay at a commit whose internal layout matches.
- **Compile:** nvcc from the NVHPC-bundled CUDA 13.1, host compiler `-ccbin nvc++`, `-std=c++20 -O3 -diag-suppress 177 -DGPU_ARCH=$(arch) -gencode arch=compute_$(arch),code=sm_$(arch)`. The architecture is auto-detected via `nvidia-smi --query-gpu=compute_cap` (compute_120/sm_120 in the development environment); manual override with e.g. `GEMMUL8_GPU_ARCH=90`.
- **Environment sanitization:** `NVCC_GEMMUL8_ENV` clears `CPATH` etc. so NVHPC headers/flags cannot leak into the nvcc compile.
- **Extra link libraries:** `-lcublasLt -lcuda -lnvidia-ml -lstdc++`.

4.8 GEMMul8 Usage Caveats

1. **Memory footprint** — workspace roughly proportional to num_moduli, complex ≈ 3×, about 1 GiB/rank at n ≈ 10,000. On shared GPUs the three-layer guard (30% cap, 1536 MiB reserve, per-turn release) prevents exhaustion, and the solver planners additionally pre-account workspaces via `openmx_gemmul8D/ZWorkspaceSize` — keep these consistent if you change defaults.
2. **Silent FP64 fallback** — on workspace refusal the bridge switches to cuBLAS FP64 with a single warning, changing **both speed and bit patterns**. When a run is "mysteriously slow / numerically jittery", run it with `level.of.stdout 2` (or `OPENMX_GEMMUL8_VERBOSE=1`) and check stderr for "GEMMul8 workspace fallback". For strictly deterministic comparisons, set `OPENMX_GEMMUL8_DISABLE=1` or leave ample memory headroom.
3. **Precision settings** — default 15 moduli, accurate mode. Out-of-range values silently reset to 15. fastmode is faster but less accurate. D and Z are tunable independently.
4. **First-call cost** — the first call (and any size growth) performs synchronous GiB-scale `cudaMalloc`s. Benchmark from the second iteration onward.
5. **Build pitfalls** — (a) unpinning the commit breaks `gemmul8_openmx.cu` when internal headers move; (b) unsanitized NVHPC environment variables can make nvcc pick up NVHPC headers and fail; (c) forgetting `-lcublasLt`/`-lstdc++`; (d) architecture autodetection assumes `nvidia-smi` works; (e) single-arch `-gencode` means the binary is not portable across GPU generations (rebuild required).
6. **Following upstream** — the migration to the current upstream layout (commit `cb4d348d`) shrank `gemmul8_openmx.cu` from 149 to 67 lines with no change to the bridge's public signatures. For future upstream updates, check first which definition headers the single TU must include.
7. **Hardware requirements** — INT8 Tensor Cores (IMMA) required; effectively a C++20-capable nvcc (CUDA 12+; only CUDA 13.1 has been verified).

5. How cuSOLVER (the gpusolver Layer) Is Used

5.1 Layer Structure and Naming Intent

Eigenvalue computation uses cuSOLVER through a vendor-neutrally named wrapper layer, **gpusolver**, organized in three tiers:

1. **Generic one-shot wrappers:** six functions in `gpusolver_Syevd.c` / `gpusolver_Syevdx.c` — host-array versions, `_openacc` versions that operate directly on OpenACC-resident arrays, and a `_cached` version that keeps handle and workspace across calls (Table 5-1). The license header shows their origin in the NVIDIA CUDA Library Samples.
2. **A shared utility header:** `openmx_gpusolver_dense_utils.h` (79 lines) statically defines the 1-based/0-based variants of `OpenMX_GpuSolver_DenseDsyevx/DenseZheevx` and calls `MPI_Abort` when `info != 0`. Twelve polarization/DMmu/CWF/LNO files include it.
3. **Per-solver resident contexts:** `BandColGpuSolverCtx`, `ClusterColGpuSolverCtx`, `DCGpuSolverCtx`, `DCLNO_GpuSolverZCtx`, the Krylov per-thread workspace pool, etc., caching handles and device buffers and calling the cuSOLVER API directly.

Backend selection: function names and the input keyword use gpusolver in both backends. CMake selects the source set before compilation; each executable uses one backend (§ 9.7). The cuSOLVER types and APIs described below belong to the CUDA build. The HIP backend uses hipSOLVER with typed-API compatibility wrappers (§ 11.5). The old input keyword `cusolver` survives as a deprecated alias.

Function (gpusolver_Syevd(x).c)	cuSOLVER API	Data location	Current callers
`gpusolver_Syevd(A,W,m)`	Xsyevd (all eigenvalues)	host⇄device transfer built in	**dormant** no callers
`gpusolver_Syevdx(A,W,m,MaxN)`	Xsyevdx (lowest MaxN)	ditto	`Eigen_lapack.c`, many via dense_utils
`gpusolver_Syevdx_openacc(…)`	Xsyevdx	OpenACC-resident (present + host_data)	`Eigen_lapack.c`
`gpusolver_Syevdx_Complex(…)`	Xsyevdx (CUDA_C_64F)	host⇄device transfer built in	`EigenBand_lapack.c`, many via dense_utils
`gpusolver_Syevdx_Complex_openacc(…)`	Xsyevdx (CUDA_C_64F)	OpenACC-resident	`EigenBand_lapack.c`
`gpusolver_Syevdx_Complex_openacc_cached(…)`	Xsyevdx (CUDA_C_64F)	OpenACC-resident	Per-atom diagonalizations of noncollinear DC and DC-LNO (new in commit `6f7f31b2`; a static context keeps handle, stream, grow-only workspaces, and d_info, eliminating per-atom create/destroy)

Table 5-1: Public API of the generic wrappers

5.2 API Details

- Everything uses the **generic (X) API with 64-bit integers**: `cusolverDnXsyevd(_bufferSize)` / `cusolverDnXsyevdx(_bufferSize)`. No legacy APIs, no separate Zheevd-style names — complex data goes to Xsyevdx as `CUDA_C_64F`. The generalized solver `cusolverDnXsygvd` is not used (§ 5.3).

- Common parameters: `params = NULL`, `uplo = CUBLAS_FILL_MODE_LOWER`; range selection is `CUSOLVER_EIG_RANGE_I` (il=1, iu=MaxN), with some implementations switching to `RANGE_ALL` when `m == MaxN`. jobz is normally `CUSOLVER_EIG_MODE_VECTOR`, but the first pass of Band(Col) — which needs eigenvalues only — solves with `CUSOLVER_EIG_MODE_NOVECTOR` and skips the back-transform GEMM (`BandCol_EigenvaluesOnlyNoVectors()`; disable with `OPENMX_BAND_SYEVDX_NOVECTOR=0`).
- Handles: the generic wrappers create `cusolverDnCreate` → non-blocking stream → compute → destroy per call; the `_cached` wrapper and the resident contexts cache them (§ 3.2).
- Workspaces: every path queries device and host sizes via `_bufferSize` before allocating; the resident contexts cache grow-only. The concurrency planners also **actually call** `_bufferSize` to fold the syevdx workspace into their VRAM estimates (falling back to $4n^2$ complex elements if the query fails; § 5.5).

gpusolver_Syevdx.c — the core call shape (64-bit generic API)

```
wait_cudafunc(cusolverDnXsyevdx_bufferSize(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu,
                                           1L, MaxN, &h_meig, CUDA_R_64F, d_W, CUDA_R_64F,
                                           &workspaceInBytesOnDevice, &workspaceInBytesOnHost));

wait_cudafunc(cudaMalloc((void **)(&d_work), workspaceInBytesOnDevice));
h_work = (workspaceInBytesOnHost == 0) ? NULL : malloc(workspaceInBytesOnHost);
...
wait_cudafunc(cusolverDnXsyevdx(cusolverH, NULL, jobz, range, uplo, m, CUDA_R_64F, d_A, lda, &vl, &vu, 1L, MaxN,
                                &h_meig, CUDA_R_64F, d_W, CUDA_R_64F, d_work, workspaceInBytesOnDevice, h_work,
                                workspaceInBytesOnHost, d_info));
```

5.3 The Generalized Eigenvalue Problem Hc = εSc

cuSOLVER's generalized solver is not used; each path implements OpenMX's traditional **Löwdin (symmetric) orthogonalization via the eigendecomposition of S** on the GPU. Cholesky is never used. The common six steps:

1. Diagonalize S with Xsyevdx (all eigenvalues) → eigenvectors U, eigenvalues λ
2. Treat small/negative eigenvalues (**differs by path**: Band/Cluster clip up to 1×10^{-10}, DC drops those below `OLP_eigen_cut`, DC-LNO takes `fabs`)
3. Scale U's columns by $1/\sqrt{\lambda}$ (`cublasDdgmm/Zdgmm`) → $\tilde{U}$. Noncollinear DC-LNO additionally builds the spin-doubling transform T = diag($\tilde{U}$, $\tilde{U}$) with a `cudaMemcpy2DAsync` block copy
4. Form $H' = \tilde{U}^\dagger H \tilde{U}$ with two GEMMs (GEMMul8; Table 3-2)
5. Diagonalize H' with Xsyevdx (il=1..MaxN) → ε and z
6. Back-transform $c = \tilde{U}z$ with a GEMM (skipped where only eigenvalues are needed)

Cluster_DFT_Col.c — S diagonalization and $1/\sqrt{\lambda}$ scaling (Löwdin on the GPU)

```
if (rebuild || !(ctx->transformed_s_valid && ctx->transformed_s_dim==n)){
    ClusterCol_GpuSolver_EigenDevice(ctx->d_S,n,n,ko0+1);

    for (int l=1; l<=n; l++){
        if (ko0[l]<1.0e-10) ko0[l] = 1.0e-10;
        ko0[l] = 1.0/sqrt(ko0[l]);
    }

    wait_cudafunc(cudaMemcpyAsync(ctx->d_W,ko0+1,sizeof(double)*(size_t)n,cudaMemcpyHostToDevice,ctx->stream));

    old_s = ctx->d_S;
    new_s = ctx->d_tmp;
    wait_cudafunc(cublasDdgmm(ctx->cublas,CUBLAS_SIDE_RIGHT,n,n,
                              old_s,n,ctx->d_W,1,new_s,n));
```

> **Caveat (numerical behavior):** the small-eigenvalue treatment of S (clip / drop / fabs) is not unified across paths, so ill-conditioned overlap matrices can behave slightly differently per solver path for the same physical system.

Choosing MaxN (number of partial eigenpairs): Band(Col) uses `MaxN = (TZ-charge)/2 + max(0.4·Ne/2, 100)`; Cluster solves everything on the first SCF iteration and afterwards `(TZ-charge)/2 + max(0.1n-0.2n, 400)`; DC-LNO solves everything up to SCF 2, then `HO_TC + 100`. Solving only the occupied bands plus a margin bounds the syevdx cost.

5.4 devInfo Checking and Failure Handling per Path

`d_info` is device-allocated on every path and checked after the solve, but the **failure policy differs per path**. Note that VRAM shortfalls are detected earlier, at the estimate/reservation stage, and fall back to ELPA/CPU (§ 5.6); this table covers **numerical failures after entering the GPU path**.

Path	info check	On failure
Generic wrappers (gpusolver_Syevdx etc.)	returned to caller	caller-dependent
Via dense_utils header (11 files)	yes	immediate `MPI_Abort`
`Eigen_lapack.c`	**info<0 only** exit(10)	info>0 (non-convergence) passes through, but the caller `Eigen_lapack()` retries with an orthogonality check, cycling to CPU solvers (up to 3 attempts)
`EigenBand_lapack.c`	yes	falls back to the CPU Householder solver `Eigen_HH`
Band_DFT_Col / Cluster_DFT_Col / Cluster_DFT_NonCol	yes (Band also checks h_meig)	immediate abort (exit / MPI_Abort) — but memory-driven infeasibility escapes to ELPA beforehand, so only numerical failures land here
Divide_Conquer (DC)	yes (h_meig too)	sets a **sticky disable flag**; that rank continues on the CPU DC solver (GEMMul8 workspaces released)
Divide_Conquer_LNO (Col/NonCol)	yes	exit / abort (no fallback; but the group memory-fit test may have chosen the CPU eigensolver up front)
Krylov	yes	falls back to CPU LAPACK (`Eigen_lapack2`) on the spot (the fused path re-uploads the result and continues)

Table 5-2: devInfo checks and failure policies

5.5 Adaptive Concurrency Control — UUID Groups and Prefix Scans

As of the 2nd edition, concurrency control meant a fixed turn group (default 4) and a fixed ladder (32→16→…→1). Now **every path uses measurement-driven adaptive control** (commits `58ea6471`, `7c414f91`, `48921b36`, `99f4014b`, `6b84a1f9`). The common skeleton:

1. **Detect the sharing group:** build the node communicator with `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)`, Allgather the 16-byte **GPU UUID** from `cudaGetDeviceProperties`, and split into per-physical-GPU communicators. Matching by UUID rather than device ordinal keeps this correct under differing `CUDA_VISIBLE_DEVICES`.
2. **Agree on free VRAM:** before measuring, run `acc_wait_all(); cudaDeviceSynchronize(); acc_clear_freelists();` so each process returns its OpenACC pool to CUDA, then everyone calls `cudaMemGetInfo` and reduces with `MPI_Allreduce(MIN)` to a group-common free figure.

3. **Per-rank requirement model:** dense matrices + vectors + the syevdx workspace (**the real `_bufferSize` value**) + the GEMMul8 workspace (`openmx_gemmul8D/ZWorkspaceSize`) + sparse→dense construction / DM staging areas + a runtime overhead (default 256 MiB, `OPENMX_BAND_GPU_RANK_OVERHEAD_MB`).
4. **Prefix scan:** Allgather the requirements, sort descending, and pick the largest c with `peak ≤ group_free and reserve ≤ group_free − peak`, where reserve = max(total VRAM/10, 1536 MiB) (raise with `OPENMX_BAND_GPU_RESERVE_MB`). The initial candidate is the device-sharing rank count (the fixed ladder and its implicit cap of 32 are gone; the environment variables now mean explicit caps only).
5. **Agreement and degradation:** the result is unified with `MPI_Allreduce(MIN)`. If all ranks fit simultaneously, the turn split itself is folded away (barriers and GEMMul8 release points disappear); if not even one rank fits, the path falls back to ELPA (§ 5.6).
6. **Publishing the need:** the full-concurrency requirement is published via `OpenMX_GpuPhaseNeed_Register("band_col" etc.)` so the Set_Hamiltonian resident cache makes room for it (§ 7.7).

5.5.1 Band (Col) — BandCol_AutoGpuTurnLimit

The requirement model is `BandCol_GpuTurnRequiredBytes(n, maxn, size_H1)` (`Band_DFT_Col.c:429`): three dense matrices ($3 \cdot n^2 \cdot 16$ B) + vectors + the syevdx workspace (both (n,n) and (n,maxn) shapes queried and memoized) + the GEMMul8 Zgemm workspace + max(construction staging, DM staging) + overhead. The decision is reported once per change by the device-lead rank:

```
<Band_DFT_Col> GPU device %d: %d k-owner rank(s), GPU concurrency=%d, turn group=%d,
    per-rank=%.3f GiB, peak=%.3f GiB, free=%.3f GiB, reserve=%.3f GiB
```

The GPU-turn serialization of the `all_knum==1` side (`OPENMX_GPUSOLVER_SERIAL_GPU_TURNS`, default 1) is retained; the source comment still notes "about 27% faster than concurrent submission with 8 owner ranks per GPU". Explicit caps are read in the order `OPENMX_BAND_GPU_MAX_CONCURRENT_K` → `OPENMX_BAND_GPU_TURN_GROUP` → `OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE`.

5.5.2 Band (NonCol) — Three Stage-Specific Estimates

Because the memory peak varies strongly within one noncollinear turn, the estimate is split into three modes — **eigenvalues-only (EIGVALS) / eigenvectors (EIGVECS) / density matrix (DM)** — each deciding its own concurrency (`BandNonCol_RootDenseDeviceBytes` / `BandNonCol_RootDenseDMDeviceBytes`, `Band_DFT_NonCol.c:803,871`). The DM accumulation footprint is far smaller than the solve, enabling e.g. "solve two at a time, accumulate DM eight at a time" (commit `7c414f91`). The report line carries the mode name:

```
<Band_DFT_NonCol> GPU device %d: %d k-owner rank(s), GPU concurrency=%d (eigenvalues only |
    eigenvectors | density matrix), turn group=%d, per-rank=%.3f GiB, ...
```

Whether multiple k-worlds may run concurrently is judged in the same framework; if not, the code prints "Serializing non-collinear dense GpuSolver k-worlds" and serializes. `BandNonCol_SetDenseGemmul8Defaults` also setenvs `OPENMX_GEMMUL8_MAX_WORKSPACE_PERCENT=50` when unset.

5.5.3 Cluster (Col/NonCol) — Real Reservations

The cluster paths have no k-turns (a single root-dense owner), so instead of concurrency control they decide feasibility by **real reservation rather than estimation**: the owner rank actually cudaMallocs the matrix buffers and the syevdx workspace (using the **real `_bufferSize` figure**) with 8-attempt bounded retries, and additionally requires `OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB` (default 1024 MiB) to remain free (`ClusterCol_OwnerReserveProbe`). The

probe counts the n^2 basis of the refined FP32 eigensolver (`OPENMX_EIGEN_REFINE`, § 10.2) as well; when the dense buffers and the reserve fit but the basis does not, the GPU path is kept and the cycle is solved by the FP64 eigensolver instead of falling back to ELPA (one line `… holds its GPU diagonalization buffers but not the … basis of the refined eigensolver …: FP64 eigensolver for this SCF cycle`; C1000 at 18 ranks on the 16 GB RTX 5080 is such a case: 1257 MiB free after the reservation against the 1024 MiB reserve plus the 1289 MiB basis). The switch to real reservations was motivated by measurement: the workspace estimate of $2n^2$ doubles versus about $4n^2$ doubles in reality (commit `cff323e6`). The noncollinear path reserves the whole diagonalization cycle as a **single cudaMalloc arena** mapped into OpenACC with `acc_map_data`, making the verdict immune to other phases filling the device mid-cycle (`ClusterNonCol_DenseArena_TryEnsure`).

For spin-polarized runs (SpinP_switch==1) where the two spin-world owners cannot fit together, a third operating mode lets **spin world 0's owner solve both spins serially** (commit `bf08a9d8`; `OPENMX_CLUSTER_GPU_DIAG_SERIAL`, default 1; 2 forces serial). The remote spin's eigenvalues and eigenvector panel travel by MPI (tags 997/998, split into 64M-element chunks). If even that fails, the run falls back to ELPA (§ 5.6).

5.5.4 DC-LNO — Group-Aggregate Memory Test

Because DC-LNO's direct per-rank dispatch keeps solver buffers resident on every rank, `DCLNO_GpuGroupMemoryFits` (`Divide_Conquer_LNO.c:1012`) sums "7·elem·Max_Msize² + GEMMul8 workspace + 256 MiB" across the device group (Allreduce SUM) and compares against the group's minimum free VRAM; if it does not fit, **the whole group switches to the CPU eigensolver** (this fixed an 18-rank NC run demanding 26.1 GiB of a 16 GiB card and spinning for 40+ minutes). The group leader prints once:

```
<DC-LNO> GPU device %d: %d rank(s) would need %.3f GiB (free %.3f GiB, reserve %.3f GiB);
    using the CPU eigensolver instead.
```

5.6 Runtime ELPA Fallback

Up to the 2nd edition, running out of VRAM after entering a GPU path meant an abort or a hang. Commits `ae3dd189` / `cff323e6` / `3ef5347a` / `a4200a04` added a **runtime ELPA/ScaLAPACK fallback to all four band/cluster paths**. Common contract:

- Verdict functions: `BandCol_GpuDenseFits` / `BandNonCol_GpuDiagFits` (one-turn estimates), `ClusterCol_GpuDiagFits` / `ClusterNonCol_GpuDiagFits` (real reservations).
- The verdict is a **collective agreement** via `MPI_Allreduce(MIN)` over `mpi_comm_level1` — the GPU/ELPA branch changes the collective-communication structure, so all ranks must land on the same side.
- Verdicts are cached (sticky) until an SCF restart or a matrix-size change.
- On fallback, all reserved buffers are released before entering the untouched ELPA/ScaLAPACK code. The notice is printed once:

```
<Cluster_DFT_Col> Falling back to the ELPA/ScaLAPACK diagonalization. Force the GPU path with
    OPENMX_CLUSTER_GPU_DIAG=1 or lower OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB.

<Band_DFT_Col> A k-owner rank cannot fit even one k-point's dense GPU diagonalization
    (%.1f MiB needed per rank, %.1f MiB free); falling back to the ScaLAPACK/ELPA diagonalization.
    Force the GPU path with OPENMX_BAND_GPU_DIAG=1 or lower OPENMX_BAND_GPU_RESERVE_MB.
```

- Force knobs: `OPENMX_BAND_GPU_DIAG=1/0`, `OPENMX_CLUSTER_GPU_DIAG=1/0` (1 = ignore the verdict and force the GPU — reservation failure then aborts with a clear diagnostic; 0 = always ELPA).

- When a mandatory allocation genuinely runs dry, the abort is a diagnostic with amounts rather than a silent hang: `"Cluster_DFT_Col.c: out of GPU memory while allocating %s (%.1f MiB requested, %.1f MiB free of %.1f MiB). Reduce the MPI ranks sharing the device or set OPENMX_CLUSTER_GPU_DIAG=0 …"`

> **Why:** previously the root-dense matrix allocation used the infinite retry (`wait_cudafunc(cudaMalloc(…))`), so on a full device the owner spun while every other rank waited in MPI_Bcast — a silent node-wide freeze (observed at n=13,000, np18, 16 GiB GPU). Bounded retries + real reservations + collective agreement + ELPA fallback eliminate this failure class by construction (commit `ae3dd189`).

5.7 MPI Rank → GPU Device Assignment

- **Collective version** (`set_cuda_default_device_from_local_rank.c`): intra-node local rank from `MPI_Comm_split_type(MPI_COMM_TYPE_SHARED)` , then `cudaSetDevice(local_rank % deviceCount)` . Must be called by all ranks together (one-sided calls deadlock).
- **Non-collective version**: for SCF hot paths; reads the local rank from `OMPI_COMM_WORLD_LOCAL_RANK` → `MV2_COMM_WORLD_LOCAL_RANK` → `SLURM_LOCALID` → `PMI_LOCAL_RANK` , falling back to the global rank (which can skew multi-node assignment).
- **OpenACC set as a pair** (`set_openacc_device_from_local_rank.c`): the same rule feeds `acc_set_device_num` ; setting **both runtimes to the same device is mandatory**.
- Which ranks actually share which physical GPU is then discovered autonomously per phase via the UUID Allgather (§ 5.5).

5.8 LU_Zinverse_GPU — Dormant NEGF Implementation

> **Dormant code:** the GPU complex LU inverse `LU_Zinverse_GPU` (`Lapack_LU_inverse.c:347-444`) is implemented not with cuSOLVER but with the **batched cuBLAS API** `cublasZgetrfBatched` / `cublasZgetriBatched` (batch=1). A is an OpenACC `deviceptr` ; singular matrices abort; results are written back in an `acc kernels` region. But the call from the public `Lapack_LU_Zinverse` is commented out (`/* LU_Zinverse_GPU(n, A, handle); */`), so the default remains the CPU implementation (LAPACK zgetrf/zgetri). The callers are NEGF surface-Green's-function routines (`TRAN_Calc_SurfGreen.c` etc.) — this is groundwork for GPU-accelerating NEGF complex inverses.

5.9 cuSOLVER (gpusolver Layer) Caveats

1. **Workspace sizing** — an n×n complex matrix is $16n^2$ bytes; for the syevdx workspace use the real `_bufferSize` value, not an estimate (a $2n^2$ estimate versus ~$4n^2$ reality has been measured). Reuse the planner models of § 5.5.
2. **`wait_cudafunc` infinite-loop risk** — same as § 3.7. For allocations that can run permanently dry, the current convention is the bounded `TryDeviceMalloc` helpers.
3. **Non-uniform devInfo policy** — Table 5-2; recovery from numerical failure differs by path. Band/Cluster/DC-LNO/dense_utils are "numerical failure = terminate" (memory shortfalls escape to ELPA earlier).
4. **The fallback hierarchy** — (a) no device: the whole run demotes to ELPA2 at startup; (b) VRAM shortfall: per-path fits verdicts fall back to ELPA/CPU (§ 5.6); (c) numerical failure: Table 5-2. Do not conflate the three layers.

5. **Symmetry and 0/1-based conversions** — all calls assume uplo=LOWER symmetric/Hermitian input; the 0/1-based eigenvalue shifts (`W[l]=W[l-1]`) are needed in many places and absorbed by dense_utils.
6. **Inconsistent conditioning of S** — see the warning in § 5.3.
7. **Reproducibility** — the transform GEMMs run on GEMMul8, whose rounding differs from cuBLAS FP64, and memory-driven fallbacks (including GPU⇄ELPA) can change the path between runs. Verdicts are sticky, so nothing flips within one SCF cycle.
8. **fits verdicts are collective** — the verdict functions internally call `MPI_Comm_split_type` / `Allreduce` / `MPI_Barrier`, so **all ranks must reach them the same number of times**; letting only some ranks skip a check deadlocks the run.

6. OpenACC versus Raw CUDA

6.1 Overview

Loop-parallel offloading is written in NVHPC OpenACC (`-acc -Minfo=accel`). Key points:

- **There are no GPU-only `#ifdef` guards.** The CUDA headers (`cublas_v2.h` etc.) are included unconditionally from `openmx_common.h`, so **this tree cannot be built CPU-only** (use the original, or `makefile.cpu_intel.bak`). All gating is at runtime.
- No `-gpu=ccXX` is specified; the target CC comes from autodetection on the build machine. **Managed memory is unused**; `async` clauses and `acc_get_cuda_stream` are unused throughout.
- About 30 files carry `#pragma acc`. The densest are `Divide_Conquer.c` (66), `Band_DFT_NonCol.c` (61), `Force.c` (49), `Band_DFT_Col.c` (46), `Cluster_DFT_NonCol.c` (43), and `Set_ProExpn_VNA.c` (21). **`Krylov.c` has zero OpenACC** (compiled without `-acc`; raw CUDA only).
- The basic shape is `parallel loop gang` + `loop vector reduction`. Reductions where reproducibility matters — density-matrix builds, grid traces — use `loop seq` or match the CPU's summation order down to the float-multiply→double-add cast position.

6.2 Four Patterns of Data Residency

1. **Per-call transient data regions (copyin/copyout)** — `Total_Energy_GPU`, `Set_Nonlocal`, etc. OpenMX's multi-level pointer arrays cannot be traversed on the device, so the rule is: **always flatten to 1-D host arrays before transfer**.
2. **SCF-scope residency via enter data / exit data + update** — the Band_DFT_NonCol root-dense path, and the Set_Hamiltonian SCF-invariant tables (present_or_copyin semantics: no transfer when already resident; § 7.7).
3. **Raw `cudaMalloc` + `deviceptr` bridging** — `Band_DFT_Col.c` / `Cluster_DFT_Col.c` run OpenACC scatter-adds (`#pragma acc parallel loop deviceptr(d_H)` + `acc atomic update`) against the static contexts' cudaMalloc regions.
4. **Single-allocation arenas + `acc_map_data` / `deviceptr`** — the newest pattern: many buffers partitioned 512-byte-aligned inside one `cudaMalloc` / `acc_malloc` region, exposed to OpenACC via `acc_map_data` (the Cluster_DFT_NonCol arena) or same-named shadow pointers with `deviceptr` (the Force batches, the Mixing_H history ring). One allocation means (a) reservation success is atomic and (b) there is no partial-allocation intermediate state on a shared device.

Pointer hand-off to libraries uses `host_data use_device`: with residency declared via `present`, array names inside the block resolve to device addresses and can be passed to cuSOLVER/cuBLAS/GEMMul8 without any H2D/D2H transfer.

gpusolver_Syevdx.c — passing OpenACC-resident arrays straight to cuSOLVER

```
#pragma acc data      present(A[0 : m * m])
#pragma acc data      present(W[0 : MaxN])
#pragma acc host_data use_device(A, W)
    {
        cusolverDnHandle_t cusolverH = NULL;
        wait_cudafunc(cusolverDnCreate(&cusolverH));
        cudaStream_t stream = NULL;
        wait_cudafunc(cudaStreamCreateWithFlags(&stream, cudaStreamNonBlocking));
        wait_cudafunc(cusolverDnSetStream(cusolverH, stream));
```

> **Caveat (present errors):** the `_openacc` wrappers assume the caller has established the data region; calling without it dies with the NVHPC runtime FATAL (present table lookup failed). The current code uniformly follows the "caller does enter data first" convention.

6.3 Synchronization Design and the OpenACC Memory Pool

The OpenACC default queue and the library CUDA streams are not connected. Ordering rests solely on explicit synchronization: (1) `#pragma acc wait` before library calls, (2) `cudaStreamSynchronize` on the library stream, (3) synchronous `cudaMemcpy`. Correctness and simplicity were chosen over performance.

For shared-GPU operation, the **behavior of NVHPC's OpenACC memory pool (freelists)** matters. Blocks freed by `exit data delete` or `acc_free` return to a process-private pool, not to CUDA, and are **reused only for requests of the same size**. Both properties bite on a shared device: (a) other ranks never see "freed" memory as free; (b) repeated allocations of slightly different sizes strand blocks in the pool — an effective leak that actually caused OOM aborts in the Force3 chunk batches (commit `28361f7e`). The current convention is therefore: (i) always run `acc_wait_all(); cudaDeviceSynchronize(); acc_clear_freelists();` before measuring free VRAM, and (ii) allocate variable-size buffers once at the per-chunk maximum and reuse them. The one deliberate exception: the Mixing_H preflight **does not call `acc_clear_freelists()`** — draining the pool mid-SCF-cycle disturbed the allocation dynamics of the resident GPU modules and provoked a Set_Hamiltonian OOM about 30 iterations later (documented in a source comment).

6.4 set_hamiltonian_gpu.cu — the Wired-In Raw CUDA Kernel

The only raw CUDA kernel called from .c code in the current tree is `set_hamiltonian_gpu.cu` (194 lines, added in commit `95fa51ed`): the inner kernel `matrix_elements_kernel` of the Set_Hamiltonian dVH+Vxc+VNA grid integrals plus the extern "C" wrapper `Set_Hamiltonian_Cuda_MatrixElements()`. Design highlights:

- One block row = one atom pair; 256 threads per block. Two shared-memory tiles — a "weighted orbital" tile (double; GridVol·Vpot·Orbs0 precomputed) and an orbs1 tile (float) — iterate over grid points in tiles of 32. The float→double cast position and the multiply-add order match the CPU/OpenACC paths (bit-identical).
- Four return values: 0 = success; 1 = kernel unsupported for lack of shared memory (after halving the tile width from 32 down to 1) → caller uses the OpenACC kernel; 2 = transient failure with a healthy context (the post-stale-error reclassification) → the same batch is redone with the OpenACC kernel; negative = fatal (caller aborts).
- Before launching, it pops stale errors with `cudaGetLastError()` — the reference implementation of the error-latch etiquette of § 3.6 (commit `c5eb6845`).
- Compiled with the same nvcc flags as GEMMul8 (`NVCC_GEMMUL8_FLAGS`) and linked into `OBJS`. `OPENMX_SETHAM_CUDA_KERNEL=0` disables it, in which case the numerically identical OpenACC kernel runs.

> **Removed dormant kernel collection:** the `openmx_cuda_kernels.cu/.h` files described in the 2nd edition as "verified but unwired" have been **deleted from the tree**. Their Set_Hamiltonian-related ideas effectively landed, wired in, as `set_hamiltonian_gpu.cu`.

6.5 NVHPC-Specific Caveats — Real Miscompilation Workarounds

NVHPC 26.3's OpenACC (`-acc`) has known bugs that break specific code, and the Makefile works around them by **dropping `-acc` per file** (with reason comments). Re-verify this list whenever the NVHPC version changes.

File	Compile setting	Reason (summarized from Makefile comments)
`Occupation_Number_LDA_U.c` / `Eff_Hub_Pot.c` / `Total_Energy.c`	`CC_NOACC` (-O1, no -acc)	NVHPC 26.3 with -acc miscompiles the LDA+U Type2/cFLL occupation/potential paths
`zero_cfrac.c`	`CC_NOACC`	-acc makes DSYGV fail, breaking the ON2 poles
`RestartFileDFT.c`	`CC_NOACC`	possible segfault in restart I/O under oversubscribed MPI
`MD_pac.c` / `Generate_Wannier.c`	`CC_NOACC`	possible segfaults in post-SCF MD control / Wannier generation
`Krylov.c`	`CC_NOACC_O3` (-O3, no -acc)	GPU work done via raw CUDA only (no OpenACC)
`Band_DFT_NonCol.c` / `Divide_Conquer.c`	`CC3` (-O2)	optimization-level adjustment
`truncation.c` / `OutData.c`	`CC2` (-O1)	ditto
`DFTD3vdW_init.c`	`CC_DFTD3` (-O0)	ditto

Table 6-1: Per-file compile adjustments (including NVHPC 26.3 workarounds)

The need to build `Total_Energy.c` without `-acc` is what spawned the split translation unit `Total_Energy_GPU.c` (built with `-acc`) for its OpenACC kernels (§ 7.10). `Force.c` re-enables an OpenMP parallel region under `#if NVHPC_VERSION >= 250000` as a workaround for an older NVHPC OpenMP bug, and the `if (use_dc_openacc)` guard in `Divide_Conquer.c` still carries the comment "// compiler's bug".

7. GPU Implementation Details by Computation Path

7.1 Band Calculation (Collinear) — Band_DFT_Col.c

The largest single modification (+4,724/−2,544 lines against the original). The dense GPU path requires `scf_eigen_lib_flag==GPUSOLVER && n ≥ 1000` and, since commit `a4200a04`, a successful **one-turn VRAM estimate via `BandCol_GpuDenseFits`** (otherwise ELPA/ScaLAPACK; § 5.6). Highlights:

- **Dense-matrix construction cache:** assembling the per-k dense matrices from the sparse H and S is OpenACC-offloaded; a fingerprinted cache of the pair structure (`entries` / `phase_r` / `phase_i`) stays resident via `enter data`, with only the phases refreshed by `update device`. The scatter-add runs with `deviceptr` + `acc atomic update`, feeding from the packed H/S cache built by Set_Hamiltonian (§ 7.7).
- **Diagonalization and transforms:** the S transform Ũ is computed once and parked in the host-side `transformed_s` for reuse across SCF iterations. H′ is formed with GEMMul8 Zgemm × 3 (§ 4.6); diagonalization is `cusolverDnXsyevdx`; eigenvectors stay on the device through the DM phase.
- **Eigenvalues-only first pass:** the first diagonalization loop of `all_knum != 1` (which only needs eigenvalues to fix the chemical potential) solves with `jobz=CUSOLVER_EIG_MODE_NOVECTOR`, skipping eigenvector output and the back-transform GEMM entirely (`BandCol_GpuSolver_SolveEigenvaluesDeviceOnly`; restore the old behavior with `OPENMX_BAND_SYEVDX_NOVECTOR=0`; commit `58ea6471`). This path requires `na_rows==n && na_cols==n`.
- **Adaptive concurrency:** § 5.5.1. The k-point wave starts at the number of k-owner ranks sharing the device (commit `48921b36`) and shrinks only when VRAM demands it; when everyone fits, the turn split, mid-flight barriers, and GEMMul8 release points all disappear. The full-concurrency demand is published via `OpenMX_GpuPhaseNeed_Register("band_col", ⋯)` so the Set_Hamiltonian resident cache clears the way (§ 7.7).
- **DM phase:** OpenACC `gang` + `loop seq` reductions keep the summation order identical to the CPU.
- Cross-iteration device-buffer persistence stays off by default (`OPENMX_BAND_GPU_PERSISTENT`); per-turn GEMMul8 workspace release stays on (`OPENMX_BAND_GEMMUL8_TURN_RELEASE`).

7.2 Band Calculation (Noncollinear) — Band_DFT_NonCol.c

- An OpenACC-resident (`enter data`-centric) root-dense path: phased dense construction (scatter-add), S diagonalization + S^−1/2, assembly of the 2n×2n spinor H2/S2, U † HU (GEMMul8 Zgemm), eigenvector back-transform, and DM/EDM (`loop seq`) all on the GPU.
- **Stage-specific adaptive concurrency (§ 5.5.2):** separate VRAM estimates and rank counts for the eigenvalues-only / eigenvectors / DM modes (commits `7c414f91`, `99f4014b`). The DM-only estimate `BandNonCol_RootDenseDMDeviceBytes` is far smaller than the solve, giving regimes like "solve ×2, accumulate ×8". Needs are published as `"band_noncol_ev" / "band_noncol_vec" / "band_noncol_dm"`.
- **ELPA fallback:** `BandNonCol_GpuDiagFits` (when not even one rank fits). Force with `OPENMX_BAND_GPU_DIAG` (§ 5.6).
- Ordering against the OpenACC queue: `#pragma acc wait` → `openmx_gemmul8Zgemm` → `cudaStreamSynchronize`.

7.3 Cluster Calculations — Cluster_DFT_Col.c / Cluster_DFT_NonCol.c

The cluster paths received the densest rework since the 2nd edition (Col: +1,534 lines vs. original; NonCol: +2,424).

Col — resident root-dense path and GPU DM

- **Feasibility by real reservation (§ 5.5.3, § 5.6):** `ClusterCol_GpuDiagFits` returns a three-way verdict — 0 = ELPA fallback; 1 = concurrent GPU (each spin world solves its own spin); 2 = serialized GPU (spin world 0's owner solves both spins, shipping the remote spin's results by MPI; commit `bf08a9d8`). The serialized mode requires the packed Set_Hamiltonian cache (the non-packed matrix assembly is a level1-wide collective and cannot be run by one owner alone).
- **GPU accumulation of DM/EDM (commit `75fc30d3`):** `ClusterCol_AccumulateDM_OpenACC` accumulates DM1/EDM1 (and PDM1 for partial charges) in OpenACC gang loops over 131,072-entry chunks, with the per-state summation order identical to the CPU loop (`loop seq`). `OPENMX_CLUSTER_GPU_DM=0` selects the CPU loop. Since 2026-10-13 every rank accumulates its slice of states on the device, not only the rank that owns the dense solve: the others stage their host panel through a halving retry and fall back to the CPU loop (now threaded over the atoms) when even the smallest block does not fit; `OPENMX_CLUSTER_GPU_DM_ALL=0` restores the owner-only accumulation (C1000 at 6 ranks: the diagonalization phase 113 → 78 s, the density matrix 37 → 6 s of host time). The sparse→dense index is cached under a geometry fingerprint (FNV-1a), removing the per-spin, per-iteration rebuild and re-upload.
- **Device-resident eigenvectors (commit `527ed524`):** right after each spin's solve, the eigenvector panel is stashed by device-to-device copy (`ClusterCol_StashDeviceEvec`) and read directly by the DM kernel through `deviceptr` (eliminating the D2H → repack → H2D round trip of EVec1). The stash allocation uses `TryDeviceMalloc` and is skipped silently on failure (it is an optimization only). Non-owner ranks stage their eigenvector slices with a block size that halves until it fits (floor 128); if even that fails, the step continues on the CPU loop.
- GEMMul8 workspaces are released after each solve (`OPENMX_CLUSTER_GEMMUL8_TURN_RELEASE`, inheriting the BAND setting when unset). The old `ClusterCol_GpuSolverDensePath` (CPU dgemm) and the host-buffer path were deleted.

NonCol — single arena and cross-iteration caches

- **Single cudaMalloc arena (commit `3ef5347a`):** S, Ss2, seven work matrices, Hs2, the dense eigenvectors, and the sparse→dense index/staging are partitioned 512-B-aligned into one reserved region, exposed to OpenACC via `acc_map_data`. If the reservation fails, the path falls back to ELPA (§ 5.6).
- **Per-iteration overhead trimming (commits `a9b6fa31`, `f5de630e`):** a per-device cublas-handle cache (previously 13 create/destroy per iteration); device residency for the transformed S and its spinor expansion Ss2 (saving ~570 MiB of re-upload per iteration at 216 atoms); a persistent zheevdx-workspace context; ~1.6 GiB of host scratch reused across iterations; and the back-transform GEMM shrunk from m=n2 to m=MaxN via the explicit-leading-dimension helper `ClusterNonCol_GEMMul8ZgemmLd_OpenACC`.
- The dense eigenvectors `dense_evec` are cached on the device and distributed later from `DFT.c` via `Cluster_DFT_NonCol_ScatterGpuSolverCachedEVec`; under memory pressure `Cluster_DFT_NonCol_DemoteGpuSolverCachedEVec` demotes them to the host (called when the density-grid GPU service is about to fail its owner allocation; § 7.9).
- MaxN: all states on the first SCF iteration, occupied + margin afterwards (§ 5.3).

7.4 The DC Method — Divide_Conquer.c

- Each rank independently handles its atoms' truncated cluster matrices (local dimension NUM). GPU condition: collinear `NUM ≥ DC_GPU_Threshold()` (default 400; `OPENMX_DC_GPU_THRESHOLD`), noncollinear `2·NUM ≥`

`DC_NonCol_GPU_Threshold()` (default 300; `OPENMX_DC_NONCOL_GPU_THRESHOLD` — the CPU zheev baseline loses to the GPU much earlier than the collinear dsyevx one).

- **The Col side is raw CUDA:** `DCGpuSolverCtx` holds cudaMalloc buffers plus pinned host buffers (`cudaMallocHost` — the only use in the code base). S diagonalization (up to SCF 2) → H′ build (`DC_GpuSolver_TryGpuDgemm`: GEMMul8 → cublasDgemm → CPU) → active subspace cut out on-device with `cudaMemcpy2DAsync` → Xsyevdx.
- **Device-resident S12 cache (S-cache, commit `6f7f31b2`):** the transformed overlap S12 = U·λ^−1/2 is SCF-invariant from iteration 3 onward, so it is parked in a per-rank single-cudaMalloc arena. Admission is polite and greedy within a budget of (free − 4096 MiB reserve) / node ranks — never evicting other phases. On a hit, the whole "host S_DC refill → repack → PCIe upload" is skipped and the GEMM reads the resident copy; the host side is restored lazily by `DCCol_FillSDCFromS12` only if a CPU fallback actually needs it. Disable with `OPENMX_DC_GPU_S_CACHE=0`. Admissions are reported as `<DC> GPU S-cache: rank %d keeps %d of %d cluster overlaps on the device (%.1f MiB)`.
- **Host transposes eliminated:** the strided O(n^2) host transposes were replaced by contiguous-row memcpy packs plus on-device `cublasDgeam` transposes (`DC_GpuSolver_DeviceTranspose`); geam is pure data movement, so downstream inputs are bit-identical.
- **Preflight skipping:** once a GEMM shape has been approved, smaller solves skip the `cudaMemGetInfo` preflight (GEMMul8/cuBLAS workspaces are grow-only, so smaller shapes cannot trigger new allocations).
- **Layered safety:** pre-allocation VRAM preflight (`GPUSOLVER_MIN_FREE_AFTER_MB`, default 1536 MiB), host-malloc failure detection, eigensolve failure detection — any of them raises the **sticky disable flag** and the rank continues on the CPU DC solver (releasing GEMMul8 workspaces). The CPU-fallback helpers pop the CUDA error latch first (§ 3.6).
- **The NonCol side is OpenACC:** S_DC/H_DC/C now use contiguous stores + row-pointer arrays so the data regions transfer one block each, and diagonalization goes through `gpusolver_Syevdx_Complex_openacc_cached` (handle/workspace reused across atoms). Measured: the diagonalization bucket of a 64-atom Si DC run (16 SCF) dropped 78.4 s → 66.4 s.

7.5 The DC-LNO Method — Divide_Conquer_LNO.c

- Current mode is **direct per-rank dispatch**: every rank calls `cudaSetDevice` and submits its own local problems (collinear: NUM ≥ 800 = `GPU_CPU_SWITCH_NUM2`, adjustable via `OPENMX_DCLNO_GPU_THRESHOLD`; noncollinear: spinor dimension ≥ 400 = `DCLNO_NONCOL_GPU_CPU_SWITCH_NUM`, via `OPENMX_DCLNO_NONCOL_GPU_THRESHOLD` — measured separately because the much slower CPU zheev makes the GPU pay off from smaller matrices). The old GPU-proxy machinery (rank 0 solving for its group via MPI tags 41001-41004) is preserved behind `DCLNO_ENABLE_SHARED_GPU_PROXY 0`.
- **Noncollinear solves now on the GPU (commit `d62cac60`):** `DCLNO_Solve_NonCol_GpuSolver` executes the whole per-cluster generalized problem on the device: zheevdx on the NUMH×NUMH S → `ko = 1/√|λ|` → the spin-doubling transform T = diag(Ũ, Ũ) built with `cublasZdgmm` plus a `cudaMemcpy2DAsync` block copy → H′ = T † HT with two GEMMul8 Zgemms → `cublasZgeam` to form the conjugate matrix (the same matrix the CPU flow diagonalizes, for numerical compatibility) → zheevdx for NUM2 states → one transposed Zgemm writing the back-transform directly in the layout the residue stage reads. Enabled when `use_gpu_accel_nc && Eigen_MPI_flag==0 && NUM ≥ threshold`.
- **Group-aggregate memory test (§ 5.5.4):** if `DCLNO_GpuGroupMemoryFits` fails, the whole group uses the CPU eigensolver (both collinear and noncollinear).

- **End-of-phase memory return:** `DCLNO_GpuTurnRelease()` (default 1; opt out with `OPENMX_DCLNO_GPU_TURN_RELEASE=0`) frees the solver buffers and GEMMul8 workspaces at the end of each phase, handing memory back to the grid/force phases (handles are kept).
- The per-node GPU count can be capped by `scf.Gpu.Num` (default 30 ≈ unlimited). Numerical failures abort (Table 5-2). A 216-atom Si NC run (cluster dimension ~2222) takes ~13 minutes at np=2 versus over an hour for the CPU solve.

7.6 The Krylov Method — Krylov.c

- **No OpenACC; raw CUDA only** (compiled with `CC_NOACC_O3`). A per-OpenMP-thread `Krylov_GPU_Workspace` pool (d_A/d_B/d_C/d_W/d_work plus a dedicated non-blocking stream).
- **Fused per-atom pipeline (commit `3ee45228`):** instead of "repack & upload the basis three times per atom + a host round trip per GEMM + a host eigensolve", `Krylov_ProjectedSolve_GPU` fuses one device visit per atom: ① fetch the basis d_KU from the KU cache (repack & upload only on a miss); ② upload H_DC once; ③ C = H·u1 (GEMMul8); ④ u1 † Hu1 (GEMMul8; only the m3×m3 result downloads); ⑤ host-side ZeroNum correction and EC_matrix addition (identical to the CPU code); ⑥ `cusolverDnXsyevd`, leaving the eigenvector panel on the device in exactly the layout the back-transform reads (on failure: LAPACK fallback + re-upload); ⑦ back-transform c = u1·b (GEMMul8) → download.
- **Engagement condition:** all three GEMMs must satisfy $m \cdot n \cdot k \geq 5\times10^{7}$ (`OPENMX_KRYLOV_GPU_DGEMM_MIN_FLOPS`) — exactly the same GEMMs that would go to the GPU on the per-call path, so the GEMM dispatch is **bit-identical** to the old code. The device eigensolve engages at `Msize3 ≥ 300` (`OPENMX_KRYLOV_GPU_EIGEN_MIN`; cusolver and LAPACK eigenvectors differ in the last bits, so such runs are not bitwise-reproducible against the CPU eigensolve — in the crossover benchmarks Uele agreed to $\leq 2\times10^{-9}$ up to Msize3=455 and to 2×10^{-6} at Msize3=715). Disable the fused path with `OPENMX_KRYLOV_GPU_FUSED=0`.
- **KU cache:** the repacked Krylov basis panels (created at SCF 1, invariant afterwards) are parked in a per-rank device arena, admitted politely within (free − 4096 MiB) / node ranks (`OPENMX_KRYLOV_GPU_KU_RESERVE_MB`); disable with `OPENMX_KRYLOV_GPU_KU_CACHE=0`. Reported as `<Krylov> GPU KU-cache: rank %d keeps %d of %d basis panels on the device (%.1f MiB)`; released by `Krylov_Release_GPU_KUCache()` just before the force phase.
- The S transform is handled during Krylov basis construction (CPU `Inverse_S_by_Cholesky`), so the GPU only sees standard eigenproblems. `OPENMX_KRYLOV_GPU_PROFILE=1` prints the fused-path breakdown (`KRYFUSE n=⋯ ku=⋯ g1=⋯ eig=⋯`).

7.7 Matrix-Element Construction — Set_Hamiltonian.c / set_hamiltonian_gpu.cu / Set_Nonlocal.c / Set_OLP_Kin.c

Set_Hamiltonian.c (+2,972/−1,149 vs. original) + set_hamiltonian_gpu.cu (194 lines)

As of the 2nd edition, "the OpenACC grid integrals exist but are permanently disabled because they were slower". Since commit `95fa51ed`, the dVH+Vxc+VNA grid integrals (`Calc_MatrixElements_dVH_Vxc_VNA`) run **on the GPU by default** (`OPENMX_SETHAM_MATRIX_GPU`, default 1). Four mechanisms made the difference:

1. **SCF-invariant cache:** pair metadata, grid indices (nolg_MN/nolg_Nc), and the orbital-value buffers (orbs0buf/orbs1buf, float) are built once per SCF cycle on the host (`SetHamiltonianMatrixElementsCache`). Per iteration only the H blocks (hbuf) and the rank-local Vpot slice are transferred.

2. **On-device Vpot gather (commit `bff0af37`):** the old host-side spin×overlap-point expansion is gone; the rank-local `Vpot_Grid` slice is transferred as is and gathered indirectly in the kernel via `nolg_MN`. The rounding order matches the old expansion — bit-identical.
3. **A dedicated CUDA kernel (§ 6.4):** the tiled kernel `matrix_elements_kernel` with a weighted-orbital tile (double: GridVol·Vpot·Orbs0) and an orbs1 tile (float) in shared memory. `OPENMX_SETHAM_CUDA_KERNEL=0` switches to the numerically identical OpenACC kernel.
4. **Adaptive GPU turn planning (`Set_Hamiltonian_CreateGpuTurnPlan`):** the same UUID group + descending prefix scan as § 5.5. The default is a **hybrid wave**: ranks in the first wave compute on the GPU while the remaining ranks compute on the CPU concurrently (`OPENMX_SETHAM_GPU_SERIAL_WAVES=1` switches to all-GPU serial waves). By default **every rank holding a local H segment is a GPU work candidate** (the old k-owner-only policy returns with `OPENMX_SETHAM_GPU_KOWNER_ONLY=1`).

The resident cache and yielding to published needs (commits `bff0af37`, `69dde552`): only when all ranks sharing the device run in a single wave, the SCF-invariant tables (four classes in descending size: orbs1 / orbs0 / nolg / meta) are made device-resident (`OPENMX_SETHAM_GPU_RESIDENT`, default 1). The admission budget is "free − (published phase needs) − floor", where the floor is 4 GiB once a diagonalization need has been published and 16 GiB before that (`OPENMX_SETHAM_GPU_RESIDENT_FLOOR_MB` overrides both). The cache re-checks the needs published by the band solvers (`OpenMX_GpuPhaseNeed_Register("band_…")`) on every call, and when free memory falls below the need it **evicts itself right before the same iteration's diagonalization**, staying transient for the rest of the MD step. Since the `copyin` clauses have present_or_copyin semantics, numbers are bit-identical whether the tables are resident, partially resident, or absent. Related stdout:

```
<Set_Hamiltonian>  Hamiltonian matrix for VNA+dVH+Vxc (GPU-accelerated)...
<Set_Hamiltonian> GPU device %d: %d Hamiltonian rank(s), GPU concurrency=%d,
    serialized waves|CPU fallback ranks=%d, peak=%.3f GiB, free=%.3f GiB, reserve=%.3f GiB
<Set_Hamiltonian> GPU resident cache: %d rank(s) hold %.3f GiB (orbs1+orbs0+nolg+meta) on device %d across SCF iterations
<Set_Hamiltonian> GPU resident cache released: a GPU phase needs %.3f GiB on device %d; staying transient for this MD step
```

The phantom-failure fix (commit `c5eb6845`): on a nearly full device, allocation failures that other components recover from internally (GEMMul8's FP64 fallback, `wait_cudafunc` retries) stayed latched in CUDA's per-thread error latch, and the CUDA kernel's error check mistook them for its own failure and aborted ("CUDA matrix-elements kernel failed with status -2" at SCF iteration 316). The kernel now pops stale errors before launching, and on failure checks the context with `cudaDeviceSynchronize()`: if healthy, it reports a recoverable status 2 and the caller **redoes the same batch with the OpenACC kernel** (restoring the possibly half-updated device H block from the intact host copy first). Only context-corrupting errors abort.

- **The base combination (H = F_Kin·H0 + …)** stays opt-in on the GPU (`OPENMX_SETHAM_BASE_GPU`; measured 10 ms vs. 2 ms on the CPU).
- **The packed H/S cache** (the API family `Set_Hamiltonian_Build_GpuSolver_HS_Cache` etc., keeping H and S in a 1-D packed form for GPUSOLVER) continues to feed the dense assembly of the band/cluster solvers; the cluster two-spin serialized mode (§ 7.3) requires it.
- **Sharing the tables:** the distributed density-grid quadrature (§ 7.9) reads the same orbital-value tables through the read-only view `Set_Hamiltonian_GetMatrixElementsTables()`; `Set_Hamiltonian_MatrixElementsTables_Ready()` probes for existence **without triggering the multi-GiB lazy build** (commit `ae3dd189`).
- Profiling: `OPENMX_BAND_PROFILE=1` prints `SETHPROF` / `SETHMEPROF` lines.

Set_Nonlocal.c (+1,738/−305 vs. original)

The 2nd edition's "OpenACC-offloaded radial k integrals" were replaced in commit `211f0684` by a **fully batched GPU evaluation of the nonlocal-projector integrals** (`SetNonlocal_CalcDSNL_OpenACC`), in three phases:

1. **Host setup:** per-species radial tables (Fourier transforms of bases and projectors) and the common k grid are built with OpenMP; the batch unit is a "row" = (atom pair, derivative direction so, angular momentum LL, basis×projector combo). To avoid re-evaluating Wigner-3j factorial sums tens of millions of times, the **Gaunt coefficients are tabulated up front** (the selection rule m = M0−M1 removes one axis).
2. **Device execution:** the spherical Bessel tables are **generated on the device** (a faithful port of the host downward recurrence, rescaling, and normalization — eliminating the old ~21 s serial host Bessel generation plus chunked transfers), and the trapezoidal radial integrals are reduced in batched gang×vector kernels. Pairs are chunked so the Bessel tables stay within a 96 MiB device budget.
3. **Host finish:** spherical harmonics, Gaunt assembly, and the complex→real transforms follow the CPU-path structure with OpenMP over pairs.

The only fallbacks are `OPENMX_SETNL_OPENACC=0` (or a non-GPUSOLVER run); there is no dynamic memory fallback. Measured (216-atom Si NC, np=2): the DS_NL stage went 23.7 s → 0.66 s; the first 8 SCF steps of a methane CPU comparison are bit-identical. The hardening that replaced fixed-size stack arrays with overflow-checked heap allocations is retained.

Set_OLP_Kin.c (+528/−180) and Spherical_Bessel.c

- The radial-integral offload for overlap/kinetic matrix elements remains **opt-in** (`OPENMX_OLPKIN_RADIAL_GPU=1`) because it is transfer-bound and slower in practice (comment: ~1.2 s vs. 0.3 s); it also requires `Nthrds==1`.
- The effective CPU-side speedups remain: the spherical-Bessel table cache keyed by the exact bit pattern of the pair distance r (bitwise-identical results, single-threaded only) and the removal of per-call malloc/free from the `Spherical_Bessel.c` hot path.

7.8 VNA Projectors — Set_ProExpn_VNA.c

In the 2nd edition this file had been "reverted to the CPU version, no diff against the original". Commit `77d2a22c` added a **full GPU batching of +2,029 lines** (216-atom NC: 160.9 s → 4.4 s). The common gate is GPUSOLVER plus `OPENMX_SETPRO_GPU` (collective, default 1). Three independent batches, each with its own memory test and CPU fallback:

1. **DS_VNA construction batch:** all pairs of the one-center integrals <ϕ|P_VNA> in 256-pair chunks, four kernels — ① device spherical-Bessel recurrences (a faithful port of `Spherical_Bessel2`), ② Gauss-Legendre quadrature sums, ③ Gaunt × Y_lm weighted (LL,m) sums, ④ complex→real transform and float store. The factorial-based Gaunt coefficients and long-double complex spherical harmonics are **tabulated on the host**; the device only reads tables. Quadrature keeps the CPU's element-wise accumulation order. Memory condition: (free − 256 MiB)/node ranks > 1 GiB.
2. **HVNA trace batch:** the energy-weighted float dot products HVNA[Mc][j] = dmp·Σ ene·DS_VNA·DS_VNA are evaluated in one kernel over all (Mc, j) pairs after archiving the halo blocks received through the unchanged MPI pipeline. Damping and stores stay on the host, exactly as the CPU loops do.
3. **Set_VNA2 / Set_VNA3 batch:** <Φ|VNA> and <VNA|Φ> are processed by the same kernels as mirror images (swapping the PAO-product species and the sign of the direction vector), in 512-pair chunks.

The insufficient-memory notice is printed once per node at `level.of.stdout` 2 (commit `b569f272`; level 3 prints every rank's own figures, § 10.3): `Set_ProExpn_VNA GPU: not enough free device memory; CPU fallback.` The DFT.c stage banner reads `<MD=···> Calculation of the VNA projector matrix (GPU-accelerated)` (commit `30148e1e`).

The free memory these tests divide among the node's ranks — like every other device-memory budget of the GPU paths — comes from `OpenMX_GpuMemGetInfo()`, i.e. `cudaMemGetInfo()` capped at the physically free memory NVML reports (§ 10.2, `OPENMX_GPU_MEMINFO_NVML`). The cap was added after 12 ranks on a 12 GB RTX 4070 Ti under WSL2 stalled here: the WDDM driver showed every rank 10.8 GiB "free" although the twelve contexts had left 0.6 GiB, the DS_VNA batches of 836 MiB per rank were paged through system memory, and one batch took anywhere from 0.4 s to several minutes. With the physical figure the batch shrinks to 128 pairs (423 MiB) and takes 2.7 s. The HVNA2/HVNA3 batch, which used to be admitted against a fixed 512 MiB, now sizes its admission the same way: the footprint of its data region (`SetPro_VNA23_DeviceBytes`, 36 MiB for Si s2p2d1 at the full 512-pair batch) against the rank's share of the free memory, the pair batch halved until it fits. `OPENMX_SETPRO_VERBOSE=1` prints these admission figures and the elapsed time of each batch per rank.

7.9 Density Grid — Set_Density_Grid.c + Set_Density_Grid_GPU.c

Building the electron density on the grid (ρ(r) = Σ DM·ϕϕ) was GPU-offloaded in the new file `Set_Density_Grid_GPU.c` (1,366 lines). It runs as a **three-stage fallback chain** (documented in a Set_Density_Grid.c comment): **distributed GPU quadrature → one-rank GPU service → CPU/OpenMP loop**. Common enabling conditions: GPUSOLVER and `Cnt_switch==0` (`OPENMX_DENSITY_GRID_GPU=0` disables everything). The distributed quadrature takes the Cluster and Band solvers and, since 2026-10-09, DC, Krylov and DC-LNO as well (every solver hands Set_Density_Grid the same DM over the FNAN neighbours, so the quadrature is solver-independent); the one-rank service, which builds the whole grid's orbital tables on one device, stays with Cluster and Band.

(a) Distributed GPU quadrature (commit `d14ef297`)

- Each rank integrates its own atoms' atomic-sphere densities `Tmp_Den_Grid` on the device and feeds the existing A→B communication (no owner serialization). The per-output-point CSR is walked in the same (h_AN, Nog) order as the CPU loop, with the same summation order.
- **Sharing the Set_Hamiltonian tables:** the orbital-value tables and grid indices come from the read-only view `Set_Hamiltonian_GetMatrixElementsTables()`; where those tables are resident, the present_or_copyin `copyin` merely attaches — per iteration only the CSR, the density matrix, and the results move.
- Mode (`OPENMX_DENSITY_GRID_GPU_LOCAL`): 0=off; 1 (default)=engage **only when the shared tables are fully device-resident** (staging them per call is slower than the owner service); 2=always. In mode 1, an Allreduce(MIN) over `Set_Hamiltonian_MatrixElementsTables_Ready` ensures every rank already built its tables, so CPU-executing ranks are never forced into multi-GiB host builds (commit `ae3dd189`). A per-node quota — (free − 256 MiB)/node ranks — must also hold.
- Participation is agreed by Allreduce(MIN); the first engagement prints (at `level.of.stdout` 2 and above) `<Set_Density_Grid> distributed GPU quadrature: every rank integrates its own atoms`. Its transient demand is published as `OpenMX_GpuPhaseNeed_Register("density_grid_local", ···)`.
- **On-the-fly ranks and their workspace (2026-10-11):** ranks without resident orbital tables evaluate their orbitals on the device for each batch. Full central-atom tiles are reused across pairs when they fit. If a pair exceeds the workspace, both atoms are evaluated only at bounded chunks of its overlap points. The chunks keep the original point order and accumulate into the same H blocks or density points; a recoverable matrix-kernel failure computes only that chunk on the CPU. Compact orbital evaluation and the OpenACC density

fallback distribute points across GPU gangs, so one pair can use the whole device. Large atoms therefore no longer require a whole sphere tile to fit (Eu: 41 orbitals, 8.0 bohr; a full self-pair can exceed 500 MiB).

- The default density workspace is 128 MiB. For large atoms, preparation still grows it when spare device memory permits, up to all of the rank's pairs in one batch. This retains the earlier Eubcc8-Band improvement (2041 MiB, one batch, 0.11 s per density step). If growth cannot hold a full pair, bounded chunks keep the rank on the GPU. An explicit `OPENMX_SETDEN_ONTHEFLY_MB` remains a hard cap and also permits chunks. Fixed density/output storage and runtime reserves are budgeted separately; a nominal 48 MiB floor never overrides the physical free-memory share.
- If workspace pressure still prevents admission, declined ranks run their CPU loops concurrently with the remaining GPU ranks. Even if all ranks decline for lack of workspace, they keep this distributed route instead of gathering onto one GPU. The one-rank service remains available when no rank supports distributed execution, including `Cnt_kind` 1 or explicitly disabled local quadrature.
- At `level.of.stdout` 2, Host_ID reports workspace declines or device failures once when the situation changes, including the all-declined case; level 3 also prints the individual rank's requirement and available payload. Normal execution and designed service routes remain silent.

(b) One-rank GPU service (commit `bff93d08`)

- The owner (Host_ID) gathers, once per geometry, all ranks' atom-centered orbital grids and overlap phase tables via `MPI_Gatherv` and caches them. In subsequent SCF iterations only **the CDM** is gathered; the owner builds the density on the whole regular grid with a CSR kernel and scatters each rank's contiguous B section via `MPI_Scatterv`.
- When the owner's allocation is about to fail, room is made in stages: first all ranks release GEMMul8 workspaces and return the OpenACC pools; then the noncollinear cluster eigenvector cache is demoted to the host (`Cluster_DFT_NonCol_DemoteGpuSolverCachedEVec`). If still short, it prints `Set_Density_Grid GPU owner needs … ; CPU fallback.` and computes that epoch on the CPU.
- By default the device side is freed right after the computation (`OPENMX_DENSITY_GRID_GPU_PERSISTENT=0`, to keep GEMMul8's `cudaMemGetInfo` decisions honest). Reserve: `OPENMX_DENSITY_GRID_GPU_RESERVE_MB` (default 256 MiB).

7.10 Total Energy — Total_Energy.c + Total_Energy_GPU.c

Why two files: NVHPC 26.3's `-acc` miscompiles the LDA+U paths, so `Total_Energy.c` itself must be built without `-acc` (`CC_NOACC`); the OpenACC kernels live in the separate translation unit `Total_Energy_GPU.c` (built with `-acc`; now 1,064 lines).

The gate is `TotalEnergyUseOpenACC()`. The 2nd edition's restriction "disabled for noncollinear (SpinP_switch==3)" was **removed by commit `19b9b69c`**; the kernels now run whenever GPUSOLVER is active (`OPENMX_TOTALENERGY_GPU=0` disables them all at once). Five grid reductions are offloaded:

1. **EXC/EH1 grid integrals** (Ena, Eef, EH1, EXC in a single `parallel loop reduction`)
2. **The six dipole-moment components** (grid coordinates GN→(n1,n2,n3) reconstructed on device)
3. **The CWF double-counting correction** (dcEH1/dcEXC)
4. **The EH0 two-center batch** — all atom pairs flattened at once, per-species spherical grids, VPS meshes, and V_H atomic tables transferred flat; `gang` (pair) × `vector reduction` (grid point) computes energies and r derivatives (forces #6/#7) together; spline interpolation as `#pragma acc routine seq` device functions.

5. **The Exc0 fine-mesh correction batch (new, commit `19b9b69c`)** — with `Exc0_correction_flag==1`, the previously serial host loop over the Lebedev spherical grid × coarse radial mesh is replaced by a two-pass GPU batch (`TotalEnergy_Exc0_Batch_OpenACC`): ① a point kernel over all local atoms × radial × Lebedev points reduces the energy while storing per-point force prefactors device-resident; ② an (atom, neighbor) pair kernel contracts the prefactors with neighbor density gradients to produce the force-#8 contributions. The XC functional (Ceperley-Alder) and the spline interpolators (KumoF/Dr_KumoF) are reimplemented as device functions.

Total_Energy_GPU.c — the typical grid reduction

```
#pragma acc data copyin(Density_Grid_B0[0:grid_count], Density_Grid_B1[0:grid_count], \
                        ADensity_Grid_B[0:grid_count], PCCDensity_Grid_B0[0:grid_count], \
                        PCCDensity_Grid_B1[0:grid_count], dVHart_Grid_B[0:grid_count], \
                        Vxc_Grid_B0[0:grid_count], Vxc_Grid_B1[0:grid_count], \
                        RefVxc_Grid_B[0:grid_count], VNA_Grid_B[0:vna_grid_count], \
                        VEF_Grid_B[0:vef_grid_count])
  {
#pragma acc parallel loop reduction(+:local_Ena,local_Eef,local_EH1,local_EXC0,local_EXC1)
    for (BN=0; BN<grid_count; BN++){
```

7.11 Forces — Force.c

Force evaluation changed more than anything else since the 2nd edition (+15,123/−11,241 vs. original). Then: "OpenACC reductions for #2/#4/#5; #4B kept on the CPU to avoid VRAM exhaustion". Now **all major traces including #3, #4B, and HNL are GPU-batched**, and in exchange an arena + peer-wait machinery guarantees that shared-device memory races can never abort the run. The parent gate is `OPENMX_FORCE_GPU` (collective, default 1).

Stage	Implementation	Gate / fallback
#1 (overlap correction)	host only	—
#2 (kinetic)	pack + GPU reduction (`Force2_Kinetic_OpenACC`)	when no SO/LDA+U/constraints/Zeeman; on allocation failure computes on the host
#3 (dn/dx·Vpot grid trace)	chunked batch `Force3_GpuTrace`: one pair = one gang, vector reduction over overlap points. **dOrbitals recomputed on device** (explicit real spherical harmonics for L0≤3 + radial splines; eliminates ~4.5 GB/rank of PCIe traffic)	budget test (residency + 192 MiB within (free − reserve)/node ranks; the reserve, 256 MiB plus 16 MiB per node rank, is the one of every force-phase budget). If only the dOrbitals recomputation is infeasible, falls back to transferring host-computed dchi (`OPENMX_FORCE3_GPU_DORB`)
#4 (ProExpn_VNA==0)	GPU reduction of the dVNA grid term (`Force4_OpenACC`)	host on allocation failure
#4B (projected VNA)	**GPU dot-product batch** (commit `7d38382d`): the four DS_VNA directions device-resident in float (direction-major); halo/case-2 blocks archived from the MPI pipeline; fused case-1/case-2 traces each in one kernel, reproducing the CPU down to the float-multiply→double-add cast. Damping tails on the host	`OPENMX_FORCE4B_GPU` (default 1) + memory test. If not: the **fused CPU trace** (`OPENMX_FORCE4B_FUSED`, commit `fe68cca6`); if that too is off, the original dHVNA path. Special pairs (q_AN==0 etc.) always on the host
#5 (overlap × EDM)	GPU reduction (`Force5_OpenACC`)	host on allocation failure

Stage	Implementation	Gate / fallback
HNL (nonlocal)	GPU dot-product batches (case-1/case-2); DS_NL stays double; case 2 handled entirely by the GPU	`OPENMX_FORCEHNL_GPU` (default 1). Scalar-relativistic traces only (NC with SO/LDA+U/constraints/Zeeman, or j-dependent VPS, stays on the CPU)
#6/#7 (EH0), #8 (Exc0)	computed inside the Total_Energy GPU batches (§ 7.10)	`OPENMX_TOTALENERGY_GPU`

Table 7-1: GPU status of the force stages

Arena + peer-wait machinery (commits `206b7e06`, `d0e1d17a`): on a shared device the ranks drift apart between stages, so "free memory at batch start" goes stale, and a lagging rank's `acc data copyin` hitting a full device made the OpenACC runtime abort the whole run. The fix removes the fatal window itself rather than dodging the timing:

- All allocations packed into single 512-B-aligned **arenas** (`Force_gpu_arena_off`); kernels read through same-named shadow pointers with `deviceptr` (kernel bodies — hence numbers — unchanged).
- Allocations with a host fallback (reduction buffers) use `Force_gpu_arena_try`: on failure it returns NULL and the reduction is computed on the host (notice once per rank: `Force: a %.1f MiB GPU reduction buffer does not fit …; using the host for this reduction.`).
- Mandatory allocations (the batches) use `Force_gpu_arena_wait`: retry every 50 ms while peers release memory (one warning after 2 s); after 180 s, an MPI_Abort with a diagnostic including the amounts. Peers' transient allocations always drain, so there is forward progress; one region = one arena, so a waiting rank holds nothing and mutual deadlock is impossible.
- **No force-stage device allocation can abort the run** (commit `d0e1d17a` converted the last two data-clause allocations to arenas). The Force() entry and each large-arena release flush the OpenACC pool back to CUDA (`Force_gpu_pool_flush`).
- Chunk buffers are allocated once at the per-chunk maximum and reused — the countermeasure to the NVHPC OpenACC pool's same-size-only reuse (§ 6.3, commit `28361f7e`).

Also: the Force4B/HNL/ProExpn "not enough memory, CPU fallback" notices print once per node at `level.of.stdout` 2 (every rank's figures at 3) (commit `b569f272`). `OPENMX_FORCE_PROFILE=1` prints per-stage `[Force profile]` lines (max/avg seconds). Commit `098adddc` reworked the Force3/4B staging from host-resident mirrors to device-resident buffers with one-atom staging, fixing multi-GB/rank host-memory consumption and a budget-accounting bug. Stage banners gain `(GPU reduction)` / `(GPU-accelerated)` suffixes when the GPU is used (§ 10.3).

7.12 Charge/DM Mixing Fast Paths — Mixing_H.c and 5 More Files

SCF mixing (`scf.Mixing.Type`) was pure CPU through the 2nd edition. Commits `94b6ca8c` / `6e94a9d3` added fast paths to every scheme, all enabled **only under GPUSOLVER** (`OPENMX_MIX_FAST=0` disables the CPU fast paths). The legacy code remains intact in else branches; Simple mixing and the NEGF variants are untouched.

RMM-DIISH (Pulay_Mixing_H) — Two Tiers (commit `94b6ca8c`; Mixing_H.c +1,350 lines)

Motivation: the residual Gram matrix swept the sparse H structure $\dim^2$ times and issued **one MPI_Allreduce per matrix element** (with History 50 on Fe fcc 5×5×5, np18: 2,415 s of a 7,352 s DFT run). `MixH_GpuPreflight` (collective; sticky while its fingerprint holds) picks one of two tiers:

- **Tier A (GPU-resident, `Pulay_Mixing_H_GPU`):** a per-rank single-cudaMalloc arena holds the ring of residual slots, the metric weights, the optimal residual, and the Gram pair buffer. History shifts are a **rotation of the logical→physical slot map (zero copies)**; the host residual history rotates pointers too, with the new residual computed simultaneously into slot 0 and its flat image. The whole weighted Gram triangle is computed by one OpenACC kernel and reduced with **a single MPI_Allreduce**. Metric, α staircase, and nan checks are identical to the legacy code. Engaged only when the device-sharing group's total arena fits.
- **Tier B (incremental CPU, `Pulay_Mixing_H_Inc`):** the Gram matrix is decomposed exactly into metric-independent block partial sums over (atom, orbital-row); since pointer rotation keeps old entries bitwise valid, a continuing iteration computes **only the new row** in O(dim · L) (legacy: O(dim² · L)). No device memory needed.
- Control: `OPENMX_MIXH_GPU` (unset=auto, 0=legacy, 1=force Tier A, 2=force Tier B), `OPENMX_MIXH_GPU_RESERVE_MB`, `OPENMX_MIXH_GPU_VERBOSE`. When Tier A does not fit, a single notice says "using the incremental CPU Pulay mixing instead". The preflight **deliberately does not call `acc_clear_freelists()`** (§ 6.3).
- Measured: Fe fcc 5×5×5, 100 iterations — Mixing_DM 504.6 s → 47.3 s (10.7×); full run wall time −25%; the converged Utot agrees to all printed digits.

The Remaining Schemes (commit `6e94a9d3`) — CPU Fast Paths, No GPU

Scheme (file)	What was accelerated
RMM-DIISK (`DIIS_Mixing_Rhok.c`) / RMM-DIISV (`Mixing_V.c`)	**Incremental Gram:** the stored residual columns are pre-weighted and only slide between iterations, so the Gram matrix slides one step along the diagonal and only the new residual row is computed by dgemv, with **one short Allreduce**. History shifts are pointer rotations plus one restoring copy (bit-identical to the legacy shift). A broken continuation (SCF rewind etc.) reseeds via the legacy computation
RMM-DIIS (`DIIS_Mixing_DM.c`) / GR-Pulay (`GR_Pulay_DM.c`)	**Batched Gram:** all pairs computed in one sparse-structure sweep (OpenMP) + **one Allreduce** (legacy: one sweep and one Allreduce per pair — up to 1,275 per iteration at History 50). The optimal residual and the mixing sweep are fused into one pass. Residual histories rotate pointers; **the DM/iDM history shift deliberately stays a legacy content copy** — other GPU modules may hold device-side state tied to the DM buffers, so the DM slot pointers must never rotate (stated in a source comment)
Kerker (`Kerker_Mixing_Rhok.c` / `Mixing_V.c`)	Only the history shift is accelerated (rotation + memmove) — it runs every pre-StartPulay iteration under RMM-DIISK/V, where the Rhok history is List_YOUSO[38] slots deep, so it is not negligible

Verification (216-atom Si, np8): Kerker bit-identical in every output line; RMM-DIISK bit-identical in the final Uele; the other schemes agree to 1e-10–1e-12. Fe fcc 5×5×5 (rmm-diisk, 40 iterations, np18): Mixing_DM 33.5 s → 20.0 s.

7.13 Derived Solvers — Polarization / DMmu / CWF / LNO (11 Stereotyped Files)

Eleven files share the same stereotyped pattern: include `openmx_gpusolver_dense_utils.h`, device assignment when GPUSOLVER is requested, a static context, and the insertion — **immediately before the ELPA2 branch** — of `scf_eigen_lib_flag==GPUSOLVER && GPU_CPU_SWITCH_NUM<=n && na_rows==n && na_cols==n` (diagonalization via `OpenMX_GpuSolver_DenseDsyevx/DenseZheevx`). If the condition fails, the original ELPA/ScaLAPACK flow runs.

File	GPU diagonalizations inserted
`Band_DFT_Col_CWF.c`	Zheevx × 3 (S, H, CWF bands)
`Band_DFT_Col_DMmu.c`	Zheevx × 4
`Band_DFT_NonCol_CWF.c`	Zheevx × 3

File	GPU diagonalizations inserted
`Band_DFT_NonCol_DMmu.c`	Zheevx × 4
`Cluster_DFT_Col_CWF.c` / `Cluster_DFT_Col_DMmu.c` / `Cluster_DFT_Col_LNO.c`	Dsyevx × 2 each
`Cluster_DFT_NonCol_CWF.c` / `Cluster_DFT_NonCol_DMmu.c`	Dsyevx × 1 + Zheevx × 1 each
`Electric_Polarization_BandCol.c`	Dsyevx × 1 + Zheevx × 4 (DM, overlap, H, ABC)
`Electric_Polarization_BandNonCol.c`	Zheevx × 5 (plus an ELPA1-side fallback-condition adjustment)

Table 7-2: Files carrying the stereotyped GPUSOLVER branch

7.14 Generic Eigensolvers — Eigen_lapack.c / EigenBand_lapack.c

- `Eigen_lapack()` (real symmetric) branches to `Eigen_gpusolver_x` (→ `gpusolver_Syevdx`) when `flag != ELPA1/2 && n ≥ 1000`. **Its 20+ callers (orbital optimization, DC, LNO, …) benefit automatically.** The caller retries with an eigenvector-orthogonality check, cycling through CPU solvers (up to 3 attempts).
- `EigenBand_lapack()` (Hermitian) branches to `gpusolver_zheevx` at N ≥ 1000, falling back to the CPU Householder solver `Eigen_HH` on failure.
- Both offer `_openacc` entries operating directly on OpenACC-resident arrays (used by the noncollinear DC path etc.); the 2-D ⇄ flat device repacking also happens on device.

8. Changes Unrelated to GPU Offloading

8.1 OpenMP Bug Fixes — Heap Corruption from Missing private Clauses

In the original OpenMX 4.0, `Stress.c` and `Total_Energy.c` omit the loop variable `i` from the `private()` clause of a `#pragma omp parallel`, so **multiple threads race on an implicitly shared `i`**. When one thread advances `i` right after another evaluated its loop bound, out-of-bounds array accesses occur, surfacing as heap corruption (broken malloc metadata). The bug was identified as the cause of runtest crashes during GPU development and fixed (commit `d451eeb8`).

Stress.c — the fix (adding `,i` to the private list; Total_Energy.c is analogous)

```
/* before (original) */
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(...),sumt1,sumt2,sumt3,sumt4,sumt5)

/* after (GPU version) */
#pragma omp parallel ... private(OMPID,Nthrds,Nprocs,Mc_AN,(...),sumt1,sumt2,sumt3,sumt4,sumt5,i)
```

- `Stress.c`: the race is in `for (i=0; i<9; i++){ sumt1[i]=0.0; ⋯ }` inside the parallel region (executed per atom). The file's entire diff is this one line.
- `Total_Energy.c`: the Lebedev spherical-grid section of `Calc_EXC_EH1()`; the racing loop indexes `Dr[i][ir][ia]` with `i` up to `FNAN[Gc_AN]`, so a race can point outside the allocation (the heap-corruption path).

> **Significance:** both bugs exist in the original OpenMX 4.0 and can strike any OpenMP-threaded run (≥2 threads) independent of GPUs — the fixes are worth applying to CPU builds too.

8.2 Memory Management and Hardening

File	Change
`openmx.c`	Calls `Raise_Stack_Limit()` right after startup, raising the soft stack limit to the hard limit (`getrlimit/setrlimit(RLIMIT_STACK)`) — protection against crashes from large stack arrays where `ulimit -s` is unset
`truncation.c` / `Free_Arrays.c`	NULL assignment after freeing `VNA_Grid` / `VNA_Grid_B` / `VEF_Grid` / `VEF_Grid_B` (double-free protection). truncation also calls the Set_Hamiltonian / Set_Density_Grid GPU-cache invalidation hooks (geometry changes stale the resident tables)
`Set_Nonlocal.c`	Fixed-size arrays replaced by overflow-checked heap allocations (§ 7.7)
`Free_Arrays.c`	At exit (wherefrom==0): GPU-cache invalidation (`Set_Density_Grid_GPU_Invalidate` / `Set_Hamiltonian_Invalidate_OpenACC_MatrixElements_Cache`) plus `openmx_gemmul8ReleaseWorkspaces()`

openmx.c — Raise_Stack_Limit (complete)

```
static void Raise_Stack_Limit(void)
{
#ifdef RLIMIT_STACK
  struct rlimit limit;

  if (getrlimit(RLIMIT_STACK,&limit)!=0) return;
  if (limit.rlim_cur==RLIM_INFINITY) return;
  if (limit.rlim_max==0 || limit.rlim_cur==limit.rlim_max) return;

  limit.rlim_cur = limit.rlim_max;
  setrlimit(RLIMIT_STACK,&limit);
#endif
}
```

8.3 Other Fixes

File	Change
`Runtest.c`	(1) Input-file detection tightened from a `strstr` substring match to the suffix check `has_dat_suffix()` (no more false hits on `foo.dat~` etc.). (2) Stale `<System.Name>.out` files are deleted before each test (with an MPI_Barrier), preventing contamination by previous results
`Spherical_Bessel.c`	Per-call malloc/free of the recurrence coefficient arrays removed from the hot path; coefficients computed inline (identical formulas)
`openmx_common.h`	Include guard added (the original had none); version string changed from "4.0" to "4.0 GPU"
`Input_std.c`	GPU-related keywords added (Chapter 10)
`mpao.c`	Only a one-line `#include <ctype.h>` difference (a utility outside the build; no impact)

9. Build System

Sections 9.1–9.3 and 9.5–9.6 retain the native CUDA build discussion and historical examples. For current builds of either backend, start with § 9.7; both use the shared ELPA described in § 9.4.

9.1 Complete Toolchain Replacement

The original `makefile` (Intel oneAPI + MKL) was rewritten as `Makefile` (NVHPC + CUDA + in-tree FFTW + GEMMul8). `makefile.cpu_intel.bak` was originally a byte-identical backup; its ELPA section has since been updated (§ 9.4).

Item	Original (makefile)	GPU version (Makefile)
C compiler	`mpiicx -O3 -xHOST -qopenmp`	NVHPC-bundled OpenMPI `mpicc` (nvc) + `-acc -Minfo=accel -std=c11 -mtune=native -march=native -fopenmp`
Fortran	`mpiifx`	`mpif90` (nvfortran) `-O2 -fopenmp`
BLAS/LAPACK	MKL (static)	NVHPC static `liblapack_lp64.a` / `libblas_lp64.a`
ScaLAPACK	MKL ScaLAPACK	HPC-X OpenMPI's `libscalapack_lp64.a`
FFT	MKL FFTW wrappers	static FFTW 3.3.11 built in-tree (third_party)
GPU libraries	—	`-lcudart -lcusolver -lcublas -lcublasLt -lcuda -lnvidia-ml -lnvJitLink -cuda` + `libgemmul8.a`
CUDA C++	—	nvcc compiles `gemmul8_openmx.cu` / `set_hamiltonian_gpu.cu` (host compiler nvc++)
Current SDK defaults	Intel oneAPI	`NVHPC_ROOT ?= /opt/nvidia/hpc_sdk/Linux_x86_64/26.9`, `NVHPC_CUDA_VERSION ?= 13.4` (overridable variables)

Build-environment isolation: the Makefile pins `PATH` to NVHPC only and `unexport`s 30+ environment variables (`CPATH` / `LD_LIBRARY_PATH` / `CC` …). User CFLAGS etc. deliberately have no effect.

Makefile — compiler definitions (excerpt)

```
COMMON_INCLUDES = $(FFTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include -I$(NVHPC_CUDA_TARGET)/include -I$(NVHPC_CUDA_HOME)/include
COMMON_CFLAGS   = -w -Minfo=accel -acc -std=c11 -mtune=native -march=native -fopenmp $(COMMON_INCLUDES)

CC  = $(MPICC) $(COMMON_CFLAGS) -O3
CC2 = $(MPICC) $(COMMON_CFLAGS) -O1
CC3 = $(MPICC) $(COMMON_CFLAGS) -O2
CC_NOACC = $(filter-out -Minfo=accel -acc,$(CC2))
CC_NOACC_O3 = $(filter-out -Minfo=accel -acc,$(CC))
CC_DFTD3 = $(MPICC) -w -O0 -std=c11 -fopenmp $(COMMON_INCLUDES)
FC = $(MPIF90) -O2 -mtune=native -march=native -fopenmp $(FFTW3_INCLUDE) -I$(NVHPC_MATH_DIR)/include
```

Where the original used one `$(CC)` for everything, the GPU version performs **per-file optimization and OpenACC adjustment** (Table 6-1). There is no `-gpu=ccXX`; the build machine's GPU is autodetected. Add to `COMMON_CFLAGS` for cross-targeting.

9.2 Link Configuration

Makefile (excerpt)

```
NVPL_SCALAPACK_LIB ?= $(NVHPC_MPI_LIBDIR)/libscalapack_lp64.a
NVPL_LAPACK_LIB    ?= $(NVHPC_COMPILER_LIB)/liblapack_lp64.a
NVPL_BLAS_LIB      ?= $(NVHPC_COMPILER_LIB)/libblas_lp64.a
NVPL_LIBS          ?= -Wl,--start-group $(NVPL_SCALAPACK_LIB) $(NVPL_LAPACK_LIB) $(NVPL_BLAS_LIB) -Wl,--end-group
CUDA_LIBS          ?= ... -Wl,--no-as-needed -lnvJitLink -Wl,--as-needed -lcudart -lcusolver -lcublas -cuda
LIB = $(FFTW3_LIBS) $(NVPL_LIBS) $(MPI_FORTRAN_LIBS) -pgf90libs -mp -lpthread -lm -ldl $(CUDA_LIBS)
...
LIB += -lcublasLt -lcuda -lnvidia-ml -lstdc++
```

The link line is `$(CC) $(OBJS) $(GEMMUL8_LIB) $(LIB) -lm -o openmx`. Additions to OBJS: `Total_Energy_GPU.o`, `Set_Density_Grid_GPU.o`, `set_hamiltonian_gpu.o`, `gemmul8_bridge.o`, `gpusolver_Syevd.o gpusolver_Syevdx.o set_cuda_default_device_from_local_rank.o set_openacc_device_from_local_rank.o`, and `OutData_Binary.o` (a dead link with no callers). The former `my_cublasDgemm.o my_cublasZgemm.o` were removed in `a7f8cba6`.

9.3 Automatic third_party Builds and the Single-make Guarantee

Since the default goal is `openmx: $(FFTW3_LIB) $(GEMMUL8_LIB) $(OBJS)`, a bare `make` builds FFTW3 → GEMMul8 → the main code automatically. Commit `9fe8ce13` guarantees that a **single `make` (parallel `-j` allowed) completes from a pristine tree**, via four fixes:

1. **Automatic FFTW3 clone:** when `third_party/fftw3` is missing, the rule now runs `git clone --branch fftw-3.3.11 --depth 1` automatically (mirroring the GEMMul8 handling) instead of erroring out. The git sources lack the generated codelets, so a sentinel file check overlays them from the release tarball (`fftw-3.3.11.tar.gz`, also committed to the repo) with `rsync --ignore-existing`, then autoreconf → out-of-tree configure (`--enable-static --disable-shared`, compilers nvc/nvc++/nvfortran) → `make install`.
2. **FFTW3_USERS completed:** `Cluster_DFT_Col.o` includes `fftw3.h` but was missing from the order-only dependency list (`| $(FFTW3_LIB)`), so -j builds compiled it before FFTW was installed, failing the first one or two makes. Now: `FFTW3_USERS = Poisson.o Stress.o Poisson_ESM.o TRAN_Poisson.o TRAN_CDen_Main.o TRAN_Set_Electrode_Grid.o Cluster_DFT_Col.o`.
3. **Historical Fortran-module dependency fix:** commit `9fe8ce13` added the missing `use` dependencies for the then-embedded ELPA, closing the race where compiles started before the `.mod` files existed. That embedded version has since been removed (§ 9.4). The current `openmx_elpa_cpu.o` adapter and GPU bridge both depend on the modern ELPA installation stamp; its own build system handles its Fortran-module dependencies.
4. **An order-only rule for gemmul8.hpp:** an empty-recipe rule prevents "No rule to make target" on a fresh clone without the GEMMul8 checkout.

Makefile — GEMMul8 fetch and single-TU build

```
$(GEMMUL8_DIR)/Makefile:
	mkdir -p $(GEMMUL8_REPO_DIR)
	cd $(GEMMUL8_REPO_DIR) && \
	  { [ -d .git ] || git init -q .; } && \
	  { git remote get-url origin >/dev/null 2>&1 || git remote add origin $(GEMMUL8_REPO_URL); } && \
	  git fetch --depth 1 origin $(GEMMUL8_COMMIT) && \
	  git checkout -q $(GEMMUL8_COMMIT)
$(GEMMUL8_OPENMX_OBJ): gemmul8_openmx.cu $(GEMMUL8_DIR)/Makefile
	$(NVCC_GEMMUL8_ENV) $(NVCC) $(NVCC_GEMMUL8_FLAGS) -c gemmul8_openmx.cu -o $(GEMMUL8_OPENMX_OBJ)
$(GEMMUL8_LIB): $(GEMMUL8_OPENMX_OBJ)
	mkdir -p $(GEMMUL8_DIR)/lib
	ar rcs $(GEMMUL8_LIB) $(GEMMUL8_OPENMX_OBJ)
```

Verification (commit message): after `make clean`, a single `make -j16 openmx` completes in 177 s on the RTX 5080 box; a second make is a no-op; compile flags are unchanged, so the binary is unchanged. `clean` includes `fftw3-clean` and `gemmul8-clean`, so third_party must rebuild after `make clean`.

9.4 Shared ELPA 2026.02.002

The old `elpa-2018.05.001/` source directory and the legacy `elpa1.f90` / `solve_evp_real.f90` / `solve_evp_complex.f90` implementations have been removed. CPU and GPU paths now link the **same bundled ELPA 2026.02.002 library**, `$(GPUSOLVER2_PREFIX)/lib/libelpa.a`, built from the release archive under `third_party/dist/`. The former symbol-isolation object is no longer generated or linked.

`openmx_elpa_cpu.c` preserves the existing Fortran-callable entry points and invokes the modern C API: `scf.eigen.lib elpa1` selects the one-stage CPU solver, and `elpa2` selects the two-stage CPU solver with generic real/complex kernels. The adapter explicitly disables ELPA's GPU options. `gpusolver2` uses the same library through `elpa_cosma_bridge.c`, enabling the CUDA or HIP GPU backend selected at build time. CPU ELPA remains the **runtime fallback when GPU memory is insufficient (§ 5.6)**.

9.5 Derived Makefiles

- **`makefile.cpu_intel.bak`** — originally a byte-identical copy of the original CPU/Intel makefile. Its ELPA section now uses the shared modern library and CPU adapter (§ 9.4).
- **`Makefile.bunshiken`** — a variant for a shared machine (modules: `nvhpc/25.9-nompi` + `openmpi/5.0.8` + `gcc-toolset/15`). NVHPC 25.9 with external OpenMPI, CUDA 13.0, g++ as the nvcc host compiler, an unpinned old GEMMul8 layout, and object names still using the pre-rename `cusolver_*.o` — **more than one generation behind** the main Makefile, and without the § 9.3 single-make fixes. Treat it as a porting example.

9.6 Build Procedure and Notes

1. `git clone --recursive` (submodules: fftw3, GEMMul8). Forgotten submodules are cloned or fetched automatically by `make` (§ 9.3).
2. Run `make` in `source/` (default goal openmx; `-j` allowed). FFTW3 fetch/overlay → configure → install, GEMMul8 fetch → compile → ar, the CUDA-kernel nvcc compiles, and the main compile/link all run in one invocation.
3. `make install` copies to `../work/openmx`; utilities via `make all`.

> **Build notes:** (1) Changing the NVHPC version requires re-verifying the miscompilation workarounds of § 6.5. (2) Check the selected MPI paths after SDK updates; override `NVHPC_MPI_BIN` and `NVHPC_MPI_LIBDIR` as needed. (3) The GEMMul8 architecture can be overridden as `make GEMMUL8_GPU_ARCH=90` (default: nvidia-smi autodetection). (4) The original's auxiliary targets (libopenmx_api, elpa_col, …) were removed.

9.7 Unified CUDA / HIP Builds with CMake

The integrated repository contains both backends. CMake 3.20+, Python 3.8+, and GNU make select and prepare an isolated native build; the existing Makefiles retain their per-file compiler and offload rules. `tools/prepare_build.py` links the selected sources into `build/<backend>/native/source/`. Common sources come from `source/`; HIP replacements and compatibility headers come from `source/backends/hip/`, according to `sources.json`. Mutable dependency sources, objects, and installations belong to each build. The original `openmx4.0_amdgpu` checkout is no longer required.

Setting	Meaning
`OPENMX_GPU_BACKEND`	`CUDA` (default) or `HIP`; use separate build directories.
`OPENMX_GPU_ARCH`	CUDA: e.g. `120` (RTX 5080), `80` (A100), `90` (H100). HIP: e.g. `gfx942` (MI300A), `gfx1200` (RX 9060 XT).
`OPENMX_BUILD_JOBS`	Native make parallelism; default `4`. The outer `cmake --build -j` does not set this.
`OPENMX_MAKE_OPTIONS`	Quoted, semicolon-separated `NAME=value` assignments for the native Makefile.

CUDA example — run at the repository root

```
git submodule update --init --recursive
cmake -S . -B build/cuda \
  -DOPENMX_GPU_BACKEND=CUDA -DOPENMX_GPU_ARCH=120 \
  -DOPENMX_BUILD_JOBS=4
cmake --build build/cuda
```

Override `NVHPC_ROOT`, `NVHPC_CUDA_VERSION`, `NVHPC_CUDA_HOME`, `NVHPC_MATH_DIR`, and the MPI paths as needed through `OPENMX_MAKE_OPTIONS`. Add `WITH_MAGMA=1` for CUDA MAGMA support. The existing `make -C source -j4 openmx` workflow also remains available.

HIP example — replace the site-specific library and MPI paths

```
git submodule update --init --recursive source/third_party/GEMMul8
cmake -S . -B build/hip \
  -DOPENMX_GPU_BACKEND=HIP -DOPENMX_GPU_ARCH=gfx942 \
  -DOPENMX_BUILD_JOBS=4 \
  '-DOPENMX_MAKE_OPTIONS=ROCM=/opt/rocm-7.2.0;MPI_HOME=/path/to/openmpi-rocmflang;AOCL_HOME=/path/to/aocl/aocc;AOCC_LIB=/path/
to/aocc/lib'
cmake --build build/hip
```

HIP requires ROCm clang/clang++, hipcc, flang, OpenMP offload, and AOCL BLIS/libFLAME, ScaLAPACK, FFTW3, and Utilities. MPI's Fortran module must match ROCm flang; a gfortran `mpi.mod` is incompatible. `AOCC_LIB` supplies AOCL's Fortran runtime dependencies. The Makefile puts `ROCM_LLVM_LIB` first in `LIBRARY_PATH` so the selected ROCm OpenMP runtime takes precedence over unrelated Intel/AOCC runtimes.

The first build prepares GEMMul8, patched MAGMA (HIP), and ELPA/COSMA. ELPA stack construction also needs autoconf, automake, libtoolize, m4, and unzip. For ROCm flang, the HIP stack script patches ELPA's host/device allocation wrappers to pass allocation outputs through `integer(c_intptr_t)`, avoiding incorrect `type(c_ptr)` interoperation. The bundled archive stays unchanged; the patch is applied to the build-local source. An existing CMake 3.24+ can be supplied to the dependency builder with `GPUSOLVER2_CMAKE=/absolute/path/to/cmake`.

The executables are `build/cuda/openmx` and `build/hip/openmx`. `selected-sources.json` records the source selection in each build directory. `cmake --install build/hip --prefix /path/to/install` installs only `bin/openmx`; keep its runtime libraries available and provide DFT data separately. Input `DATA.PATH` is relative to the working directory: run existing inputs from `work/`, or use an absolute path to `DFT_DATA19`. Full configuration details are in `docs/GPU_BACKENDS.md`.

9.8 Regression Checks and Recorded Validation

`scf.eigen.lib elpa1` / `elpa2` select CPU execution in either build; `OPENMX_GPU=0` forces the default GPU selection to ELPA2. These are CPU execution paths inside a CUDA or HIP executable, so the build's runtime libraries are still required.

```
ctest --test-dir build/cuda --output-on-failure
ctest --test-dir build/hip --output-on-failure
python3 tests/run_elpa_cpu_mpi.py \
  --native-source build/cuda/native/source
```

The adapter check compares real/complex, one-stage/two-stage solves with LAPACK, including partial eigenvectors, rectangular process grids, subcommunicators, and empty local rows/columns. Use `build/hip/native/source` for the HIP build. For the standard CPU suites, use a fresh output directory for each run:

```
python3 tools/run_gpu_suites.py --binary build/cuda/openmx \
  --output build/cpu-elpa-S --suite S --ranks 18 --threads 1 \
  --env OPENMX_GPU=0 --together
python3 tools/compare_gpu_suites.py --reference build/cpu-elpa-S

python3 tools/run_gpu_suites.py --binary build/cuda/openmx \
  --output build/cpu-elpa-L --suite L --ranks 18 --threads 1 \
  --env OPENMX_GPU=0 --together
python3 tools/compare_gpu_suites.py --reference build/cpu-elpa-L
```

S corresponds to `-runtest`, L to `-runtestL`. The reference comparison checks total energies and every recorded atomic force component. Substitute the HIP executable to check that backend. The commands are reproducible procedures, not new benchmark results.

Recorded change	Correctness checks
`bdcba87d`: shared ELPA	CPU S: 14/14; L: 16/16; explicit ELPA1: 4/4. MPI adapter: 20 cases per rank on six ranks.
`57a3a3fc`: HIP cleanup without visible GPUs	16 MPI ranks × 1 thread: S 14/14 and L 16/16; energies and forces within 1e-7 of references. HIP CTest: 9/9.

These results are the recorded validations of the cited commits. The AMD GPU validation described in `docs/GPU_BACKENDS.md` used one Radeon RX 9060 XT with ROCm 7.2.0, OpenMPI 5.0.9, and AOCL 5.2.0. It covered CPU/GPU comparisons, the full S suite, selected L3 cases, forced GPU solves, ELPA/COSMA, and selected capacity recovery paths. It did not validate MI300A hardware, physical multi-GPU or multi-node execution, or all L/L2/L3 GPU cases.

10. Runtime Control Reference

10.1 Input-File Keywords

Besides OpenMX's own `level.of.stdout`, which now also sets how much of the GPU diagnostics is printed (§ 10.3), only two input keywords were added or extended for the GPU features (everything else is tuned via environment variables).

Input_std.c — the addition (complete)

```
/* default=gpusolver; input_string2int() returns i_vec[0] when the keyword is absent ...
   "cusolver" is a deprecated alias of "gpusolver"; ... */
s_vec[0]="gpusolver"; s_vec[1]="lapack"; s_vec[2]="elpa1"; s_vec[3]="elpa2"; s_vec[4]="cusolver";
i_vec[0]=GPUSOLVER;   i_vec[1]=0;        i_vec[2]=ELPA1;   i_vec[3]=ELPA2;   i_vec[4]=-GPUSOLVER;
input_string2int("scf.eigen.lib", &scf_eigen_lib_flag, 5, s_vec,i_vec);
if (scf_eigen_lib_flag==-GPUSOLVER){
  if (myid==Host_ID){
    printf("Warning: scf.eigen.lib=cusolver is deprecated; use scf.eigen.lib=gpusolver instead.\n");
  }
  scf_eigen_lib_flag = GPUSOLVER;
}
scf_eigen_lib_flag_input = scf_eigen_lib_flag;

/* use GPU? (added by H.Kawai, February 2024) */
input_int("scf.Gpu.Num", &SCF_Gpu_Num, 30);
```

Keyword	Type	Default	Meaning
`scf.eigen.lib`	string choice	`gpusolver`	Adds **`gpusolver`** (enables the whole GPU stack) to the conventional `elpa 2` / `lapack` / `elpa1`, and is the default, so the GPU paths run unless one of the CPU eigensolvers is requested explicitly. `cusolver` is a deprecated alias with a warning. Demotes automatically to ELPA2 when no GPU is present. **`gpusolver2`** additionally distributes the mainline collinear/non-collinear *cluster* diagonalization over all ranks and their GPUs with ELPA 2026.02.002 (CUDA or HIP GPU kernels) and COSMA (pdgemm) — the multi-node option for Pegasus-class machines; any non-cluster calculation stops with an input error, and everything outside the cluster diagonalization behaves exactly like `gpusolver`
`scf.fp32.enable`	logical	`on`	`off` forces FP64 arithmetic: disables FP32 eigenvector refinement, FP32/double-float grid quadrature and double-float orbital evaluation, and promotes legacy float products in VNA matrix, force and stress contractions to FP64. Overrides FP32/DF environment requests. GEMMul8 remains controlled independently by `scf.gemmul8.enable` and its environment settings; cuSOLVER FP64 fixed-point emulation is also independent. Existing float table storage is unchanged. The default `on` permits the existing precision policies.
`scf.Gpu.Num`	int	30	Upper bound on GPUs used per node, applied to every rank→GPU assignment (central GPUSOLVER initialization, the local-rank device helpers of the Band/Cluster solver paths, and the DC-LNO proxy). `scf.Gpu.Num 1` confines the run to one GPU however many the node carries — a debugging aid; 30 effectively means "use every GPU found". Values ≤0 or ≥ the detected count are ignored

Keyword	Type	Default	Meaning
`level.of.stdout`	int	1	OpenMX's amount of standard output (−1 to 3), extended to the GPU diagnostics (§ 10.3): 1 prints only errors, a GPU that cannot be used and a GPU eigensolver given up; 2 adds the device plans and banners, the final FP64 re-solves and the fallbacks and retries of single stages and solves; 3 adds the per-step, per-k-point and per-rank details

10.2 Environment Variables

GPU tuning is done through environment variables (adjustable from the job script without touching the input file). Where both `OPENMX_`- and `GEMMUL8_`-prefixed names exist, the OPENMX_ name wins. Many variables were added since the 2nd edition, so they are grouped by subsystem. **Nearly every GPU path is on by default**; the variables mainly serve to disable, to adjust memory reserves, or to force.

Environment variable	Default	Meaning
Device-memory accounting (set_cuda_default_device_from_local_rank.c)		
`OPENMX_GPU_STACK_BYTES`	the largest stack frame of OpenMX's own kernels (cuobjdump on the linked binary at link time; 1000 bytes in the current build)	Per-thread kernel stack limit set at device initialization (`cudaLimitStackSize` , `OpenMX_GpuPresizeKernelStack`) when it exceeds the driver's default 1 KiB, so the driver sizes the context's local-memory pool while the device is empty instead of at the first launch of a kernel with a large frame. That launch-time growth (about 26 MiB per context for a 1224-byte frame on an RTX 5080: (frame − 1024) × every resident thread of the device) raced with the force-phase arenas, which fill the device down to their reserve, and lost at 18 ranks on 16 GB (`cuLaunchKernel` : out of memory in `Force3_GpuTrace` , C1000, whose dOrbitals kernel had that frame; its spherical-harmonic tables are now flat, 936 bytes, so the step is idle at present). The Makefile reads the figure from the linked binary (the kernels of the objects; the linked libraries' kernels, such as ELPA's 1328-byte one, are not counted) and relinks once with it. 0 disables the step; a raised limit costs (limit − 1024) × resident threads of device memory per rank.
`OPENMX_GPU_MEMINFO_NVML`	1	Every device-memory budget of the GPU paths starts from `OpenMX_GpuMemGetInfo()` : `cudaMemGetInfo()` with the free figure capped at the memory that is physically free according to NVML. On WDDM platforms (WSL2) the CUDA runtime reports a per-process view in which the other ranks' contexts and arrays are invisible, and an allocation past the physical memory is paged through system memory at a fraction of the speed instead of failing: 12 ranks on a 12 GB RTX 4070 Ti each saw 10.8 GiB "free", took 836 MiB each for the DS_VNA batch and stalled Set_ProExpn_VNA for minutes (0.6 GiB was left after the contexts). On Linux the two figures agree. A one-time stdout notice (level.of.stdout 2 and above) reports both figures when they differ by more than 256 MiB. 0 = plain `cudaMemGetInfo()`
GEMMul8 (gemmul8_bridge.cu)		
`(OPENMX_)GEMMUL8_DISABLE` / `_D` / `_Z`	off	Disable GEMMul8, pin to cuBLAS FP64 (all / real only / complex only)
`(OPENMX_)GEMMUL8_NUM_MOD_D` / `_Z`	15	Number of moduli (precision); range 2–20, out-of-range resets to 15
`(OPENMX_)GEMMUL8_FASTMODE_D` / `_Z`	0	Fast scaling mode (lower accuracy)

Environment variable	Default	Meaning
`(OPENMX_)GEMMUL8_MAX_WORKSPACE_PERCENT`	30	Workspace cap as a fraction of total VRAM (Band_DFT_NonCol setenvs 50 when unset)
`(OPENMX_)GEMMUL8_MIN_FREE_AFTER_MB`	1536	Minimum free VRAM required after workspace allocation
Band calculations (Band_DFT_Col.c / Band_DFT_NonCol.c)		
`OPENMX_BAND_GPU_DIAG`	auto	**1** = force the dense GPU path ignoring the fits verdict; **0** = always ELPA/ScaLAPACK (§ 5.6)
`OPENMX_BAND_GPU_SWITCH_NUM`	800	GPU/CPU crossover dimension of the collinear band dense path (paths without their own knob keep `GPU_CPU_SWITCH_NUM` =1000). Measured crossover on one RTX 5080 shared by 8 ranks: n ≈ 300–400
`OPENMX_BAND_NONCOL_GPU_SWITCH_NUM`	800	Same for the noncollinear band dense path, compared against 2n (measured crossover 2n ≈ 250–400)
`OPENMX_BAND_GPU_DM`	1	**0** = CPU loop for the DM accumulation of the collinear band ScaLAPACK/ELPA fallback. By default the fallback keeps the DM on the GPU: the eigenvector panel fits even when the full dense solve does not
`OPENMX_BAND_NONCOL_GPU_DM`	1	**0** = CPU loop for the per-k DM accumulation of the noncollinear ScaLAPACK/ELPA-fallback multi-k loop (the k-dense GPU flow keeps its own device DM regardless)
`OPENMX_BAND_GPU_RESERVE_MB`	max(total/10, 1536)	Raises the reserve used by the concurrency planner and the fits verdicts
`OPENMX_BAND_GPU_RANK_OVERHEAD_MB`	256	Runtime overhead added to the per-rank requirement model
`OPENMX_BAND_SYEVDX_NOVECTOR`	1	Solve the eigenvalues-only first pass with NOVECTOR (0 restores the old behavior)
`OPENMX_BAND_GPU_MAX_CONCURRENT_K` / `OPENMX_BAND_GPU_TURN_GROUP` / `OPENMX_BAND_GPU_MAX_RANKS_PER_DEVICE`	uncapped	**Explicit caps** on the concurrent rank count (first found in this order). The old "initial value" meaning is gone; actual concurrency always comes from the adaptive scan
`OPENMX_GPUSOLVER_SERIAL_GPU_TURNS`	1	GPU-turn serialization of the all_knum==1 path (0 = concurrent submission)
`OPENMX_BAND_GPU_PERSISTENT` / `_MIN_FREE_MB`	0 / 2048	Device-buffer persistence across SCF iterations (not recommended on shared GPUs)
`OPENMX_BAND_GEMMUL8_TURN_RELEASE`	1	Per-turn GEMMul8 workspace release (0 keeps them)
`OPENMX_BAND_PROFILE`	0	Performance counters; also gates the per-rank stage lines SETHPROF (Set_Hamiltonian), SETHOTFPROF / SETHOTFDENPROF (the on-the-fly batches: pack, evaluation, kernel), SETNLPROF (Set_Nonlocal: DS_NL, the received-atom products, the local products), CLUSTERDMPROF (the density-matrix accumulation of the cluster solver), CLUSTERDISTPROF (the eigenvector distribution; on the ranks that do not own the dense solve it includes the wait for it), FORCEHNL2PROF (the second-case Force_HNL batch: build, device, post-processing), the section timers of truncation and Total_Energy, SETVPOTPROF (Set_Vpot: xc, the Vxc D-to-B copy, VNA, assembly, mixing FFT, the B-to-C redistribution) and SETDENPROF (Set_Density_Grid: decision, quadrature, A-to-B, superposition, B-to-D)
Cluster calculations (Cluster_DFT_Col.c / Cluster_DFT_NonCol.c)		
`OPENMX_CLUSTER_GPU_DIAG`	auto	**1** = force the GPU even if the reservation fails (then aborts with a diagnostic); **0** = always ELPA (§ 5.6)

Environment variable	Default	Meaning
`OPENMX_CLUSTER_GPU_SWITCH_NUM`	400	GPU/CPU crossover dimension of the collinear cluster dense path (the GPU won at every measured size down to n=240)
`OPENMX_CLUSTER_NONCOL_GPU_SWITCH_NUM`	400	Same for the noncollinear cluster dense path, compared against 2n (the GPU won down to 2n=480)
`OPENMX_CLUSTER_GPU_DIAG_RESERVE_MB`	1024	Free VRAM required to remain after the reservation probe
`OPENMX_CLUSTER_GPU_DIAG_SERIAL`	1	Serialized mode when both spin owners cannot fit (0 = disable, 2 = force serial)
`OPENMX_CLUSTER_GPU_DM`	1	GPU accumulation of DM/EDM (0 = CPU loop)
`OPENMX_CLUSTER_GPU_DM_ALL`	1	Device accumulation of the cluster density matrix on every rank (its slice of states, staged with a halving retry); 0 = only the rank that owns the dense solve, the others on the CPU loop (threaded over the atoms)
`OPENMX_CLUSTER_GEMMUL8_TURN_RELEASE`	1	Per-solve GEMMul8 workspace release (inherits the BAND setting when unset)
`OPENMX_EIGEN_REFINE`	2 on GPUs whose FP32:FP64 throughput ratio is at least 8 (GeForce, RTX PRO 6000 Blackwell, ...) when the GEMMul8 products of the solver's kind (real for collinear, complex for noncollinear) are on; else 0	Cluster dense paths (collinear, per spin; noncollinear) and the band solvers (one k-point per rank: the estimates of all k-points are gathered before the refinement; several k-points per rank: the first pass refines with the previous step's chemical potential and keeps the basis on the host, bounded by `OPENMX_BAND_REFINE_BASIS_MB` per rank (default 1024), and the second pass refines from it with the step's chemical potential): K > 0 solves all n (2n) pairs in FP32 and refines the MaxN wanted ones by K Ogita–Aishima steps (products through GEMMul8, module `eigen_refine_gpu.c`) plus a Rayleigh–Ritz step in partially occupied clusters; from the second SCF step on the refinement starts from the previous step's vectors when they are within 2e-5 of the new Hamiltonian (`OPENMX_EIGEN_REFINE_WARM`). The step that ends the SCF is solved once more in FP64, the last step `scf.maxIter` allows is FP64. With spin polarization the two spin worlds' owners exchange their eigenvalue estimates for the chemical potential (the band solvers gather the estimates of all k-points before the refinement); a single rank solving both spins, the serialized spin mode, XANES and emptied orbitals or states keep FP64. **0** = off. On an RTX 5080 (18 ranks) noncollinear sidia333 (2n = 5616) takes 197.9 s instead of 261.3 s with the same SCF steps The band solvers refine the maxn wanted columns and rotate the other n − maxn columns against them every step (`OPENMX_EIGEN_REFINE_COMPLEMENT`, default 1): left alone, their FP32-level components along the occupied subspace tilt the refined columns' fixed point out of it by about 1e-8, which cost 2 to 5 extra SCF steps and biased warm starts. When the k-owners do not fit in one GPU turn group, the one-k-point-per-rank solves are staged like the multi-k-point ones (pass 1 with the previous step's chemical potential and the basis on the host, pass 2 from it), so a 16 GB card refines them too. Diagnostics: `OPENMX_EIGEN_REFINE_CHECK=1` prints the residuals and orthogonality of the refined columns by occupation class with plain FP64 products, `OPENMX_EIGEN_REFINE_ALL=1` refines every column

Environment variable	Default	Meaning
`OPENMX_EIGEN_REFINE_UNTIL`	0 (off)	Printed NormRD below which the refined eigensolver hands the rest of the SCF cycle to the FP64 eigensolver. Off by default: an SCF that ends on a refined step has that step's Hamiltonian solved once more in FP64 (one eigensolve, no density matrix) for the vectors the post-SCF energy density matrix and the forces are built from Invalid (NaN/Inf) refinement products, eigenvalues or final vectors now reject the refinement before it overwrites the preserved FP64 matrix. This safety check does not change the successful arithmetic or tolerances.
`OPENMX_EIGEN_REFINE_WARM`	1	Warm start of the refined eigensolver: the vectors of the previous step's refined solve are refined against the new Hamiltonian instead of solving in FP32 first, as long as their residual on it is at most 2e-5 (sidia333 on an A100: the eigensolver phase 706 -> 140 ms per step, 20 of 22 steps); **0** = an FP32 solve every step The band solvers warm-start too (`OPENMX_EIGEN_REFINE_WARM_BAND` , default 1); `OPENMX_EIGEN_REFINE_WARM_PERIOD` =N makes them solve in FP32 again every N SCF steps (default 0 = never: since the unrefined columns are rotated against the refined ones, a chain of warm starts agrees with FP64 to 2e-12 Ha on sidia333)
`OPENMX_EIGEN_REFINE_STOP_GUARD`	100	An SCF stop on a refined FP32 step also needs the printed NormRD below this factor times `scf.criterion` ; OpenMX's test on dUele alone fired on a coincidence (RTX 5080, sidia333_col_cluster: dUele 7e-12 at step 15 with NormRD 4.3e-8, five steps early and 2e-9 Ha off). 0 = off. The band solvers also drop their warm-start bases when the SCF cycle ends, so the forces start with the device memory free (the resident bases of the NC band left Force3 without room on a 16 GB card)
`OPENMX_EIGEN_REFINE_UNBLOCKED`	1	The refinement products with the full GEMMul8 workspace when the device has room for it (about 1.7x faster than the blocked products at the 256 MiB cap); **0** = the blocked products
`OPENMX_EIGEN_FP32_TWO_STAGE`	1	The full FP32 solve of the refined eigensolver uses the two-stage cusolverDnXsyevd (CUSOLVER_ALG_2; cuSOLVER 12.3.4 and later; RTX 5080, n = 5616: 323 ms instead of 638), **0** = the one-stage cusolverDnXsyevdx. Since 2026-10-09 a two-stage solve that fails to converge (info > 0: sporadic on the RTX 5080, e.g. info=297 on a 2808 x 2808 band problem) is retried once with the one-stage algorithm from a fresh conversion of the FP64 matrix before the solver falls to FP64; before, one such failure left the rest of the SCF cycle in FP64 (sidia333 band_on D: Diagonalization 68 s to 95 s)
`OPENMX_CLUSTER_GPU_DM_GEMM`	1	Noncollinear cluster dense path: the density matrix as one product $W^{H}W$ over the occupied states (GEMMul8) plus a gather of the sparse entries, and a broadcast of the root's buffer instead of an all-reduce; **0** = the per-entry sums over the states (sidia333 on an RTX 5080: 432 ms per step)
DC method (Divide_Conquer.c)		
`OPENMX_DC_GPU_THRESHOLD`	400	Minimum local cluster dimension NUM for the GPU (collinear)
`OPENMX_DC_NONCOL_GPU_THRESHOLD`	300	Minimum spinor dimension 2 · NUM for the GPU (noncollinear)
`OPENMX_DC_GPU_S_CACHE` / `_RESERVE_MB`	1 / 4096	Device-resident S12 cache and its budget reserve

Environment variable	Default	Meaning
`(OPENMX_)GPUSOLVER_MIN_FREE_AFTER_MB`	1536	Minimum free VRAM after eigensolver allocations (preflight)
`OPENMX_CUBLAS_MIN_FREE_AFTER_MB`	—	Same for cuBLAS
DC-LNO method (Divide_Conquer_LNO.c)		
`OPENMX_DCLNO_GPU_THRESHOLD`	800	Minimum local dimension for the GPU (collinear)
`OPENMX_DCLNO_NONCOL_GPU_THRESHOLD`	400	Minimum spinor dimension for the GPU (noncollinear)
`OPENMX_DCLNO_GPU_TURN_RELEASE`	1	End-of-phase release of solver buffers + GEMMul8 (inherits the BAND setting when unset)
Krylov method (Krylov.c)		
`OPENMX_KRYLOV_GPU_FUSED`	1	Fused per-atom pipeline (0 = per-call sequence)
`OPENMX_KRYLOV_GPU_KU_CACHE` / `_KU_RESERVE_MB`	1 / 4096	Device-resident Krylov basis cache and its budget reserve
`OPENMX_KRYLOV_GPU_DGEMM_MIN_FLOPS`	5×10^{7}	Minimum $m\cdot n\cdot k$ for dispatching a GEMM to the GPU
`OPENMX_KRYLOV_GPU_EIGEN_MIN`	300	Minimum projected dimension Msize3 for the device eigensolve (§ 7.6)
`OPENMX_KRYLOV_GPU_PROFILE`	0	Fused-path breakdown (KRYFUSE lines)
Set_Hamiltonian (Set_Hamiltonian.c / set_hamiltonian_gpu.cu)		
`OPENMX_SETHAM_MATRIX_GPU`	1	GPU execution of the dVH+Vxc+VNA grid integrals (0 = CPU)
`OPENMX_SETHAM_CUDA_KERNEL`	1	The dedicated CUDA kernel (0 = OpenACC kernel only; numerically identical)
`OPENMX_SETHAM_GPU_RESIDENT` / `_FLOOR_MB`	1 / 4096 or 16384	Device residency of the SCF-invariant tables and the floor left free afterwards (default depends on whether a diagonalization need has been published; § 7.7)
`OPENMX_SETHAM_GPU_MAX_RANKS_PER_DEVICE`	uncapped	Explicit cap on ranks per wave
`OPENMX_SETHAM_GPU_SERIAL_WAVES`	0	1 = run all waves on the GPU serially (default: wave 1 on GPU with remaining ranks on CPU concurrently) Under memory pressure on the A100 (whole-rank allocations for 2-3 of 8 ranks) the serial waves took 37.6 s against 165 s of the hybrid default for 12 SCF steps of sidia333, and the 11-14 CPU-fallback ranks of an RTX 5080 run are the same situation
`OPENMX_SETHAM_STREAMING`	1	Streamed pair batches (256 MiB workspace per rank, `OPENMX_SETHAM_STREAM_MB`) when the device holds a single whole-rank allocation; **2** streams whenever it cannot hold every rank's (A100 emulation: 52 s against 165 s of the hybrid default and 37.6 s of the serial waves); **0** = off

Environment variable	Default	Meaning
`OPENMX_SETHAM_ONTHEFLY`	1	A rank whose orbital tables do not fit on the device evaluates the orbitals of every batch of atom pairs on the device (the device code of Set_Orbitals_Grid, orbs_grid_gpu.h) instead of computing on the CPU or streaming the tables: only its atoms' sphere indices stay resident (a few MiB) beside a batch workspace of `OPENMX_SETHAM_ONTHEFLY_MB` (default 128) MiB. The density of Set_Density_Grid takes the same tiles on such ranks. 0 = off (the CPU or the streamed path as before) The workspace of a rank is the smaller of that bound and its share of the plan's free device memory left after the staged wave's peak and a margin of `OPENMX_SETHAM_ONTHEFLY_MARGIN_MB` (default 0) MiB, without exceeding either the explicit workspace cap or the physical free-memory share (the budget comes from the plan, not from a reading taken while the staged ranks allocate, so every rank sees the same value). Oversized atom pairs use bounded overlap-point chunks; allocation failures retain the CPU fallback. When the device cannot hold every rank's tables, the plan shrinks the staged wave until each rank outside it gets its bound: the on-the-fly ranks, not the staged ones, set the time of the phase (RTX 5080: 1.4 s against 0.65 s per step). The on-the-fly state is released when the SCF cycle ends. The evaluation takes cos/sin of the angles straight from the coordinates and the radial mesh index from the logarithmic mesh (no bisection), which agrees with the host's Get_Orbitals to the last bits of the double value
`OPENMX_SETDEN_ONTHEFLY_MB`	128	Batch workspace bound (MiB) for on-the-fly density. The physical budget subtracts a 512 MiB node reserve, divides among on-the-fly ranks, then subtracts each rank's fixed density/output storage and 64 MiB runtime reserve. Explicit caps must be at least 32 MiB; smaller values retain the default. If a full pair exceeds the default bound, automatic growth uses spare memory, up to all pairs in one batch. Explicit caps suppress growth. Pairs that still exceed the workspace are divided into overlap-point chunks; neither a large atom nor an explicit cap alone causes a rank to decline (§ 7.9)

Environment variable	Default	Meaning
`OPENMX_SETHAM_ONTHEFLY_PLAN_CACHE`	1	The on-the-fly paths keep a plan per rank: the batches (which pairs, their offsets), the pair descriptors and the per-point index lists (Nc, and the potential's grid index MN for the matrix elements) in one pinned host image per batch laid out like the device arena, uploaded in one copy per batch; per step only the H blocks (pinned buffer), the potential rows (through a pinned staging buffer of the code's own; registering the OpenMX-owned rows with cudaHostRegister made a later unrelated copy fail sporadically with CUDA_ERROR_INVALID_VALUE) and the tiles move, the density accumulator is zeroed on the device and its host buffers are pinned and persistent. The plans live until the end of the SCF cycle or until the workspace, the spin count or the atom partition or orbital-grid geometry change (about 8 bytes of host memory per overlap point of the rank's pairs). Before, each batch was rebuilt every step and uploaded in ~90 small synchronous copies (one per pair for the Nc lists), ~0.3 ms each under MPS with 18 ranks: on the RTX 5080 (sidia333 non-collinear cluster) the matrix elements' pack went 0.25 s to 0.10 s and the density's 0.20 s to 0.02 s per rank and step, an on-the-fly rank's matrix elements 0.68 s to 0.48 s, its density 0.41 s to 0.26 s, Set_Density_Grid 18.1 s to 14.8 s, D 107.7 s to 103.2 s (band_nc D 144.4 s to 133.8 s), Utot within 1e-12 Ha. **0** = rebuild the plan on every call
`OPENMX_SETHAM_MATEL_SPLIT`	4	Blocks per atom pair along the grid points in the df matrix-element kernel (each block integrates whole 32-point tiles of its share and adds its partial sums atomically onto the H blocks); 1 = one block per pair, up to 64
`OPENMX_ORBS_EVAL_PRECISION`	df / fp64	The arithmetic of the on-the-fly orbital evaluation (the device code of Set_Orbitals_Grid that the Hamiltonian and density tiles use when a rank has no resident tables): **df** = double-float, the default. After the FP64 grid coordinates every quantity is a pair of floats (hi + lo, about 2^-48 relative): the PAO radial tables and the per-mesh spline constants are held as such pairs, the mesh index comes from an FP32 logarithm and the index search makes it exact, and the float written for an orbital is the hi part of its product, i.e. the float nearest the FP64 value unless that value lies within ~1e-14 of a rounding boundary (about one value in 10^6 differs by one ulp from the FP64 routine; points inside the first mesh point take the FP64 routine). **fp64** = the FP64 transcription of the host's Get_Orbitals (bitwise the tables' values). A GeForce runs FP64 at 1/64 of its FP32 rate and the FP64 evaluation is bound by that pipe: sidia333 non-collinear cluster on the RTX 5080 (18 ranks + MPS, 23 steps), evaluation per rank and step 0.77 s to 0.41 s (Hamiltonian tiles) and 0.42 s to 0.07 s (density tiles), Set_Hamiltonian 43.5 s to 35.8 s, Set_Density_Grid 32.6 s to 24.1 s, D 141.4 s to 129.5 s with Utot identical to 1e-12 Ha; band_nc D 167.5 s to 156.8 s. The tables themselves (Orbs_Grid of the resident ranks) are always built in FP64

Environment variable	Default	Meaning
`OPENMX_XC_GRID_GPU`	1	The GGA-PBE exchange-correlation potential on the grid (Set_XC_Grid, called by Set_Vpot every SCF step and by Total_Energy / Force) is evaluated on the device: the 27-point gradient stencil, the per-point functional (device transcriptions of XC_PBE, XC_PW92C and XC_EX), the divergence stencil and the non-collinear rotation as four OpenACC kernels over the rank's D grid, in FP64 as on the host, from one transient acc_malloc arena (about 22 doubles per grid point). Falls back to the host loops for LDA, for the LDA+U cFLL rotation (dc_Type 3/4), for XC_P_switch 3 and when the arena cannot be allocated. The host loops are single-threaded per rank and cost 0.36 s per step on every rank of the RTX 5080 (18 ranks, sidia333 non-collinear cluster), the device a few milliseconds: Set_Vpot per step 0.35 s to 0.18 s (the rest is the grid redistribution), Set_Hamiltonian 24.9 s to 20.2 s, D 117.1 s to 111.1 s; 1 rank, the whole grid: 2.38 s to 0.21 s per step. Utot agrees with the host evaluation to 1e-12 Ha (the device library's pow / exp / log differ from glibc in the last bits). **0** = host loops
`OPENMX_CUSOLVER_EMULATION`	1	Every cuSOLVER handle the code creates runs its FP64 eigensolves in the fixed-point FP64 emulation of cuSOLVER 12.2+ (CUSOLVER_FP64_EMULATED_FIXEDPOINT_MATH, performant strategy, dynamic mantissa control), which is what makes the FP64 reference R0 affordable on a GeForce; **0** = native FP64. The emulated solver fails to converge now and then (info > 0: RTX 5080, sidia333, band Col info=395 and band NonCol info=126, twice in roughly 700 solves on 2026-10-08), and the solvers abort on info ≠ 0. Since 2026-10-08 the compat wrapper of cusolverDnXsyevd / cusolverDnXsyevdx therefore keeps a copy of the matrix of every FP64 solve made in an emulated handle (on the device, in host memory when the device is short), answers info > 0 by restoring the matrix and solving once more in native FP64 (message "solving again in native FP64"), and puts the handle back into emulation; the workspace queries return the larger of the two modes' requirements so the retry fits

Environment variable	Default	Meaning
`OPENMX_GRID_PRECISION`	df / fp64	The arithmetic of the grid-integral kernels (the dVH+Vxc+VNA matrix elements and the density): **df** = double-float, FP32 arithmetic with error-free transformations (hi/lo splits, products through FMA, compensated sums per tile, FP64 across tiles; FP64-grade results at about 12 FP32 operations per term), the default on GPUs whose FP32:FP64 throughput ratio is 8 or more (GeForce); **fp64** = the FP64 kernels (the default on data-center GPUs) Since 2026-10-08 the df kernels do no FP64 arithmetic per term: the matrix-element kernel splits grid_vol*vpot once per tile into (hi, lo) and forms the weights with FMAs, the density kernel accumulates its rows as float pairs (TwoSum) and uses FP64 only in the atomic adds, and the lo parts are summed plainly (10 FP32 operations per term); on the RTX 5080 (18 ranks, sidia333 non-collinear cluster) an on-the-fly rank's matrix elements went from 1.42 s to 0.61 s per step, Set_Hamiltonian 35.8 s to 24.9 s, D 129.5 s to 116.3 s, Utot within 1e-12 Ha of the FP64 kernels Since 2026-10-09 the on-the-fly density of mode df runs as a CUDA kernel (set_hamiltonian_gpu.cu density_kernel): the pair's density matrix staged once per block in shared memory as (hi, lo) float pairs, the point's orbital rows in registers (tiers of 16 / 32 / 48 orbitals), the same arithmetic; 0.37 s to 0.098 s per rank and step on the RTX 5080 (1 rank: 3.55 s to 0.23 s), Set_Density_Grid 21.9 s to 17.8 s, Utot within 1e-12 Ha; the OpenACC kernel remains the fallback Since 2026-10-09 the df matrix-element kernel is register-tiled: a thread owns a 2x2 block of elements (spin, i0..i0+1, j0..j0+1) and loads two weighted phi0 pairs and two phi1 values per grid point for its four products (half the shared-memory reads), and the grid points of a pair are split over OPENMX_SETHAM_MATEL_SPLIT blocks (default 4) whose partial sums are added atomically onto the H blocks, which keeps enough blocks in flight to hide the staging gathers (one block per pair alone was slower); RTX 5080, sidia333 non-collinear cluster: the kernel 0.24 s to 0.11 s per on-the-fly rank and step at 18 ranks (1 rank: 2.88 s to 1.09 s), the matrix elements 0.48 s to 0.35 s, Utot identical
`OPENMX_GRID_FP32`	0	Experimental: plain FP32 kernels while the previous step's NormRD is above `OPENMX_GRID_FP32_UNTIL` (1e-5), FP64 afterwards (`OPENMX_GRID_FP32_RESTART` =1 restarts the mixing history at the switch). The FP32 steps stall the SCF near the kernels' error level (sidia333: 27-40 SCF steps instead of 23), so it is off. The configured last SCF step always uses the non-lossy grid kernels, including when `scf.maxIter=1`. Non-finite residuals also end the FP32 phase. A live reduction of maxIter through `_SCF_keywords` after a completed FP32 step cannot retroactively recompute that step
`OPENMX_SETHAM_GPU_RESERVE_MB` / `_RANK_OVERHEAD_MB`	max(total/10,1536) / 512	Reserve and per-rank overhead estimate
`OPENMX_SETHAM_GPU_KOWNER_ONLY`	0	1 restores the old k-owner-only GPU execution
`OPENMX_SETHAM_BASE_GPU`	0	GPU execution of the base combination of H (CPU is faster in practice)
`OPENMX_SETHAM_GPU_TEST_NEED_MB` / `_FROM_CALL`	—	Test hook injecting a fake phase need
Other matrix-element construction		

Environment variable	Default	Meaning
`OPENMX_SETNL_OPENACC`	1	The Set_Nonlocal GPU batch (0 = CPU path)
`OPENMX_SETPRO_GPU`	1	The three Set_ProExpn_VNA batches (0 = CPU)
`OPENMX_SETPRO_VERBOSE`	0	One stderr line per rank and device stage of Set_ProExpn_VNA: the admission figures (need, budget, free memory, pair batch, rank turns), the batch sizes, the elapsed time and the device memory free at that point
`OPENMX_OLPKIN_RADIAL_GPU`	0	GPU radial integrals in Set_OLP_Kin (single thread required; usually slower — transfer-bound)
Density grid (Set_Density_Grid_GPU.c)		
`OPENMX_DENSITY_GRID_GPU` (alias `OPENMX_SETDENSITY_GPU`)	1	Master gate for the density-grid GPU code
`OPENMX_DENSITY_GRID_GPU_LOCAL`	1	Distributed quadrature mode: 0=off, 1=only when the shared tables are resident, 2=always
`OPENMX_DENSITY_GRID_GPU_RESERVE_MB` / `_PERSISTENT`	256 / 0	Owner-service reserve and post-computation device persistence
Total energy and forces (Total_Energy.c / Force.c)		
`OPENMX_TOTALENERGY_GPU`	1	Master gate for the total-energy device kernels (including NC)
`OPENMX_FORCE_GPU`	1	Parent gate for all force GPU paths (collective)
`OPENMX_FORCE3_GPU` / `OPENMX_FORCE3_GPU_DORB`	1 / 1	Force3 GPU trace / on-device dOrbitals recomputation
`OPENMX_FORCE4B_GPU` / `OPENMX_FORCE4B_FUSED`	1 / 1	Force4B GPU batch / the fused CPU trace (also a prerequisite of the GPU batch)
`OPENMX_FORCEHNL_GPU`	1	Force_HNL GPU batch
`OPENMX_FORCE_PROFILE`	0	[Force profile] lines (per-stage max/avg seconds)
Charge/DM mixing (Mixing_H.c etc.)		
`OPENMX_MIX_FAST`	1	CPU fast paths of RMM-DIIS/DIISK/DIISV/GR-Pulay/Kerker (0 = legacy)
`OPENMX_MIXH_GPU`	auto	RMM-DIISH tier selection: unset=auto, 0=legacy, 1=force GPU tier, 2=force incremental CPU tier
`OPENMX_MIXH_GPU_RESERVE_MB` / `_VERBOSE`	max(total/10,1536) / 0	GPU-tier reserve / print the tier decision
gpusolver2 (ELPA-GPU / COSMA)		
`OPENMX_GS2_ELPA_SOLVER`	2	ELPA solver used by gpusolver2: 1 = 1stage, anything else = 2stage (default)
`OPENMX_CLUSTER_GS2_FALLBACK`	auto	Capacity fallback of `gpusolver` in the cluster solvers when the dense solve does not fit on one GPU: MAGMA on the GPUs of rank 0's node first, then (multi-node runs) the distributed gpusolver2 path on every rank of the run (a spin-polarized collinear run solves its spins in turn on a grid over all ranks; ELPA/COSMA use the GPU of each rank), with a collective GPU admission. `OPENMX_CLUSTER_HOST_MEM_AVAILABLE_MB` overrides the host memory of the MAGMA test. `off` = never the distributed step, `on` = also on one node, `force` = skip MAGMA. Forced MAGMA (gpusolver3, `OPENMX_MGPU_MEMORY=host`) never takes the distributed step

Environment variable	Default	Meaning
`OPENMX_GS2_HOST_RESERVE_MB`	a quarter of the node memory (at least 4096)	gpusolver2 bridge (explicit or as the capacity fallback): host memory that must stay free on every node after COSMA allocates the buffers it keeps for the rest of the run (memory pool and COSTA send/receive buffers, estimated from COSMA's own strategy); otherwise the transform runs on ScaLAPACK

Table 10-1: GPU-related environment variables by subsystem

10.3 GPU Diagnostic Lines on stdout

Commit `30148e1e` and friends made it possible to read off which stages ran on the GPU. How many of these lines appear follows OpenMX's input keyword `level.of.stdout` (the notices on stderr included; the `OPENMX_*_PROFILE` and `*_VERBOSE` variables of § 10.2 work independently of it):

`level.of.stdout`	GPU lines printed
1 (default)	The "(GPU-accelerated)" and "(GPU reduction)" marks on OpenMX's own stage lines, errors, a GPU that cannot be used (`GPUSOLVER requested, but GPU initialization failed on ⋯ MPI ranks; using ELPA2.`), and a GPU eigensolver given up for lack of device memory or after a failure (`Falling back to the ELPA/ScaLAPACK diagonalization⋯` , a DC or DC-LNO rank that falls back to the CPU solver), and a cluster dense diagonalization kept on the GPU without the basis of the refined eigensolver (`⋯ holds its GPU diagonalization buffers but not the ⋯ basis of the refined eigensolver ⋯: FP64 eigensolver for this SCF cycle`)
2	Also the device plans, cache admissions and banners, the kernel stack limit raised at device initialization (`<GPU> kernel stack limit raised from ⋯ to ⋯ bytes per thread`), the FP64 re-solve of the step that ends the SCF, and the fallbacks and retries of single stages and solves, one line per node (Force4B, Force_HNL, Set_ProExpn_VNA, Set_Density_Grid, Mixing_H, GEMMul8's workspace fallback, the cuSOLVER emulation and FP32 retries)
3	Also the per-step, per-k-point and per-rank details: the refinement residuals and warm starts of the refined FP32 eigensolver (about 38,000 of the 85,600 lines of an 18-rank `-runtestL` before this gating), every rank's own figures in the Force4B, Force_HNL and HVNA fallback notices and in the on-the-fly density workspace notices of Set_Density_Grid, and the on-the-fly banner at every change of the workspace

From level 2 on, OpenMX itself prints its per-atom debugging output (the 216-atom noncollinear sidia333 cluster, 4 SCF steps on 18 ranks: 726 lines at level 1, about 51,000 at level 2 or 3), so the GPU details of level 3 add little. `tools/run_forward_variants.sh` runs at level 3 unless told otherwise, as its summary reads these lines. Representative lines (printed by Host_ID or each device's lead rank):

- Stage banners (level 1): `<Set_Hamiltonian> Hamiltonian matrix for VNA+dVH+Vxc (GPU-accelerated)...` , `<MD=⋯> Calculation of the VNA projector matrix (GPU-accelerated)` , `Force calculation #2 (GPU reduction)` / `#3 (GPU-accelerated)` / `#4 (GPU-accelerated)` / `#5 (GPU reduction)` , `Force calculation #6/#7 (GPU-accelerated)` (Total_Energy side)
- Concurrency decisions (level 2, printed only on change): `<Band_DFT_Col> GPU device 0: 8 k-owner rank(s), GPU concurrency=3, turn group=3, per-rank=1.9 GiB, peak=5.7 GiB, free=7.4 GiB, reserve=1.5 GiB` (NonCol adds the mode name; Set_Hamiltonian says "Hamiltonian rank(s)")
- Plans and cache admissions (level 2): `<DC> GPU S-cache: rank 0 keeps 12 of 16 cluster overlaps on the device (812.5 MiB)` , `<Krylov> GPU KU-cache: ⋯` , `<Set_Hamiltonian> GPU resident cache: ⋯` , `<Set_Density_Grid> distributed GPU quadrature: every rank integrates its own atoms` , `<Set_Density_Grid> one-rank GPU service cache: owner=⋯ atoms=⋯ atom-grids=⋯ terms=⋯ device=⋯ GiB`

- Fallback notices: `Falling back to the ELPA/ScaLAPACK diagonalization⋯` (§ 5.6, level 1), `Force4B GPU: not enough free device memory ⋯; CPU fallback.` (level 2, once per node; level 3 prints every rank's `Force4B GPU: ⋯ GiB free for ⋯ rank(s) but ⋯ GiB needed per rank; CPU fallback.`), `<Mixing_H> The RMM-DIISH residual history ⋯ does not fit on the GPU ⋯` (level 2), `<Set_Density_Grid> ⋯ cannot hold a minimum on-the-fly chunk ⋯` and `<Set_Density_Grid> CPU loop on ⋯ rank(s) with atoms: ⋯` (level 2 on Host_ID when the situation changes; level 3 adds every rank's own line, § 7.9), and GEMMul8's `workspace fallback` (stderr, level 2 or `OPENMX_GEMMUL8_VERBOSE=1`)

> **How to read them:** when a run that "should be on the GPU" is slow, run it with `level.of.stdout 2` and check these lines first. The classic cases: (a) an ELPA-fallback line means diagonalization dropped to the CPU for lack of VRAM; (b) a GEMMul8 workspace-fallback line means the GEMMs dropped to FP64; (c) concurrency=1 means the sharing ranks were serialized; (d) a `<Set_Density_Grid> CPU loop on ⋯` line means ranks integrated the density on the CPU (§ 7.9).

10.4 Tuning Guidelines

1. **Run with the defaults first** — they are tuned for "a 16 GB-class GPU shared by many ranks", and concurrency, residency, and fallbacks are all automatic. Unlike in the 2nd edition, manually setting concurrency via environment variables is normally unnecessary.
2. **Check the stdout/stderr diagnostics** (§ 10.3) — always the starting point.
3. **With plenty of VRAM (exclusive GPU)** — `OPENMX_BAND_GPU_PERSISTENT=1`, `OPENMX_GPUSOLVER_SERIAL_GPU_TURNS=0`, and `OPENMX_*_GEMMUL8_TURN_RELEASE=0` can reduce transfers and reallocations. On Krylov runs the diagonalization now moves onto the device by default (`OPENMX_KRYLOV_GPU_EIGEN_MIN`=300; measured 117.5 s → 78.8 s; § 7.6) — lowering it further rarely pays.
4. **When VRAM is tight** — reducing the rank count helps most; then raise the `*_RESERVE_MB` knobs to avoid GEMMul8 fallbacks, or disable residency with `OPENMX_SETHAM_GPU_RESIDENT=0`.
5. **For maximum reproducibility** — pin the GEMMs to FP64 with `OPENMX_GEMMUL8_DISABLE=1` (§ 4.8); to also freeze the GPU⇄ELPA choice, set `OPENMX_BAND_GPU_DIAG` / `OPENMX_CLUSTER_GPU_DIAG` explicitly.
6. **Bisecting problems** — every subsystem has its own gate (`OPENMX_SETHAM_MATRIX_GPU=0`, `OPENMX_FORCE_GPU=0`, `OPENMX_MIX_FAST=0`, …), and every fast path preserves the legacy code, so a suspected GPU issue can be bisected gate by gate.

10.5 scf.eigen.lib=gpusolver2 — Distributed Multi-Node Cluster Diagonalization

For collinear/non-collinear **cluster** calculations (a single k point), the default gpusolver path starts with one GPU and can recover from capacity limits using MAGMA and distributed solvers. `scf.eigen.lib gpusolver2` explicitly selects distributed cluster diagonalization with **ELPA 2026.02.002 (CUDA or HIP GPU kernels, the modern object-based C API)** and **COSMA (communication-optimal distributed GEMM, GPU backend)**, engaging every rank and every GPU. The original CUDA target is a Pegasus-class layout (one H100 per node, many nodes). CUDA uses the selected NVHPC/MPI toolchain; HIP needs MPI compatible with ROCm flang (§ 9.7).

- **Design** — the existing ELPA-path skeleton (2-D block-cyclic distribution, the `myworld1` two-spin split, eigenvector redistribution, DM construction) is reused verbatim; only the call sites change. Col: S diagonalization → `openmx_gs2_eigen_real(n,n,⋯)`, the three transformation GEMMs → `cosma_pdgemm_`, the transformed H (all eigenvalues, lowest MaxN eigenvectors) → `openmx_gs2_eigen_real(n,MaxN,⋯)`. NonCol: real S

solve + 12 real GEMMs + the complex 2n×2n `openmx_gs2_eigen_complex(n2,MaxN,⋯)` + back-transform `cosma_pzgemm_`. OpenMX's own numerics (small-eigenvalue clamping of S, etc.) are untouched. The bridge lives in the new `elpa_cosma_bridge.c` (an ELPA-handle cache keyed by (BLACS ctx, n, nev, type), `nvidia-gpu=1` or `amd-gpu=1`, `use_gpu_id`, 2stage solver by default).

- **Shared CPU/GPU ELPA** — `elpa1`, `elpa2`, and `gpusolver2` all use the bundled ELPA 2026.02.002 library through its modern C API. `openmx_elpa_cpu.c` selects CPU solvers; `elpa_cosma_bridge.c` selects the GPU backend. The former embedded ELPA 2018.05 has been removed (§ 9.4).
- **Scope and the error stop** — mainline cluster (Col/NonCol) only; any other `scf.EigenvalueSolver` stops with an input error (Band with `scf.Kgrid 1 1 1`, which auto-converts to Cluster, is allowed). Everything outside the cluster diagonalization (SetH/density/force/mixing GPU paths, CWF post-processing, …) behaves exactly like gpusolver (internally `scf_eigen_lib_flag` stays GPUSOLVER and `gpusolver2_flag` switches just the diagonalization). Without a GPU the run demotes to ELPA2, as with gpusolver.
- **Why not DLA-Future (for the record)** — the feature was first implemented and tested with DLA-Future + COSMA, but DLA-Future's nonblocking collectives reliably break on the HPC-X Open MPI 4.1 with 3+ ranks (MPI_ERR_TRUNCATE/hangs; PMPI traces confirmed our collective streams were correct; only Open MPI 5 cures it). To avoid swapping the MPI of the whole code, ELPA's GPU kernels — proven on Open MPI 4 — were adopted instead. COSMA itself was verified healthy on Open MPI 4.1 (np1/4/8 agree with ScaLAPACK to ≤1e-12).
- **Self-contained build** — `third_party/dist/` bundles the official release archives of ELPA, COSMA, COSTA, Tiled-MM, and cmake (≈23 MB total, with `README.md` and `SHA256SUMS`). `make openmx` triggers `make gpusolver2-stack` automatically on the first build and installs into `third_party/gpusolver2-install/` without network access. ELPA comes as a GitLab tag archive, so autoconf/automake/libtool/m4/python3 are required (standard on Ubuntu). Key Makefile variables: `GPUSOLVER2_GPU_ARCH` (default: nvidia-smi autodetect; 90 for H100), `GPUSOLVER2_HOST_CC/CXX` (GCC for COSMA).
- **Runtime knob** — `OPENMX_GS2_ELPA_SOLVER=1` switches to ELPA's 1stage solver (default 2stage); see Table 10-1.
- **Pegasus recipe** — after `module load nvhpc`: `cd source && make openmx && make install` (the stack builds once), then run with the usual HPC-X `mpirun`. The GPU architecture can be forced with `make GPUSOLVER2_GPU_ARCH=90 gpusolver2-stack`.

11. Summary of Constraints and Caveats

11.1 Build and Runtime Environment Assumptions

- The CMake interface selects `CUDA` or `HIP`; it has no CPU-only backend. Both executables support CPU execution with `OPENMX_GPU=0` or `elpa1` / `elpa2`, using the shared modern ELPA (§ § 9.4, 9.8).
- CUDA defaults to NVHPC 26.9 + CUDA 13.4. HIP requires ROCm, compatible MPI, and AOCL (§ 9.7). Re-verify the CUDA miscompilation workarounds of § 6.5 when changing NVHPC.
- GEMMul8's CUDA/HIP build and the native FP64 selection follow the target device. Rebuild the GPU libraries for a different architecture; CUDA-specific controls do not automatically apply to HIP.
- Flat MPI is the first choice while every core can hold a rank. When the device memory bounds the number of ranks below the core count, OpenMP threads (`-nt`, one rank bound to several cores: `mpirun --map-by ppr:6:node:PE=3 --bind-to core -np 6 ./openmx in.dat -nt 3`) now pay on the host-side phases (§ 10.4): the Bessel cache of Set_OLP_Kin is per thread, the cluster density matrix and the received-atom products of Set_Nonlocal are threaded over atoms, and the vanilla OpenMP regions (truncation, Set_ProExpn_VNA, Set_Orbitals_Grid, Force, Total_Energy) run as they do on the CPU. The device phases do not change with the thread count, and with threads the summation order of the vanilla OpenMP reductions makes runs non-deterministic at the 1e-11 level; some mixing fast paths (batched Gram) match the legacy summation order only with one thread. The opt-in OpenACC radial integration of Set_OLP_Kin (`OPENMX_OLPKIN_RADIAL_GPU=1`) still requires one thread.

11.2 Memory-Planning Rules of Thumb

- Per-rank VRAM demand for a band run of dimension n: 3 complex dense matrices of $16n^2$ bytes + the syevdx workspace (use the real `_bufferSize` figure; roughly $4n^2$ complex elements) + the GEMMul8 workspace (~1 GiB; complex ≈ 3× real) + 256 MiB overhead — exactly the model implemented by `BandCol_GpuTurnRequiredBytes`.
- On a shared GPU, "concurrent ranks × the above" must fit in free VRAM, but **every path now shrinks, serializes, or falls back to ELPA automatically**; an oversubscribed configuration degrades performance instead of hanging or aborting. The 2nd edition's caveat that the cluster root-dense and DC-LNO paths lacked automatic throttling is obsolete.
- Watch host memory too: the Set_Hamiltonian / density-grid shared tables are multi-GiB, and the Ready probe exists precisely to keep CPU-executing ranks from building them; per-rank host workspaces (mixing history, cluster scratch) scale with the rank count.

11.3 Numerical Reproducibility

- **What can break bitwise reproducibility:** (1) the memory-driven GEMMul8 ⇄ cuBLAS FP64 switch; (2) the runtime GPU ⇄ ELPA fallback (verdicts are sticky within an SCF cycle but depend on the memory situation of the run); (3) `acc atomic` scatter-adds (nondeterministic order); (4) the per-path small-eigenvalue treatment of S; (5) GEMMul8 environment settings (num_moduli / fastmode).
- **What preserves it:** `loop seq` reductions in DM builds and grid traces; force batches reproducing the CPU down to the float-multiply→double-add cast; pointer-rotated mixing histories (bitwise equivalence); cublasDgeam transposes (pure data movement); GEMMul8's own bitwise reproducibility; the bit-pattern-keyed Bessel cache. Most GPU commits used "Uele/forces bit-identical to the CPU" as their acceptance test.

- For validation runs, pin the conditions: `OPENMX_GEMMUL8_DISABLE=1`, ample VRAM, and if needed explicit `OPENMX_*_GPU_DIAG` settings.

11.4 Hangs and Aborts

- `wait_cudafunc` loops forever on a permanent error (§ 3.6). However, the "permanently out of VRAM" cases have moved to bounded retries plus fallbacks (the `TryDeviceMalloc` helpers, the 180 s cap of `Force_gpu_arena_wait`), so silent freezes are far rarer; the force stages contain no allocation that can abort at all (§ 7.11).
- Behavior on eigensolver failure differs per path — abort / CPU fallback / pass-through (Table 5-2).
- The fits verdicts, wave planning, and mixing preflight perform MPI collectives internally, so **all ranks must reach them the same number of times**; skipping them on a subset of ranks deadlocks (§ 5.9).
- The collective device-assignment helpers must be called by all ranks together.
- The `cudaGetLastError()` per-thread latch may contain leftovers of recovered failures; follow the pop-before-launch etiquette when adding kernel checks (§ 3.6, § 7.7).

11.5 Integrated AMD HIP Backend

The AMD implementation is part of this repository. The source manifest and CMake build select its files without placing HIP compatibility headers on the CUDA include path. CUDA and HIP are separate executables; there is no runtime switch between vendors.

Layer	CUDA	HIP
Runtime and kernels	CUDA, NVHPC OpenACC	HIP, ROCm OpenMP target offload
Dense libraries	cuBLAS / cuSOLVER	hipBLAS / hipSOLVER and compatibility wrappers
GEMMul8	CUDA instantiation	HIP instantiation; MI300A/Radeon arithmetic policies
Distributed solve	ELPA NVIDIA kernels / COSMA CUDA	ELPA AMD kernels / COSMA ROCm
CPU eigensolvers	Shared ELPA 2026.02.002, with GPU options disabled	

`hip_runtime_compat.h`, `hipblas_compat.h`, and `hipsolver_compat.h` expose the required interfaces to C sources; C++ translation units include ROCm's headers. The hipSOLVER wrappers translate generic calls to typed FP32/FP64 real/complex entry points. CUDA-named shim headers remain for bundled libraries that include those names. GEMMul8 and native FP64 are selected by the AMD device policy; the NVIDIA-only details of Chapters 3–6, including OpenACC pool handling and cuSOLVER emulation, must be read in that scope.

CPU execution without a visible GPU: the non-collinear dense-workspace cleanup now skips HIP synchronization and device restoration when it owns no GPU resources. Live or partially allocated workspaces still release their resources, and host state is always reset. This fixes CPU-only runs of the HIP executable with hidden devices (`57a3a3fc`; validation in § 9.8).

MI300A: set `HSA_XNACK=1` for GPU execution. The launcher `source/backends/hip/tools/mi300a_rank_launcher.py` fixes CPU placement before ROCm initialization, using OpenMPI rank information and `OPENMX_AFFINITY_DIR`, and checks the allocated CPU count against `-nt`. The full launch recipe is in `docs/GPU_BACKENDS.md`. NVIDIA MPS is specific to CUDA; it is not the HIP launch mechanism. The recorded Radeon tests do not establish MI300A or multi-node hardware validation (§ 9.8).

11.6 Non-GPU Behavioral Differences from the Original

- The two OpenMP heap-corruption bugs are fixed (§ 8.1) — bugs that exist in the original.
- The soft stack limit is raised automatically at startup (§ 8.2).
- runtest's detection and execution are stricter (§ 8.3).
- The version string is "4.0 GPU".

11.7 Dormant and Preparatory Code (to Avoid Misreading)

Code	Status and role
`gpusolver_Syevd` (the Xsyevd variant)	Zero callers (unified onto Syevdx); still carries the old "global rank % device count" assignment
`LU_Zinverse_GPU`	Implemented, call commented out; groundwork for GPU NEGF surface Green's functions (§ 5.8)
GEMMul8 helpers in the DMmu/CWF/LNO files, `BandCol_GpuSolver_Zgemm_OpenACC`, etc.	Defined only (groundwork for future GEMM offloads). `my_cublasDgemm/Zgemm` are deleted; `openmx_cuda_kernels.cu` is removed from the tree (§ 6.4)
The DC-LNO GPU proxy machinery	Preserved behind `DCLNO_ENABLE_SHARED_GPU_PROXY 0` (experimental)
`use_force4b_openacc = 0` (Force.c)	Permanent disable constant of the old Force4B OpenACC path (the live paths are the GPU batch and the fused CPU trace)
`OutData_Binary.o` / the `getDeviceCount()` declaration	Dead link / dead declaration
The `OPENMX_SETHAM_GPU_TEST_NEED_MB` hooks	Test-only (phase-need injection); not for production

Table 11-1: Dormant and preparatory code

Appendix A. Changed Files (July 2026; 47 files)

+n/−m are raw diff line counts against the original OpenMX 4.0 sources (as of 2026-07-23). Wholly re-indented files show raw counts larger than their substantive change. Categories: **GPU** = GPU offloading, **OMP** = OpenMP fix, **FIX** = bug fix / hardening, **OPT** = CPU optimization, **BLD** = build, **MISC** = other.

File	+n/−m	Cat.	Summary
`Force.c`	+15,123/−11,241	GPU	GPU offloading of forces #2/#3/#4/#4B/#5/HNL (fused traces, chunked batches, on-device dOrbitals, arenas + peer waiting). Part of the raw count is whole-file re-indentation
`Divide_Conquer.c`	+4,816/−3,038	GPU	DC on GPU: per-atom Xsyevdx, GEMMul8→cuBLAS→CPU GEMM cascade, device-resident S12 cache, cublasDgeam transposes, sticky disable
`Band_DFT_Col.c`	+4,724/−2,544	GPU	Full GPU offload of the collinear band path: dense-construction cache, Xsyevdx (NOVECTOR first pass), GEMMul8 congruence transform, adaptive concurrency, ELPA fallback
`Band_DFT_NonCol.c`	+3,582/−88	GPU	Noncollinear band on GPU: OpenACC-resident root-dense path, stage-specific (ev/vec/DM) adaptive concurrency, ELPA fallback
`Set_Hamiltonian.c`	+2,972/−1,149	GPU	dVH+Vxc+VNA grid integrals on GPU (default on): adaptive wave scheduling, SCF-invariant cache, on-device Vpot gather, resident cache with need-yielding, packed H/S cache API
`Cluster_DFT_NonCol.c`	+2,424/−218	GPU	Root-dense diagonalization on a single arena, GEMMul8 × 12, cross-iteration caches for S/Ss2/index/handle/workspace, eigenvector residency/demotion, ELPA fallback
`Divide_Conquer_LNO.c`	+2,315/−1,896	GPU	Direct per-rank dispatch (collinear) plus GPU noncollinear cluster solves (spin-doubling transform + Zgemm×3); group-aggregate memory test
`Set_ProExpn_VNA.c`	+2,029/−0	GPU	Three GPU batches: DS_VNA construction, HVNA traces, VNA2/3 (device Bessel recurrences; host-tabulated Gaunt/transforms)
`Set_Nonlocal.c`	+1,738/−305	GPU/FIX	Batched GPU nonlocal-projector integrals (device-generated Bessel tables + Gaunt pre-tabulation) + heap-based array hardening
`Cluster_DFT_Col.c`	+1,534/−90	GPU	Root-dense GPU diagonalization, real-reservation probe + ELPA fallback, two-spin serialization, GPU DM/EDM accumulation, device-resident eigenvectors
`Mixing_H.c`	+1,350/−0	GPU	Two-tier RMM-DIISH fast path (GPU-resident Gram / incremental CPU)
`Krylov.c`	+840/−24	GPU	Fused per-atom projected-solve pipeline (3 GEMMs + Xsyevd in one device visit), KU cache, per-thread workspaces
`Set_OLP_Kin.c`	+528/−180	GPU/OPT	OpenACC radial integrals (opt-in) + bit-exact Bessel table cache
`Total_Energy.c`	+367/−123	GPU/OMP	Calls into the GPU reductions for the EH0 two-center batch, EXC/EH1, dipole, CWF-Dc, and the Exc0 fine mesh (NC included) + the OpenMP private fix
`DIIS_Mixing_DM.c`	+265/−2	OPT	Batched Gram for RMM-DIIS (one sweep + one Allreduce) + pointer-rotated residual history
`Mixing_V.c`	+246/−53	OPT	Incremental Gram for RMM-DIISV; faster Kerker (VKSk) shifts
`DFT.c`	+235/−4	GPU	GPU device init, Set_Hamiltonian work-rank selection, the pre-force bulk GPU-cache release hooks, the VNA banner, NonCol eigenvector scatter

File	+n/−m	Cat.	Summary
`DIIS_Mixing_Rhok.c`	+228/−44	OPT	Incremental Gram for RMM-DIISK (new residual row only via dgemv + one short Allreduce)
`GR_Pulay_DM.c`	+204/−2	OPT	Batched Gram + fused mixing sweep for GR-Pulay
`Eigen_lapack.c`	+143/−8	GPU	Branch to gpusolver_Syevdx at n≥1000 plus an OpenACC-direct entry; 20+ callers benefit automatically
`openmx_common.h`	+142/−1	GPU/MISC	CUDA headers, GPUSOLVER enum, wait_cudafunc (with latch cleanup), GPU prototypes, the SetHamiltonianMETables type, include guard, Version "4.0 GPU"
`Lapack_LU_inverse.c`	+134/−4	GPU	LU_Zinverse_GPU via cublasZgetrf/ZgetriBatched (call commented out = dormant)
`EigenBand_lapack.c`	+112/−3	GPU	Complex variant: gpusolver_zheevx branch, Eigen_HH fallback
11 DMmu/CWF/LNO/polarization files (§ 7.13)	+27 to +76 each	GPU	The stereotyped GPUSOLVER branch before the ELPA2 branch (Dsyevx/Zheevx via dense_utils)
`Kerker_Mixing_Rhok.c`	+62/−10	OPT	Faster Rhok history shifts (rotation + memmove)
`openmx_common.c`	+60/−0	GPU	The GPU phase-need registry (OpenMX_GpuPhaseNeed_*)
`Set_Density_Grid.c`	+40/−5	GPU	The three-stage dispatch: distributed GPU quadrature → one-rank GPU service → CPU
`Free_Arrays.c`	+28/−17	GPU/FIX	GPU-cache invalidation + GEMMul8 workspace release at exit + NULL after free
`truncation.c`	+23/−13	FIX/GPU	NULL after freeing VNA/VEF grids + GPU-cache invalidation hooks
`Runtest.c`	+19/−7	FIX	Strict .dat detection; stale .out removal before each test
`openmx.c`	+16/−0	FIX	Raise_Stack_Limit() (automatic stack-limit raise)
`Input_std.c`	+15/−3	GPU	gpusolver added to scf.eigen.lib (cusolver deprecated alias); scf.Gpu.Num
`Spherical_Bessel.c`	+12/−24	OPT	Per-call malloc/free removed from the hot path
`tran_prototypes.h`	+10/−0	GPU	CUDA_CHECK macro
`Set_Orbitals_Grid.c`	+8/−3	GPU	GPU-cache invalidation hooks on orbital-grid rebuilds
`Stress.c`	+1/−1	OMP	Adds i to the OpenMP private list (heap-corruption fix)
`mpao.c`	+1/−0	MISC	One include line (outside the build)
`makefile` → `Makefile`	full rewrite	BLD	From Intel oneAPI+MKL to NVHPC+CUDA+in-tree FFTW+GEMMul8 (Chapter 9); single-make pristine-tree builds

Notes: the deleted sources are `my_cublasDgemm.c` / `my_cublasZgemm.c` (dormant wrappers) in the historical comparison above. The old `elpa-2018.05.001/` tree and legacy solver wrappers have since been removed in favor of the shared ELPA 2026.02.002 library and `openmx_elpa_cpu.c` (§ 9.4). `Set_ProExpn_VNA.c` — "no diff against the original" in the 2nd edition — plus the six mixing files and the density grid were newly modified by the 40 commits leading to this edition.

Appendix B. Newly Added Files (July 2026)

B.1 New Sources (14 files, about 4,000 lines)

File	Lines	Role
`Set_Density_Grid_GPU.c`	1,366	Density grid on GPU (one-rank GPU service + distributed GPU quadrature)
`Total_Energy_GPU.c`	1,064	OpenACC kernels for the total energy (EXC/EH1, dipole, CWF-Dc, EH0 batch, Exc0 fine-mesh batch); a separate TU because of the -acc split build
`gpusolver_Syevdx.c`	516	cusolverDnXsyevdx partial-eigensolver wrappers (real/complex × host/OpenACC/cached — five variants); the workhorse of GPU diagonalization
`gemmul8_bridge.cu`	419	GEMMul8 bridge: workspace cache, size-query API, env-var control, cuBLAS fallback
`set_hamiltonian_gpu.cu`	194	Shared-memory-tiled CUDA kernel for the Set_Hamiltonian grid integrals (stale-error tolerant)
`gpusolver_Syevd.c`	131	cusolverDnXsyevd wrapper (from the NVIDIA CUDA Library Samples); currently unused
`openmx_gpusolver_dense_utils.h`	79	Shared dense-diagonalization dispatch helpers (0/1-based conversion + MPI_Abort on failure); used by 11 files
`gemmul8_openmx.cu`	67	Single-TU explicit instantiation of the GEMMul8 templates (4 symbols)
`set_cuda_default_device_from_local_rank.c/.h`	73+9	Round-robin CUDA device assignment by intra-node local rank
`set_openacc_device_from_local_rank.c/.h`	79+12	The OpenACC twin (acc_set_device_num)
`LU_Zinverse_GPU.h`	9	Declaration of the GPU complex LU inverse (dormant)

Notes: the former generic wrappers `my_cublasDgemm.c` / `my_cublasZgemm.c` were deleted (§ 3.5); the unwired kernel collection `openmx_cuda_kernels.cu/.h` from the 2nd edition is gone from the tree (§ 6.4).

B.2 Build and Documentation Files

File	Role
`Makefile` (1,377 lines)	The new build system, with commented configuration examples for several supercomputers. NVHPC 26.3 + CUDA 13.1 defaults. A single make (-j allowed) from a pristine tree builds everything including third_party (§ 9.3)
`Makefile.bunshiken` (1,366 lines)	Variant for a shared machine (NVHPC 25.9-nompi + OpenMPI 5.0.8 + gcc-toolset-15); more than a generation old
`makefile.cpu_intel.bak` (1,383 lines)	Byte-identical backup of the original makefile (Intel CPU configuration)
`third_party/`	GEMMul8 (submodule, pinned to 6bf409b = v3.1.1), fftw3 (submodule, tag fftw-3.3.11), fftw-3.3.11.tar.gz (codelet overlay). All fetched automatically by make when missing
`doc/`	This guide (Japanese/English, HTML/PDF)

Appendix C. Development History (July 2026; 91 commits)

#	Hash	Title (verbatim)
1	`0cf2acbc`	Initial commit
2	`60b6aa1e`	Performed GPU acceleration
3	`2f94ca06`	Performed GPU acceleration on Divide_Conquer_LNO.c
4	`e3fc656e`	Optimized Band_DFT_NonCol.c
5	`24e8b34f`	Optimized Cluster_DFT_Col.c
6	`33e9459b`	Optimized Cluster_DFT_Col.c
7	`82579d12`	Bug fix
8	`33a07450`	Fixed bugs in Band_DFT_Col.c and Cluster_DFT_Col.c
9	`b4221bba`	Fixed a bug in runtestL
10	`18d92700`	Bug fix
11	`563f69bd`	Add files that were not included in the commit
12	`1b020264`	Optimized Band_DFT_Col.c and Band_DFT_NonCol.c
13	`f4041307`	Optimized Set_Hamiltonian.c
14	`9f1633da`	Optimized Set_OLP_Kin.c
15	`d6b463a8`	Explicitly indicate that the generation of overlap matrices is GPU-accelerated
16	`5383544b`	Optimized Total_Energy.c
17	`c9e111ea`	Optimized Set_ProExpn_VNA.c
18	`8162b8ec`	Optimized Set_ProExpn_VNA.c and Total_Energy.c
19	`91416fb1`	Bug fix
20	`24a113a9`	Add files that were not included in the commit
21	`87ea7062`	Bug fix
22	`faf76fa2`	Bug fix
23	`a8224724`	Add .gitignore for build artifacts and runtime output
24	`d3c45cb3`	Optimized Bnad_DFT_NonCol.c
25	`db7b9327`	Add files that were not included in the commit
26	`a61f24b3`	Bug fix
27	`d6d19f99`	Add README.md
28	`a91f920f`	Bug fix
29	`88b135f4`	Optimized Krylov.c
30	`34cf8bc1`	Optimized Divide_Conquer_LNO.c
31	`685a2bc9`	Reverted Set_ProExpn_VNA.c to the CPU version
32	`2aa31a43`	Add files that were not included in the commit
33	`d3effce1`	Bug fix of Divide_Conquer.c
34	`34b52269`	GPU acceleration for paths where all_knum !=1 in Band_DFT_Col.c
35	`75ef67c7`	GPU acceleration for paths where all_knum !=1 in Band_DFT_NonCol.c
36	`1e0c777e`	Remove runtest.result from Git

#	Hash	Title (verbatim)
37	`c8c1e3e1`	Modify the Makefile
38	`cb4d348d`	Now compatible with the latest GEMMul8
39	`d451eeb8`	Fixed OpenMP bugs in Total_Energy.c and Stress.c
40	`2302f965`	The name has been changed from CuSOLVER to GPUSOLVER
41	`d5b897fd`	Band_DFT_Col: serialize GPU turns in pairs and release GEMMul8 workspace per turn
42	`788acb4c`	Delete unnecessary files, such as the MAGMA library
43	`d8219384`	Rename the identifier from cusolver to gpusolver
44	`8647a5ed`	Set large2_example and Large3_example to be processed on the GPU.
45	`f508a2a6`	Enable direct GPUSOLVER dispatch for DC-LNO
46	`3545a9d5`	Edit README.md
47	`ca3d97c2`	Edit README.md
48	`43e30c65`	Limit band GPU solver concurrency adaptively
49	`66ce1e62`	Use GEMMul8 for remaining GPU GEMM paths
50	`a7f8cba6`	Remove obsolete cuBLAS wrapper files
51	`8c7801fa`	Limit band GPU k-turn concurrency
52	`64da9a3d`	Add Implementation Manual
53	`bff93d08`	Add single-rank GPU density grid service
54	`95fa51ed`	Accelerate Hamiltonian construction with adaptive GPU scheduling
55	`fe68cca6`	Accelerate force evaluation with fused contractions
56	`58ea6471`	Adapt band GPU k-turn concurrency to device memory
57	`7c414f91`	Adapt non-collinear band GPU concurrency to device memory
58	`75fc30d3`	Accumulate cluster density matrix on the GPU
59	`a9b6fa31`	Trim per-iteration GPU overhead in noncollinear cluster path
60	`d62cac60`	Run noncollinear DC-LNO cluster solves on the GPU
61	`f5de630e`	Cut per-iteration overhead in the noncollinear cluster GPU path
62	`527ed524`	Keep collinear cluster eigenvectors on the GPU for the DM step
63	`211f0684`	Batch the nonlocal-projector integrals on the GPU in Set_Nonlocal.c
64	`6b84a1f9`	Start the Set_Hamiltonian GPU wave at the full device-sharing rank count
65	`48921b36`	Start the Band_DFT_Col k-point wave at the full device-sharing rank count
66	`99f4014b`	Start the Band_DFT_NonCol k-point wave at the full device-sharing rank count
67	`697383e6`	Run the Force3 grid trace on the GPU
68	`7d38382d`	Batch the Force4B and Force_HNL traces on the GPU, move dOrbitals on device
69	`19b9b69c`	Run the Total_Energy device kernels in the NC case, batch the Exc0 mesh
70	`77d2a22c`	Batch the Set_ProExpn_VNA construction and traces on the GPU
71	`28361f7e`	Reuse fixed-size device buffers across Force3 trace chunks
72	`80a3a8f7`	Drop the one-rank GPU service cache diagnostic line
73	`b569f272`	Print the GPU-fallback notice once per node, not once per rank
74	`098adddc`	Fixed a bug in Force.c

#	Hash	Title (verbatim)
75	`bff0af37`	Reduce Set_Hamiltonian GPU staging with on-device Vpot gather and a resident cache
76	`69dde552`	Make the Set_Hamiltonian resident cache yield to published GPU-phase needs
77	`30148e1e`	Label the GPU-accelerated VNA projector and force stages on stdout
78	`d14ef297`	Add a distributed GPU quadrature to Set_Density_Grid
79	`6f7f31b2`	Cut per-atom staging overhead in the GPU divide-conquer solver
80	`3ee45228`	Fuse the per-atom GPU pipeline in the Krylov O(N) solver
81	`ae3dd189`	Keep large cluster GPU runs from freezing or exhausting memory
82	`206b7e06`	Make the force-stage GPU batches immune to shared-device memory races
83	`cff323e6`	Fall back to ELPA when the cluster GPU diagonalization cannot reserve its buffers
84	`d0e1d17a`	Close the last fatal allocation windows in the force-stage GPU batches
85	`3ef5347a`	Fall back to ELPA when the non-collinear cluster GPU diagonalization cannot reserve its buffers
86	`a4200a04`	Fall back to ELPA when one k-point's band GPU diagonalization cannot fit
87	`94b6ca8c`	Accelerate RMM-DIISH mixing with a two-tier GPU/incremental fast path
88	`6e94a9d3`	Add fast paths to the remaining charge/DM mixing schemes
89	`c5eb6845`	Fix phantom Set_Hamiltonian abort caused by stale latched CUDA errors
90	`bf08a9d8`	Serialize the two-spin cluster GPU diagonalization when one device cannot hold both owners
91	`9fe8ce13`	Make a pristine tree build with a single make invocation

References

1. OpenMX official site: `http://www.openmx-square.org/` (code, manual, input keywords)
2. GEMMul8 (GEMMulate) repository: `https://github.com/RIKEN-RCCS/GEMMul8` (MIT license; lead developer Yuki Uchino, RIKEN R-CCS)
3. Y. Uchino, K. Ozaki, T. Imamura, "High-Performance and Power-Efficient Emulation of Matrix Multiplication using INT8 Matrix Engines", SC Workshops '25, doi:10.1145/3731599.3767539
4. K. Ozaki, Y. Uchino, T. Imamura, "Ozaki Scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique", arXiv:2504.08009
5. Y. Uchino, S. Ma, T. Imamura, K. Ozaki, T. Gutsche, "Emulation of Complex Matrix Multiplication based on the Chinese Remainder Theorem", arXiv:2512.08321 (theory behind the complex support)
6. Y. Uchino, K. Ozaki, T. Imamura, "Double-Precision Matrix Multiplication Emulation via Ozaki-II Scheme with FP8 Quantization", arXiv:2603.10634
7. H. Ootomo, K. Ozaki, R. Yokota, "DGEMM on integer matrix multiplication unit", Int. J. High Perform. Comput. Appl. (2024), doi:10.1177/10943420241239588 (the ozIMMU / Ozaki Scheme I predecessor)
8. H. Kawai et al., benchmarks of GPU-accelerated OpenMX: J. Phys. Soc. Jpn. 94, 124003 (2025), doi:10.7566/JPSJ.94.124003
9. NVIDIA cuSOLVER Documentation (cusolverDnXsyevd/Xsyevdx): `https://docs.nvidia.com/cuda/cusolver/`
10. NVIDIA cuBLAS Documentation: `https://docs.nvidia.com/cuda/cublas/`
11. NVIDIA HPC SDK Documentation (OpenACC): `https://docs.nvidia.com/hpc-sdk/`
12. ELPA (Eigenvalue soLvers for Petaflop Applications): `https://elpa.mpcdf.mpg.de/`
13. FFTW: `https://www.fftw.org/`

About this document: the 4th edition updates CUDA/HIP integration, CMake builds, shared ELPA, and the HIP CPU-cleanup fix against commit `c0689442` (2026-10-17). Its sources include `CMakeLists.txt`, `tools/prepare_build.py`, `docs/GPU_BACKENDS.md`, `source/backends/hip/`, and the validation records of `bdcba87d` and `57a3a3fc`. Detailed CUDA listings, source line numbers, and Appendices A–C retain the July 23 survey baseline (`9fe8ce13`, 91 commits); historical measurements are not current performance results. Treat the source code as the primary reference.